



Fachhochschul-Bachelorstudiengang
SOFTWARE ENGINEERING
A-4232 Hagenberg, Austria

Digitale Signalverarbeitung mit FAUST

Bachelorarbeit

zur Erlangung des akademischen Grades
Bachelor of Science in Engineering

Eingereicht von

Xianglei Wu

Begutachtet von FH-Prof. DI (FH) Dr. Erik Pitzer

Hagenberg, 05.08 2026

Erklärung

Ich erkläre eidesstattlich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen nicht benutzt und die den benutzten Quellen entnommenen Stellen als solche gekennzeichnet habe. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt.

Die vorliegende, gedruckte Bachelorarbeit ist identisch zu dem elektronisch übermittelten Textdokument.

05.08.2026
Datum

Unterschrift

Inhaltsverzeichnis

Abstract	v
Kurzfassung	vi
1 Einleitung	1
1.1 Forschungsfrage und Zielsetzung	2
1.2 Aufbau der Arbeit	3
2 Grundlagen	4
2.1 Digitale Audiosignale und blockweise Verarbeitung	4
2.1.1 Abtastung und diskrete Sample-Folgen	4
2.1.2 Audio-Blöcke und Echtzeitdeadline	5
2.2 Elementare DSP-Bausteine	6
2.2.1 Verstärkungsstufe	6
2.2.2 Rekursionsfilter (IIR-Filter) und Biquad-Struktur	7
2.2.3 FIR-Filter und Faltung	8
2.3 Frequenzanalyse mit diskreter Fourier-Transformation (DFT) und Fast-Fourier-Transformation (FFT)	9
2.3.1 Zeitbereich, Frequenzbereich und DFT	9
2.3.2 Fast-Fourier-Transformation	9
2.4 Performanzrelevante Implementierungsaspekte	10
2.4.1 Compileroptimierung und automatische Vektorisierung	10
2.4.2 Messgrößen	10
2.5 Zusammenfassung	11
3 Faust: Functional Audio Stream	12
3.1 Einführung	12
3.2 Programmmodell	13
3.3 Syntax: Blockdiagramm-Komposition	13
3.4 Semantik	14
3.5 Compiler-Architektur	15
3.6 Codegenerierung	16
3.6.1 Skalarer Modus	16
3.6.2 Vektor-Modus	17
3.6.3 Parallel-Modus	17
3.6.4 Generierte Code	17

3.7	Backends und Architekturdateien	17
3.8	Performance	18
3.9	Zusammenfassung	18
4	Versuchsdesign und Benchmark-Methodik	19
4.1	Definition der untersuchten DSP-Bausteine	19
4.1.1	Fast-Fourier-Transformation	25
4.2	Benchmark-Tools und Messgrößen	26
4.2.1	Messwerkzeuge	26
4.2.2	Messgrößen	27
4.3	Hardwareumgebung	28
4.4	Compilerkonfigurationen	29
4.4.1	Aufruf des Faust-Compilers	29
4.4.2	Flag-Matrix	29
4.5	Testdatengenerierung	30
4.6	Auswertungsverfahren	31
4.6.1	Kennzahlen und statistische Prüfung	31
4.6.2	Auflösungsgrenze des Messaufbaus	31
4.7	Übersicht der Messreihen	32
4.7.1	Voraussetzungen	32
4.7.2	Laufzeit und Compilerverhalten	33
4.7.3	Ressourcenverbrauch	33
4.7.4	Entwicklungsaufwand	34
4.7.5	Einordnung und Grenzen des Messplans	34
4.8	Zusammenfassung	34
5	Implementierung der Benchmarks	36
5.1	Umsetzung der DSP-Bausteine	37
5.1.1	Die Faust-Beschreibungen	37
5.1.2	Die manuellen C++-Implementierungen	38
5.1.3	Zustandshaltung und Speicherbedarf	40
5.2	Gemeinsame Test-Schnittstelle	40
5.3	Build-System	41
5.3.1	Aufruf des Faust-Compilers	41
5.3.2	Gegenprobe mit dem Faust-Vektor-Modus	42
5.4	Ablauf der Messkette	42
5.5	Sicherstellung einer fairen Vergleichsbasis	43
5.6	Verifikation der Ergebnisse	44
5.7	Verfügbarkeit des Quelltexts	45
5.8	Zusammenfassung	45
6	Evaluation der Messergebnisse	47
6.1	Gültigkeit der Messung	47
6.1.1	Numerische Übereinstimmung	47
6.1.2	Keine Speicheranforderungen im Audio-Pfad	47
6.1.3	Auflösungsgrenze des Messaufbaus	47
6.1.4	Stabilität und Signifikanz	48

6.2	Laufzeit der Bausteine	48
6.3	Skalierungsverhalten	49
6.3.1	Blockgröße	49
6.3.2	Filterlänge	50
6.3.3	Transformationslänge	51
6.4	Compilerkonfigurationen	51
6.5	Speicherbedarf und Binärgröße	53
6.6	Codeumfang	54
6.7	Übersetzungsdauer des Faust-Compilers	55
6.8	Zusammenfassung der Messergebnisse	56
7	Diskussion	57
7.1	Interpretation der Ergebnisse im Hinblick auf die Forschungsfrage	57
7.1.1	Der Laufzeitvergleich hängt an der Wahl der Referenz	57
7.1.2	Die Compilerkonfiguration wiegt schwerer als die Sprachwahl	58
7.1.3	Skalierungsverhalten	58
7.1.4	Speicherbedarf und Binärgröße	59
7.1.5	Codelänge und Implementierungskomplexität	59
7.1.6	Antworten auf die Teilfragen	60
7.1.7	Antwort auf die übergeordnete Forschungsfrage	60
7.2	Limitationen des Versuchsaufbaus	61
7.3	Bedrohung der Validität	62
7.3.1	Interne Validität	62
7.3.2	Konstruktvalidität	63
7.3.3	Externe Validität	64
8	Zusammenfassung und Ausblick	65
8.1	Zusammenfassung der zentralen Ergebnisse	65
8.2	Ausblick auf künftige Forschungsarbeiten	65
	Quellenverzeichnis	66
	Literatur	66
	Online-Quellen	67

Abstract

This should be a 1-page (maximum) summary of your work in English.

Kurzfassung

An dieser Stelle steht eine Zusammenfassung der Arbeit, Umfang max. 1 Seite. ...

Kapitel 1

Einleitung

Die digitale Signalverarbeitung (Digital Signal Processing, DSP) ist ein wichtiges Anwendungsfeld in der Informatik. Besonderen Stellenwert hat sie Sie findet unter anderem Anwendung in der Telekommunikation, Audiotechnik, Sprachverarbeitung und Medizintechnik (Ifeachor & Jervis, 2002). In diesen Anwendungsbereichen sind Laufzeiteffizienz, deterministische Latenz und numerische Präzision von besonderer Bedeutung.

DSP-Algorithmen werden traditionell in C oder C++ implementiert, unter anderem wegen des direkten Hardwarezugriffs und der stark optimierten Codeerzeugung (Levine & Skolnick, 1997).

Die manuelle Übertragung mathematischer Modelle in ausführbaren Code ist jedoch mit erheblichem Aufwand verbunden und erschwert die Portierung auf unterschiedliche Zielplattformen. In den letzten zwanzig Jahren wurden daher domänenspezifische Sprachen (Domain-Specific Languages, DSLs) für die Audio-Signalverarbeitung entwickelt, die durch Abstraktion und Code-Generierung diese Herausforderungen adressieren sollen.

parentcite[S. 2-6]orlarey:hal-02159014

Orlarey et al. (2009) beschreibt die domänenspezifische Sprache FAUST (Functional Audio Stream), die am GRAME in Lyon entwickelt wurde, als eine funktionale, blockbasierte Programmiersprache. Sie wurde speziell für Audio-DSP entwickelt. Programme bestehen aus einer algebraischen Kombination von Signalverarbeitungsoperatoren. Die aktuelle FAUST-Dokumentation beschreibt die Codegenerierung für mehrere Zielsprachen und Zielplattformen, darunter C++, Rust, JavaScript und WebAssembly (GRAME - Centre National de Création Musicale, 2025). Sie dokumentiert zudem Architekturdateien, mit denen aus einer FAUST-Quellbeschreibung unter anderem Audio-Plug-ins, Anwendungen für eingebettete Systeme und Webanwendungen erzeugt werden können (GRAME - Centre National de Création Musicale, 2025).

Die praktische Leistungsfähigkeit dieses Ansatzes wird am Beispiel des FAUST-STK untersucht. FAUST-STK ist eine Sammlung virtueller Musikinstrumente, deren Modelle überwiegend auf Wellenleitersynthese und teilweise auf modaler Synthese beruhen (Michon & Smith, 2011, S. 1). Die aus dem Synthesis ToolKit und SynthBuilder übernommenen Modelle wurden für die FAUST-Semantik angepasst und hinsichtlich ihrer Effizienz optimiert (Michon & Smith, 2011, S. 5–6).

Die Quelltextgrößenanalyse zeigt, dass die FAUST-Implementierungen bei den un-

tersuchten Modellen kompakter sind als die entsprechenden STK-C++- Implementierungen. Der ausgewiesene Größenvorteil beträgt je nach Modell zwischen 22,91 Prozent und 80,20 Prozent (Michon & Smith, 2011, Tab. 1, S. 6). Zusätzlich vergleicht die Studie die CPU-Last von Pure-Data-Plug-ins, die auf STK-C++-Code beziehungsweise FAUST-generiertem Code basieren (Michon & Smith, 2011, Tab. 2, S. 6).

Für die praktische Eignung ist es erforderlich, dass der durch FAUST generierte Code sowohl funktional korrekt als auch laufzeiteffizient ist. Scaringella et al. (2003) thematisieren die automatische Vektorisierung, während Letz et al. (2010) Möglichkeiten der Parallelisierung im FAUST-Umfeld behandeln. Die FAUST-Dokumentation verweist zudem auf Optimierungen, die den vollständigen Signalfluss berücksichtigen (Orlarey et al., 2009). Diese Eigenschaften lassen jedoch keine allgemeine Aussage darüber zu, ob FAUST-generierter C++-Code einer manuell implementierten C++-Referenz bei konkreten DSP-Bausteinen überlegen, gleichwertig oder unterlegen ist.

Direkte Vergleiche liegen für zeitbereichsbasierte Algorithmen vor. Für den Reverb-Algorithmus *Freeverb* berichten Orlarey et al. (2009) Benchmarkergebnisse, deren Ausprägung von Compiler und Codegenerierungsstrategie abhängt. Die in der vorliegenden Arbeit berücksichtigte Vergleichsliteratur untersucht überwiegend zeitbereichsbasierte Algorithmen wie Reverbs, Delays und physikalische Modelle. Ergänzend wird daher ein Vergleich FFT-bezogener Bausteine durchgeführt. Für einen fairen Vergleich folgen beide Implementierungen demselben algorithmischen Ablauf und werden über eine gemeinsame Schnittstelle in die Benchmark-Infrastruktur eingebunden. Auf explizit programmierte Single-Instruction-Multiple-Data-Intrinsics (SIMD-Intrinsics) wird verzichtet. Die automatische Vektorisierung des Compilers bleibt in beiden Varianten aktiv.

1.1 Forschungsfrage und Zielsetzung

Diese Arbeit untersucht das Verhalten von FAUST-generiertem C++-Code im Vergleich zu einer manuell implementierten C++-Referenz für grundlegende DSP-Bausteine. Berücksichtigt werden eine Gain-Stufe, ein Biquad-Filter, Finite-Impulse-Response-Filter (FIR-Filter) verschiedener Längen sowie FFTs unterschiedlicher Größen. Die Anforderungen umfassen reproduzierbare Vergleiche unter unterschiedlichen Datenabhängigkeiten und Skalierungseigenschaften. Neben der Laufzeit werden auch Speicherverbrauch und Binärgröße analysiert. Zusätzlich erfolgt eine Analyse der Codelänge und Implementierungskomplexität.

Die übergeordnete Forschungsfrage lautet:

Wie unterscheidet sich FAUST-generierter C++-Code von einer manuell implementierten C++-Referenz bei ausgewählten DSP-Bausteinen und FFTs hinsichtlich Laufzeit, Speicherverbrauch, Binärgröße, Codelänge und Implementierungskomplexität?

Zur Beantwortung dieser Forschungsfrage werden folgende Teilfragen untersucht:

- Wie groß ist der Performanzunterschied hinsichtlich der CPU-Zeit pro Audio-Block zwischen FAUST-generiertem und manuell implementiertem C++-Code bei Gain-Stufen, Biquad-Filtern, FIR-Filtern und FFTs?
- Wie verändert sich der Laufzeitunterschied zwischen beiden Implementierungen in Abhängigkeit von Blockgröße, FIR-Filterlänge und FFT-Größe?

- Wie unterscheiden sich FAUST-generierter und manuell implementierter C++-Code hinsichtlich des Speicherverbrauchs während der Ausführung der untersuchten DSP-Bausteine?
- Welche Unterschiede bestehen zwischen FAUST-generiertem und manuell implementiertem C++-Code hinsichtlich der Binärgröße der erzeugten Programme?
- Welchen Einfluss haben Compileroptimierungen und automatische Vektorisierung auf beide Implementierungen?
- Welche Unterschiede bestehen hinsichtlich Codelänge und Implementierungskomplexität der beiden Ansätze?

1.2 Aufbau der Arbeit

Die Bachelorarbeit gliedert sich in acht Kapitel. Sie behandelt die theoretischen Grundlagen, die Beschreibung von FAUST, das Versuchsdesign, die Implementierung der Benchmark-Kerne, die Evaluation und die Einordnung der Ergebnisse.

Nach dieser Einleitung vermittelt Kapitel 2 die für die Arbeit relevanten Grundlagen der digitalen Signalverarbeitung. Dazu gehören Audio-DSP, Verstärkungsstufen, rekursive Filter und FIR-Filter, die Fast-Fourier-Transformation sowie Anforderungen an Echtzeitverarbeitung.

Kapitel 3 stellt die domänenspezifische Sprache FAUST vor. Es erläutert die funktionale Syntax und Semantik, die zugrunde liegende Compiler-Architektur sowie die für die Arbeit relevanten Codegenerierungs-Backends.

Kapitel ?? beschreibt das Versuchsdesign und die Benchmark-Methodik. Es definiert die untersuchten DSP-Bausteine, die Messgrößen, die Zielplattformen, die Compilerkonfigurationen und die Maßnahmen zur Sicherung reproduzierbarer Messungen.

Kapitel 5 dokumentiert die praktische Umsetzung der FAUST- und C++-Varianten. Neben den DSP-Bausteinen werden die gemeinsame Schnittstelle, das Build-System und die Maßnahmen zur Gewährleistung einer fairen Vergleichsbasis beschrieben.

Kapitel 6 enthält die Evaluation. Die Ergebnisse der Mikrobenchmarks werden hinsichtlich Laufzeit, Speicherverbrauch und Binärgröße ausgewertet. Zusätzlich werden Skalierungseffekte durch unterschiedliche Blockgrößen, Filterlängen und FFT-Größen sowie der Einfluss von Compileroptimierungen analysiert. Im Anschluss wird die Erfahrung mit der Handhabung von FAUST bewertet.

Kapitel 7 diskutiert die Ergebnisse im Hinblick auf die Forschungsfrage. Dabei werden Limitationen des Versuchsaufbaus, Bedrohungen der Validität und sowie die Codelänge und Implementierungskomplexität analysiert.

Kapitel 8 fasst die zentralen Ergebnisse zusammen und gibt einen Ausblick auf mögliche Erweiterungen, etwa die Untersuchung vollständiger DSP-Pipelines oder weiterer Zielplattformen.

Kapitel 2

Grundlagen

Dieses Kapitel stellt die signaltheoretischen und laufzeitbezogenen Grundlagen vor. Sie sind nötig, um Faust-generierten mit manuell geschriebenem C++-Code zu vergleichen. Der Fokus liegt auf digitalen Audiosignalen, einer Verstärkungsstufe (Gain-Stufe), Biquad- und Finite-Impulse-Response-(FIR)-Filtern, der Fast-Fourier-Transformation (FFT) und den Anforderungen der blockbasierten Echtzeitverarbeitung.

2.1 Digitale Audiosignale und blockweise Verarbeitung

2.1.1 Abtastung und diskrete Sample-Folgen

Ein analoges Audiosignal ist zeitkontinuierlich. Für die digitale Verarbeitung wird es in regelmäßigen Zeitabständen abgetastet. Es wird als diskrete Folge von Sample-Werten dargestellt (Rossing, 2007). Jedes Sample hat einen festen zeitlichen Abstand zum vorherigen Sample (Rossing, 2007). Die Abtastfrequenz f_s bestimmt die Anzahl der Messungen pro Sekunde. Das Abtastintervall beträgt

$$T_s = \frac{1}{f_s}. \quad (2.1)$$

Die Beziehung zwischen Abtastfrequenz und Abtastintervall ergibt sich aus der Definition der Abtastfrequenz für digitale Audiosignale (Rossing, 2007, S. 520-522). Die digitale Darstellung eines kontinuierlichen Signals $x(t)$ lautet damit

$$x[n] = x(nT_s), \quad n \in \mathbb{Z}. \quad (2.2)$$

Diese zeitdiskrete Darstellung beschreibt die Umwandlung eines kontinuierlichen Signals in eine Folge von Abtastwerten, siehe (Rossing, 2007, S. 520–522). Das Nyquist-Shannon-Kriterium fordert, dass die Abtastfrequenz mindestens doppelt so hoch wie die höchste im Signal enthaltene Frequenz sein muss. Andernfalls können Frequenzanteile nicht eindeutig rekonstruiert werden (Tan & Jiang, 2018, Kap. 2.1, S. 19–26). Die Abtastung mit einer Rate oberhalb von $2f_B$ entspricht die Nyquist-Abtastung und behandelt die zugehörige Bandbegrenzung als Voraussetzung einer aliasfreien digitalen Darstellung (Zölzer, 2022, S. 63–65).

$$f_s \geq 2f_{\max}. \quad (2.3)$$

In der vorliegenden Arbeit werden Audiosignale als Folgen von Gleitkommawerten verarbeitet. Die Quantisierung ist daher nur als Übergang von der analogen in die digitale Darstellung relevant. Eine separate Untersuchung der Bittiefe oder der Quantisierungsqualität ist nicht Gegenstand der Benchmarks.

2.1.2 Audio-Blöcke und Echtzeitdeadline

Audiosysteme zur digitalen Signalverarbeitung (DSP) können Eingangssamples entweder samplebasiert oder in Blöcken verarbeiten, siehe (Tan & Jiang, 2018, S. 727). Blockbasierte Verfahren kommen vor allem bei der digitalen Filterung, Faltung, FFT und anderen echtzeitfähigen DSP-Anwendungen eingesetzt (Tan & Jiang, 2018, S. 727). Ein Block besteht aus N aufeinanderfolgenden Samples. Bei einer Abtastfrequenz f_s und einer Abtastperiode $T_s = 1/f_s$ besitzt ein Block die zeitliche Dauer

$$T_{\text{Block}} = NT_s = \frac{N}{f_s}. \quad (2.4)$$

Die Blockdauer hängt von der Anzahl der Samples und dem zeitlichen Abstand zwischen zwei aufeinanderfolgenden Samples ab. Bei der überlappenden Blockverarbeitung werden die Eingangsdaten in zeitlich versetzten Analysefenstern verarbeitet, wobei aufeinanderfolgende Blöcke mit N Samples einen gemeinsamen Bereich haben (Tan & Jiang, 2018, S. 499). Bei einer Überlappung von 50 Prozent besteht der zweite Block aus der zweiten Hälfte des ersten Blocks und einer neuen zweiten Hälfte (Tan & Jiang, 2018, S. 499). Die Rekonstruktion der überlappenden Ausgangsblöcke erfolgt anschließend mittels Overlap-Add (Tan & Jiang, 2018, S. 500). Für eine Echtzeitverarbeitung muss die Berechnung rechtzeitig vor dem Eintreffen des nächsten Datenabschnitts beziehungsweise dessen Ausgabe abgeschlossen sein. Für eine samplebasierte Echtzeitverarbeitung müssen die Analog-Digital-Umwandlung (ADC), die DSP-Verarbeitung und die Ausgabe innerhalb des zeitlichen Intervalls bis zum nächsten Ausgabezeitpunkt abgeschlossen werden (Tan & Jiang, 2018, S. 761). Für eine blockbasierte Verarbeitung lässt sich diese Echtzeitbedingung durch

$$T_{\text{exec}} \leq T_{\text{Deadline}} \quad (2.5)$$

ausdrücken. Dabei steht T_{exec} für die tatsächliche Ausführungszeit des Algorithmus. Die Deadline hängt davon ab, wie die Blockverarbeitung organisiert ist. Bei nicht überlappenden Blöcken entspricht die verfügbare Zeit der Blockdauer:

$$T_{\text{Deadline}} = T_{\text{Block}} = \frac{N}{f_s}. \quad (2.6)$$

Bei überlappender Verarbeitung startet für alle H Samples ein neuer Verarbeitungszyklus. Die Zeitspanne zwischen zwei aufeinanderfolgenden Zyklen entspricht deshalb der Hop-Zeit.

$$T_{\text{Deadline}} = T_{\text{Hop}} = \frac{H}{f_s}. \quad (2.7)$$

Die Hop-Zeit ist also die wichtigste Rechenzeit für eine überlappende, blockbasierte Verarbeitung in Echtzeit. Die Blockgröße und die Hop-Größe wirken sich jeweils auf verschiedene Eigenschaften des Systems aus. Eine kleinere Blockgröße senkt die Latenz, die durch das Puffern entsteht. Gleichzeitig verringert sie jedoch die verfügbare Zeit für

die Verarbeitung (Tan & Jiang, 2018, S. 499, 500, 761). Bei einer Überlappung von 50 Prozent gilt zum Beispiel

$$H = \frac{N}{2}. \quad (2.8)$$

Die Blockgröße und die Hop-Größe sind deshalb zentrale Parameter für die Konfiguration und Bewertung von Echtzeit-Benchmarks (Zölzer, 2022).

2.2 Elementare DSP-Bausteine

2.2.1 Verstärkungsstufe

Eine Verstärkungsstufe multipliziert jedes Eingangssignal mit einem konstanten Verstärkungsfaktor g . Für ein Eingangssignal $x[n]$ ergibt sich das Ausgangssignal $y[n]$ zu

$$y[n] = g \cdot x[n]. \quad (2.9)$$

Diese Skalierung ist eine grundlegende lineare Operation in der digitalen Audioverarbeitung (Smith, 2007). Der Baustein benötigt pro Sample eine Multiplikation und dient daher als einfacher Referenzfall zum Vergleich des von Faust erzeugten Codes mit der C++-Referenz. Bei einer Angabe der Verstärkung in Dezibel gilt

$$g = 10^{G_{\text{dB}}/20}. \quad (2.10)$$

Diese Umrechnung ergibt sich aus der Definition eines Amplitudenverhältnisses in Dezibel (Smith, 2007). Abbildung 2.1 zeigt eine Verstärkungsstufe mit $g = 0.5$. Das Ausgangssignal besitzt für jedes Sample die halbe Amplitude des Eingangssignals. Das entspricht einer Absenkung von ungefähr -6.02 dB. Die Verstärkungsstufe bildet einen ein-

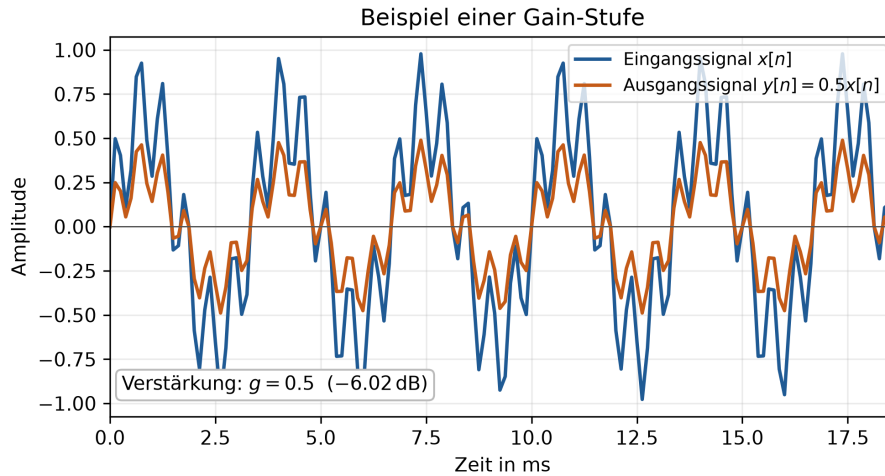


Abbildung 2.1: Beispiel einer Verstärkungsstufe mit $g = 0.5$. Die Amplitude jedes Eingangssamples wird halbiert. Eigene Darstellung nach (Smith, 2007).

fachen, vollständig datenparallelen Benchmark-Kern. Vektorisierung bezeichnet dabei

die Ausführung derselben Operation auf mehreren Datenwerten durch Single-Instruction-Multiple-Data-Instruktionen (SIMD-Instruktionen) (Scaringella et al., 2003). Eine sampleweise Skalierung weist daher ein hohes Vektorisierungspotenzial auf. Der Faust-Compiler kann solche Ausdrücke über seine Codegenerierungsstrategien in vektorisierbaren C++-Code übersetzen (Orlarey et al., 2009, S. 2–3, 15).

2.2.2 Rekursionsfilter (IIR-Filter) und Biquad-Struktur

Ein Rekursionsfilter, auch Infinite-Impulse-Response-Filter (IIR-Filter) genannt, benutzt neben aktuellen und vergangenen Eingangswerten auch vergangene Ausgangswerte. Die Rückkopplung führt dazu, dass der Filterzustand über mehrere Samples bestehen bleibt. Für die Untersuchung wird ein Biquad-Filter, also ein IIR-Filter zweiter Ordnung (Zölzer, 2022). Seine allgemeine Differenzengleichung lautet bei normiertem $a_0 = 1$

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] - a_1y[n-1] - a_2y[n-2]. \quad (2.11)$$

IIR-Filter lassen sich durch solche Differenzengleichungen beschreiben. Die Koeffizienten b_i bilden den Vorwärtzweig. Die Koeffizienten a_i bestimmen den Rückkopplungsweig (Smith, 2007). Abbildung 2.2 zeigt die Wirkung eines Biquad-IIR-Tiefpasses zweiter Ordnung mit einer Grenzfrequenz von 800 Hz. Wie beim FIR-Beispiel enthält das synthetische Eingangssignal Anteile bei 300 Hz und 1800 Hz. Der Frequenzgang und das Ausgangssignal veranschaulichen die Dämpfung hoher Frequenzanteile durch die rekursive Filterstruktur. Eine IIR-Implementierung muss die vergangenen Ein- und Ausgangs-

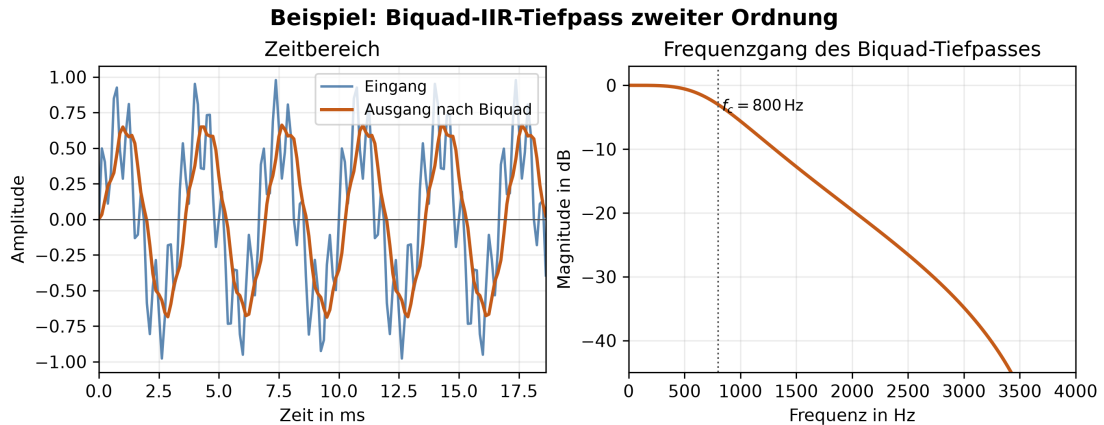


Abbildung 2.2: Beispiel eines Biquad-IIR-Tiefpasses zweiter Ordnung mit einer Grenzfrequenz von $f_c = 800$ Hz. Der Frequenzgang zeigt die Dämpfung hoher Frequenzanteile. Darstellung nach (Smith, 2007).

werte als Zustände speichern und bei jedem Sample aktualisieren. Für den Benchmark ist besonders wichtig, dass diese Rückkopplung Datenabhängigkeiten schafft und die Möglichkeit zur Vektorisierung im Vergleich zu nur vorwärts gerichteten Operationen einschränken kann. Deshalb unterscheidet man zwischen vektorisierbaren und skalarischen Ausdrücken. Rekursive Filter sind dabei ein wichtiger Sonderfall (Scaringella et al., 2003).

2.2.3 FIR-Filter und Faltung

Ein Finite-Impulse-Response-Filter (FIR-Filter) hat keine Rückkopplung. Jedes Ausgangssample wird aus einer gewichteten Summe des aktuellen und der vorherigen Eingangssamples berechnet. Für eine Filterlänge L gilt

$$y[n] = \sum_{k=0}^{L-1} b_k x[n-k]. \quad (2.12)$$

Diese Differenzengleichung entspricht der direkten Faltungsimplementierung eines FIR-Filters (Smith, 2007). Abbildung 2.3 zeigt einen FIR-Tiefpass mit 31 Filterkoeffizienten (Taps) und einer Grenzfrequenz von 800 Hz. Das synthetische Eingangssignal enthält einen niederfrequenten Anteil bei 300 Hz. Es enthält außerdem einen höherfrequenten Anteil bei 1800 Hz. Der FIR-Filter dämpft den höherfrequenten Anteil. Im Unterschied

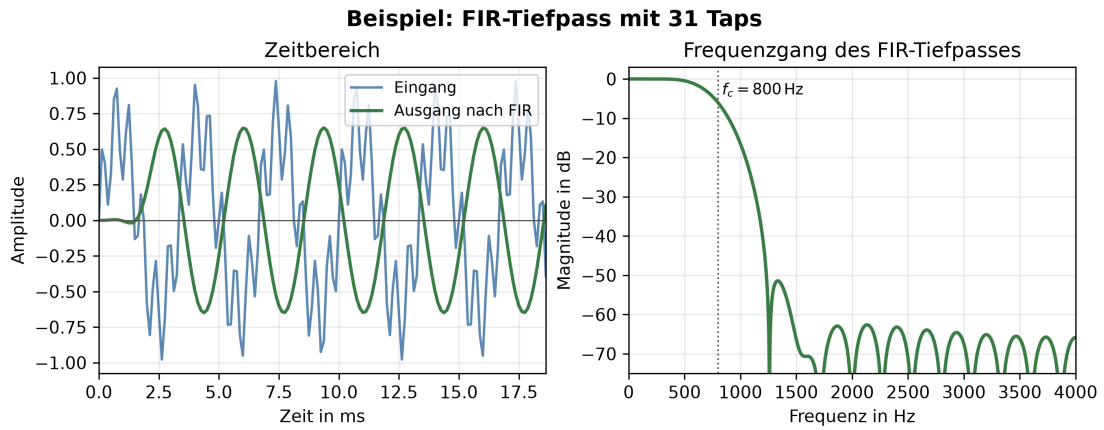


Abbildung 2.3: Beispiel eines FIR-Tiefpasses mit 31 Filterkoeffizienten und einer Grenzfrequenz von $f_c = 800$ Hz. Der Filter arbeitet ohne Rückkopplung und dämpft hohe Frequenzanteile. Darstellung nach (Smith, 2007).

zum Biquad hängt $y[n]$ nicht von früheren Ausgangswerten ab. Der Aufwand steigt mit der Zahl der Taps. Für jedes Ausgangssample liegt die asymptotische Komplexität daher

$$O(L). \quad (2.13)$$

Die lineare Abhängigkeit von der Filterlänge ergibt sich aus der direkten Auswertung der Faltungssumme (Smith, 2007). Die Filterlänge ist für die Evaluation wesentlich. Sie erhöht sowohl die Zahl der Rechenoperationen als auch die Größe des benötigten Zustands. Nichtrekursive Filterstrukturen können vollständig vektorisiert werden, sofern keine weiteren Datenabhängigkeiten entgegenstehen (Scaringella et al., 2003).

2.3 Frequenzanalyse mit diskreter Fourier-Transformation (DFT) und Fast-Fourier-Transformation (FFT)

2.3.1 Zeitbereich, Frequenzbereich und DFT

Im Zeitbereich zeigt ein digitales Signal seinen Amplitudenverlauf über dem Sample-Index. Im Frequenzbereich wird derselbe Signalabschnitt mit seinen spektralen Komponenten dar (Tan & Jiang, 2018, Kap. 4.1, 4.2, S. 97, 103). Die diskrete Fourier-Transformation (DFT) weist einer endlichen Folge $x[n]$ der Länge N komplexe Koeffizienten $X[k]$ zu.

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, \dots, N-1. \quad (2.14)$$

Die Bedeutung der Indizes n , k und N folgt unmittelbar aus der DFT-Definition (Tan & Jiang, 2018, S. 97). Der Betrag eines Koeffizienten beschreibt die Stärke der jeweiligen Frequenzkomponente, während sein Argument die zugehörige Phase angibt. Der Abstand zwischen benachbarten Frequenz-Bins beträgt

$$\Delta f = \frac{f_s}{N}. \quad (2.15)$$

Abbildung 2.4 vergleicht Zeitbereich und Frequenzbereich anhand eines Einzeltons und eines Mischsignals. Diese Frequenzauflösung ergibt sich aus der Abtastfrequenz und der Transformationslänge (Tan & Jiang, 2018, S. 101, 103). Eine größere Transformationslänge N verbessert die Frequenzauflösung, erhöht jedoch den Rechenaufwand.

2.3.2 Fast-Fourier-Transformation

Die Fast-Fourier-Transformation (FFT) berechnet dieselben DFT-Koeffizienten, aber mit einer effizienteren Zerlegung des Problems. Während eine direkte DFT eine Komplexität von

$$O(N^2) \quad (2.16)$$

aufweist, reduziert eine radix-2-FFT (Lyons et al., 2004) den Aufwand auf

$$O(N \log_2 N). \quad (2.17)$$

Diese Beschleunigung entsteht, weil die N -Punkt-DFT in kleinere Teiltransformationen zerlegt wird. Die asymptotischen Laufzeiten ergeben sich als Folge dieser Zerlegung (Tan & Jiang, 2018, S. 127–134). Die FFT-Größe ist im Benchmark ein wichtiger Skalierungsparameter. Sie beeinflusst sowohl den theoretischen Rechenaufwand als auch die praktische Speicher- und Cache-Nutzung (Tan & Jiang, 2018, S. 127-134). Dieses Kapitel behandelt nur die Vorwärtstransformation, da die Forschungsfrage einzelne FFT-Bausteine vergleicht. Eine vollständige Short-Time-Fourier-Transform-(STFT)-Pipeline oder eine Overlap-Add-Pipeline gehört nicht zu diesem Grundlagenkapitel.

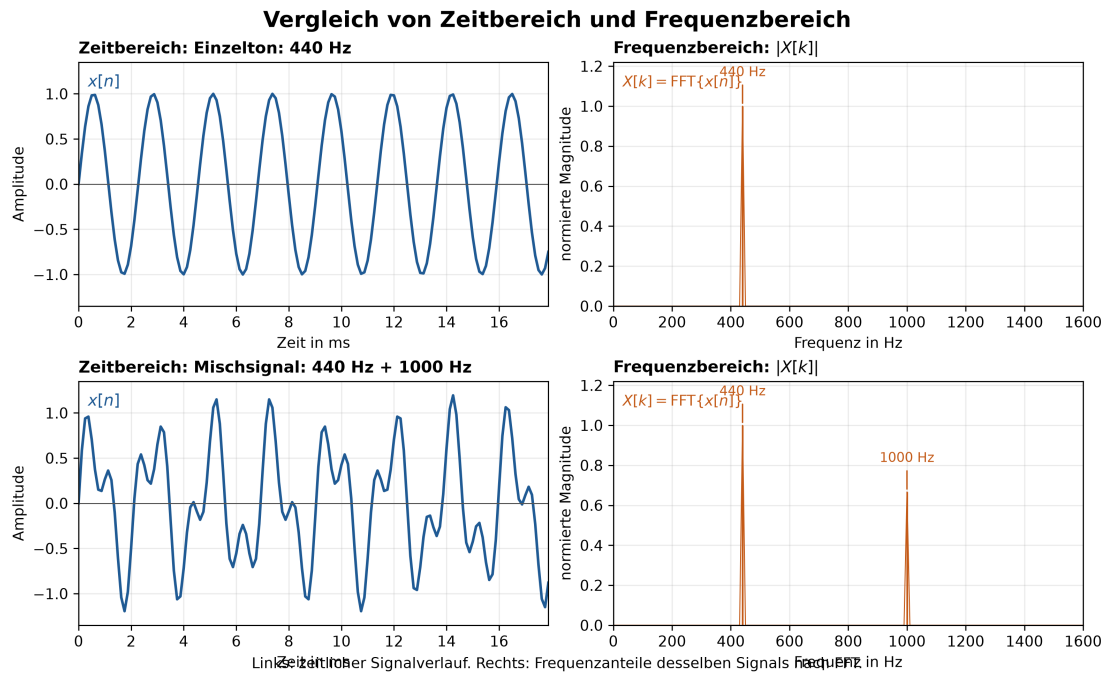


Abbildung 2.4: Vergleich von Zeitbereich und Frequenzbereich. Oben ist ein 440-Hz-Einzelton dargestellt, der im Spektrum einen dominanten Frequenzanteil bei 440 Hz aufweist. Unten ist ein Mischsignal aus 440 Hz und 1000 Hz dargestellt. Beide Anteile erscheinen im Frequenzbereich als separate Peaks. Darstellung basiert auf der DFT/FFT-Spektralanalyse (Tan & Jiang, 2018, S. 102).

2.4 Performanzrelevante Implementierungsaspekte

2.4.1 Compileroptimierung und automatische Vektorisierung

Die Laufzeit eines DSP-Kerns hängt nicht nur von seinem Algorithmus ab. Sie wird auch durch die Codegenerierung und die Optimierungen des Compilers beeinflusst. Bei automatischer Vektorisierung versucht der Compiler, mehrere unabhängige skalare Operationen mit SIMD-Instruktionen zusammenzufassen. Für Faust gibt es eine Analyse, die vektorisierbare Ausdrücke von solchen mit Datenabhängigkeiten trennt (Orlarey et al., 2009, S. 15). Diese Unterscheidung zeigt, warum Verstärkungsstufen und lange FIR-Strukturen andere Optimierungsmöglichkeiten haben können als rekursive Biquad-Filter. Die Arbeit vergleicht beide Implementierungen unter denselben Compiler- und Optimierungsbedingungen. Es kommen keine expliziten SIMD-Intrinsics zum Einsatz. So bleibt die Wirkung der automatischen Optimierung für Faust und C++ sichtbar. Die genaue Auswahl von Compiler, Flags und Zielplattform steht erst im Versuchsdesign.

2.4.2 Messgrößen

Die zentrale Messgröße ist die Prozessorzeit pro Audioblock. Ergänzend werden Speicherverbrauch und Binärgröße erfasst, da sie die praktische Einsetzbarkeit einer Implementierung neben der reinen Laufzeit beeinflussen. Codelänge und Implementierungs-

komplexität ergänzen den quantitativen Vergleich um Aspekte der Entwicklererfahrung. Die genaue Operationalisierung dieser Größen, die Zahl der Wiederholungen sowie die statistische Auswertung gehören in Kapitel 4.

2.5 Zusammenfassung

Die Grundlagen beschreiben die für die Benchmarks benötigte Verarbeitungskette von digitalen Samples über Gain-, IIR- und FIR-Operationen bis zur FFT. Blockgröße, Filterlänge und FFT-Größe bestimmen den Rechenaufwand und sind die zentralen Skalierungsparameter der Evaluation. Anschließend erläutern die folgenden Kapitel Faust und das konkrete Versuchsdesign.

Kapitel 3

Faust: Functional Audio Stream

3.1 Einführung

FAUST (Functional Audio Stream) ist eine domänenspezifische Programmiersprache für die Audio-Signalverarbeitung. Sie ist als effiziente Ergänzung zu C/C++ für Signalverarbeitungsbibliotheken, Audio-Plug-ins und eigenständige Anwendungen konzipiert (Orlarey et al., 2009).

Anders als visuelle Sprachen wie Max oder Pure Data ist FAUST eine kompilierte Sprache. Der FAUST-Compiler übersetzt FAUST-Programme direkt in C++-Code. Dieser Code ist lauffähig, selbstständig und benötigt keine externe Laufzeitumgebung. Er arbeitet deterministisch und mit konstantem Speicherverbrauch. Vgl. Orlarey et al., 2009, S. 2

Für die vorliegende Arbeit ist FAUST relevant, weil die Sprache Algorithmen der digitalen Signalverarbeitung (Digital Signal Processing, DSP) als funktionale Blockdiagramme beschreibt. Der Compiler erzeugt daraus C++-Code, der in dieselbe Benchmark-Infrastruktur wie eine manuell implementierte C++-Referenz eingebunden werden kann (Orlarey et al., 2009, S. 2–3). Der Vergleich betrifft die vom FAUST-Compiler abgeleitete und eine manuelle Implementierung desselben DSP-Algorithmus.

Die deklarative Blockdiagrammbeschreibung und die Trennung von Spezifikation und Implementierung können Codelänge und Implementierungskomplexität beeinflussen. Compileranalyse und Codegenerierung können sich zudem auf Laufzeit, Speicherverbrauch und Binärgröße auswirken. Ergebnisse aus FAUST-STK deuten für bestimmte physikalische Modelle auf kompaktere FAUST-Quelltexte hin (Michon & Smith, 2011, S. 5–6). Diese Ergebnisse sind nicht ohne Weiteres auf Gain-Stufen, Biquad- und FIR-Filter oder Fast-Fourier-Transformationen (FFTs) übertragbar. Die Überprüfung für diese DSP-Bausteine ist daher Gegenstand der vorliegenden Untersuchung.

Die Besonderheit von FAUST liegt in der formalen Semantik: Jedes Programm beschreibt eine mathematische Funktion, die Eingangssignale in Ausgangssignale transformiert. Der Compiler übersetzt nicht den Programmtext, sondern die dahinterstehende Funktion. Dadurch können zwei syntaktisch unterschiedliche, aber semantisch äquivalente Programme denselben Code erzeugen. (Orlarey et al., 2009)

Programm 3.1: Minimale FAUST-Signalprozessoren

```

1  process = 0;          // erzeugt Stille
2  process = _;          // kopiert Eingang auf Ausgang
3  process = +;          // addiert zwei Kanäle (Stereo zu Mono)
4

```

3.2 Programmmodell

FAUST kombiniert funktionale Programmierung mit Blockdiagrammen. Digitale Signale werden als diskrete Funktionen der Zeit modelliert. Signalprozessoren verarbeiten diese Signale, und Kompositionsoperatoren verbinden mehrere Signalprozessoren zu größeren Blockdiagrammen (Orlarey et al., 2009, S. 3–4).

Ein FAUST-Programm beschreibt keinen Klang, sondern einen Signalprozessor, also eine Maschine mit Eingängen und Ausgängen. Das Schlüsselwort **process** definiert diesen Signalprozessor. Programm 3.1 zeigt drei minimale Definitionen.

FAUST-Primitive funktionieren wie C-Funktionen, aber auf Signalen. Beispielsweise berechnet **sin** den Sinus jedes Samples eines Eingangssignals. Der Verzögerungsoperator **@** bildet ein Signal mit einer zeitvariablen Verzögerung ab (Orlarey et al., 2009).

3.3 Syntax: Blockdiagramm-Komposition

Tabelle 3.1 fasst die fünf Kompositionsoperatoren der FAUST-Blockdiagrammalgebra zusammen (Orlarey et al., 2009, S. 5).

Tabelle 3.1: Kompositionsoperatoren der Faust Block-Diagramm-Algebra

Operator	Bezeichnung	Beschreibung
$f \sim g$	Rekursive Komposition	Feedback-Schleife mit 1-Sample-Delay
f, g	Parallele Komposition	Zwei Signalprozessoren nebeneinander
$f : g$	Sequenzielle Komposition	Ausgang von f mit Eingang von g verbinden
$f < : g$	Split	Ein Ausgang auf mehrere Eingänge verteilen
$f > : g$	Merge	Mehrere Ausgänge auf einen Eingang zusammenführen

Abbildung 3.1 stellt dieselben Operatoren als Blockdiagramme dar.

Beispiel für sequenzielle Komposition: **process = + : abs;** verbindet die Ausgabe einer Addition mit einem Absolutwert. (Orlarey et al., 2009, S. 4)

Beispiel für rekursive Komposition: **process = + _;** erzeugt einen Integrator mit einer impliziten Verzögerung um ein Sample $Y(t) = X(t) + Y(t - 1)$. (Orlarey et al., 2009, S. 4)

Programm 3.2 zeigt einen Rauschgenerator mit rekursiver Pseudozufallszahlenerzeugung und einem Regler für die Signalamplitude (Orlarey et al., 2009, S. 5)

Die rekursive Definition erzeugt eine Pseudozufallsfolge. Die nachfolgenden Definitionen skalieren sie in den Bereich von minus eins bis eins und verknüpfen sie mit einem

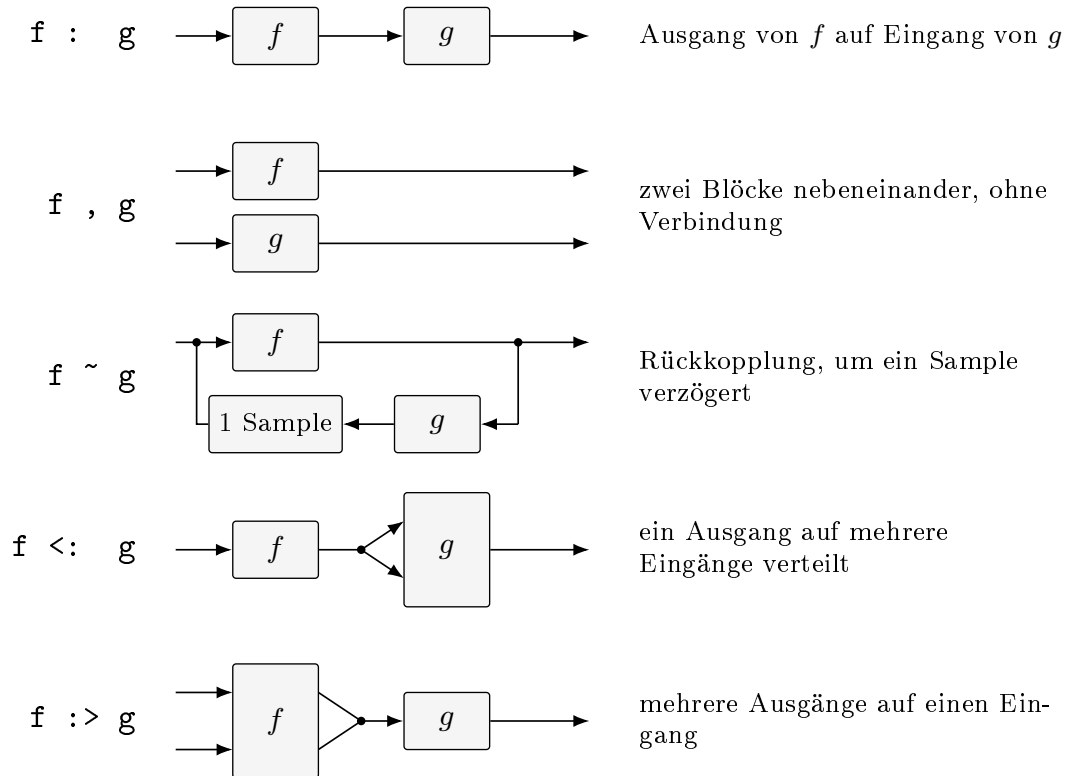


Abbildung 3.1: Die fünf Kompositionsoperatoren der Faust-Blockdiagramm

Programm 3.2: Rauschgenerator mit Lautstärkeregler in FAUST

```

1  random = +(12345) ~ *(1103515245);
2  noise = random / 2147483647.0;
3  process = noise * vslider("noise[style:knob]", 0, 0, 100, 0.1) / 100;
4

```

interaktiven Lautstärkeregler, um den resultierenden Signal zu steuern (Orlarey et al., 2009, S. 6).

3.4 Semantik

FAUST besitzt eine denotationale Semantik. Ein Programm beschreibt nicht eine Folge imperativer Einzelschritte, sondern die mathematische Bedeutung eines Signalprozessors. Digitale Signale werden dabei als diskrete Funktionen der Zeit aufgefasst. Ein Signalprozessor ist entsprechend eine Funktion, die Eingangs- in Ausgangssignale überführt (Orlarey et al., 2009, S. 2–4).

Diese Sichtweise trennt die Spezifikation eines DSP-Algorithmus von dessen konkreter Implementierung. Die FAUST-Quellbeschreibung legt fest, welche Signaltransforma-

Programm 3.3: Semantisch äquivalente FAUST-Beschreibungen

```

1    process = /(2) : @(10);
2    process = *(2) : @(7) : /(4) : @(3);
3

```

tion berechnet werden soll. Der Compiler entscheidet anschließend, wie sie effizient in C++ umgesetzt wird. Semantisch äquivalente Blockdiagramme können nach der Normalisierung dieselbe Implementierung erzeugen (Orlarey et al., 2009, S. 3, S. 12).

Für die vorliegende Arbeit ist diese Trennung wesentlich. Die FAUST-Variante und die manuelle C++-Referenz können unterschiedlich strukturiert sein. Beide müssen jedoch denselben Algorithmus und dieselbe Ein-/Ausgabesemantik realisieren. Der Vergleich bewertet daher die Qualität der abgeleiteten Implementierung und nicht lediglich Unterschiede in der Oberflächensyntax.

3.5 Compiler-Architektur

Der FAUST-Compiler verarbeitet eine Quellbeschreibung in mehreren aufeinander aufbauenden Analysephasen. Diese Phasen trennen die sprachliche Auswertung, die Ermittlung der mathematischen Bedeutung und die Optimierungsvorbereitung von der anschließenden C++-Codegenerierung (Orlarey et al., 2009, S. 10–13).

1. **Einlesen und Parsen der Quelldateien.** Der Compiler liest die Hauptdatei sowie über `import`-Direktiven eingebundene Dateien rekursiv ein. Das Ergebnis ist eine Menge benannter Definitionen. (Orlarey et al., 2009, S. 10–11).
2. **Auswertung der Blockdiagrammdefinitionen.** Algorithmische Definitionen werden ausgewertet und in eine explizite Blockdiagrammbeschreibung in Normalform überführt. Für die weitere Analyse liegen damit die primitiven Signaloperationen und ihre Verschaltung unmittelbar vor (Orlarey et al., 2009, S. 11–12).
3. **Symbolische Semantikanalyse und Normalisierung.** Der Compiler propagiert symbolische Eingangssignale durch das Blockdiagramm. Daraus leitet er für jedes Ausgangssignal eine mathematische Gleichung ab. Anschließend normalisiert er diese Gleichungen, sodass semantisch äquivalente Blockdiagramme dieselbe Repräsentation erhalten. Aufeinanderfolgende Verzögerungen können zusammengefasst werden. Konstante Faktoren können so angeordnet werden, dass Verzögerungsleitungen wiederverwendbar sind (Orlarey et al., 2009, S. 12).

Programm 3.3 zeigt zwei syntaktisch unterschiedliche Programme mit derselben Signalgleichung.

Beide Beschreibungen ergeben nach der Normalisierung dieselbe Ausgangsgleichung. Sie können deshalb auf dieselbe optimierte Implementierung abgebildet werden (Orlarey et al., 2009, S. 12).

4. **Typ- und Bereichsanalyse.** Für jede resultierende Signalgleichung ermittelt der Compiler den numerischen Datentyp, den möglichen Wertebereich, den Berechnungszeitpunkt, die Ausführungsgeschwindigkeit und die Parallelisierbarkeit. Konstante Signale können einmalig berechnet werden. Bedienelemente können block-

Programm 3.4: Zwischenspeicherung eines gemeinsam verwendeten Teilausdrucks

```

1  // Stereo zu Mono: kein Caching nötig
2  process = +;
3  → output0[i] = input0[i] + input1[i];
4
5  // Split: Ergebnis wird zwischengespeichert
6  process = + <: ,;
7  → float fTemp0 = input0[i] + input1[i];
8  output0[i] = fTemp0;
9  output1[i] = fTemp0;
10

```

weise und Audiosignale sampleweise verarbeitet werden. Diese Unterscheidungen sind Implementierungsdetails; aus Sicht der FAUST-Programmierung bleiben alle Werte reguläre Audiosignale (Orlarey et al., 2009, S. 12–13).

5. **Vorkommenanalyse.** Der Compiler untersucht, in welchen Kontexten Teilausdrücke auftreten und ob sie gemeinsam verwendet werden. Teure gemeinsame Teilausdrücke können in temporären Variablen zwischengespeichert werden. Dies gilt auch für Ausdrücke, die zwischen unterschiedlichen Geschwindigkeitskontexten benötigt werden. Nicht jeder mehrfach auftretende Teilausdruck wird jedoch automatisch ausgelagert (Orlarey et al., 2009, S. 13).

Erst nach diesen Phasen beginnt die Codegenerierung. Der Compiler übersetzt somit nicht die ursprüngliche Schreibweise des Programms, sondern eine analysierte und normalisierte Repräsentation seiner Signalverarbeitung (Orlarey et al., 2009, S. 13).

Erst nach diesen Phasen beginnt die Codegenerierung. Der Compiler übersetzt somit nicht die ursprüngliche Schreibweise des Programms, sondern eine analysierte und normalisierte Repräsentation seiner Signalverarbeitung (Orlarey et al., 2009, S. 13).

3.6 Codegenerierung

Auf Basis der analysierten Signalgleichungen erzeugt der FAUST-Compiler C++-Code. Die skalare Variante bildet die Grundlage. Der Vektor-Modus strukturiert diesen Code für die automatische Vektorisierung um. Der Parallel-Modus ergänzt den Vektor-Code um Direktiven für die nebenläufige Ausführung (Orlarey et al., 2009, S. 13–21).

3.6.1 Skalarer Modus

Im skalaren Modus berechnet eine Schleife die Ausgänge sampleweise. Für jede Addition $E_1 + E_2$ wird der Code von E_1 und E_2 generiert und mit einem $+$ verbunden. Wird das Ergebnis mehrfach verwendet, wird eine Zwischenspeicher-Variable angelegt, um redundante Berechnungen zu vermeiden (Orlarey et al., 2009, S. 13–14).

Programm 3.4 veranschaulicht den Unterschied zwischen einer direkten Berechnung und einer zwischengespeicherten Berechnung.

3.6.2 Vektor-Modus

Der Vektor-Modus erzeugt mehrere einfachere Schleifen, die über Vektoren kommunizieren. Dadurch erhält der nachgelagerte C++-Compiler eine Codeform, die die automatische Vektorisierung erleichtert. Gemeinsam verwendete, rekursive oder für Verzögerungsleitungen benötigte Ausdrücke können in eigenen Schleifen berechnet werden. Das Ergebnis ist ein gerichteter azyklischer Graph aus Berechnungsschleifen (Orlarey et al., 2009, S. 15–17).

Die automatische Vektorisierung unterscheidet vektorisierbare von nicht vektorisierbaren Ausdrücken anhand von Typ- und Kontextinformationen. Die dadurch erzielbare Beschleunigung hängt von Algorithmus, Compiler und Zielarchitektur ab und darf daher nicht als allgemeiner Leistungsgewinn interpretiert werden (Scaringella et al., 2003).

3.6.3 Parallel-Modus

Der Parallel-Modus baut auf dem Vektor-Modus auf und ergänzt den erzeugten C++-Code um Open Multi-Processing (OpenMP)-Direktiven. Unabhängige Schleifen desselben Ausführungsniveaus können parallel ausgeführt werden. Rekursive Abhängigkeiten begrenzen diese Parallelisierung (Orlarey et al., 2009, S. 17–21).

Spätere Arbeiten untersuchen zusätzlich dynamische Scheduling-Verfahren für den Schleifengraphen. Diese Verfahren ändern jedoch nichts daran, dass nutzbarer Parallelismus und der Aufwand der Synchronisierung von der Struktur des Signalprozessors abhängen (Letz et al., 2010).

3.6.4 Generierte Code

Der erzeugte C++-Code wird als Klasse ausgegeben. Zu den zentralen Methoden zählen die Abfrage der Ein- und Ausgänge, die Initialisierung, die Beschreibung der Benutzeroberfläche und die eigentliche Signalverarbeitung (Orlarey et al., 2009, S. 6–7).

- `getNumInputs()` / `getNumOutputs()` - Anzahl Ein-/Ausgänge
- `init(int samplingFreq)` - Initialisierung
- `buildUserInterface(UI*)` - GUI-Beschreibung
- `compute(int count, float** input, float** output)` - Signalverarbeitungsmethode

3.7 Backends und Architekturdateien

Architekturdateien trennen die Signalverarbeitungslogik von der Einbettung in ein Audio- oder Benutzeroberflächensystem. Sie umschließen die generierte C++-Klasse. Dadurch kann dieselbe FAUST-Quellbeschreibung für verschiedene Anwendungen und Plug-in-Formate verwendet werden (Orlarey et al., 2009, S. 8).

Die in diesem Abschnitt dargestellte Architekturauswahl stammt aus einer älteren FAUST-Version. Für aktuelle Zielsprachen, Zielplattformen und Architekturdateien ist deshalb die offizielle FAUST-Dokumentation maßgeblich (GRAME - Centre National de Création Musicale, 2025).

3.8 Performance

Die in der Grundquelle dargestellten Benchmarks vergleichen mehrere Testprogramme auf bestimmten Hardware- und Compilerkonfigurationen. Sie zeigen, dass der Nutzen von Vektorisierung und Parallelisierung von der Struktur des Signalprozessors, dem verfügbaren Parallelismus und der Speicherbandbreite abhängt (Orlarey et al., 2009, S. 22–28).

Die Messungen wurden mit einer historischen FAUST-Version sowie den damals verfügbaren Compilern und Testsystemen durchgeführt. Sie dienen deshalb der Einordnung der Compilerstrategien, erlauben jedoch keine Vorhersage für die in dieser Arbeit verwendete Hardware, Compilerkonfiguration oder Benchmark-Infrastruktur (Orlarey et al., 2009, S. 22–23).

- **Speicherbandbreitenbegrenzte Programme.** Im Benchmark `copy1` waren die skalare und die vektorisierte Variante ähnlich schnell. Die parallele Variante schnitt wegen der geteilten Speicherbandbreite schlechter ab (Orlarey et al., 2009, S. 22–23).
- **Programme mit begrenztem Parallelismus.** Für die Freeverb-Implementierung berichten die Autoren bei Verwendung des Intel-C++-Compilers auf zwei der drei Testsysteme moderate Zugewinne durch Vektorisierung und Parallelisierung (Orlarey et al., 2009, S. 24).
- **Programme mit hohem Parallelismus.** Für `karplus32` und `rms8` wurden mit dem Intel-C++-Compiler deutliche Leistungssteigerungen durch Vektorisierung und Parallelisierung beobachtet. Die Höhe des Gewinns blieb jedoch von Hardware und Speicherbandbreite abhängig (Orlarey et al., 2009, S. 24, S. 27–28)

3.9 Zusammenfassung

FAUST ist eine kompilierte Sprache für Audio-Signalverarbeitung mit formaler Semantik. Die Sprache trennt die Beschreibung eines Signalprozessors von dessen Implementierung und ermöglicht dem Compiler, daraus eigenständigen C++-Code zu erzeugen (Orlarey et al., 2009). Für die vorliegende Arbeit ist insbesondere relevant, dass dieselben DSP-Algorithmen sowohl als FAUST-Programm als auch als manuelle C++-Referenz implementiert und unter einer gemeinsamen Benchmark-Infrastruktur verglichen werden könne

Kapitel 4

Versuchsdesign und Benchmark-Methodik

Dieses Kapitel beschreibt, wie der Vergleich zwischen Faust-generiertem und manuell implementiertem C++-Code durchgeführt wird. Es legt fest, welche DSP-Bausteine untersucht werden, mit welchen Werkzeugen gemessen wird, auf welcher Plattform die Messungen durchgeführt werden, welche Compilerkonfigurationen verwendet werden und welche Testdaten zum Einsatz kommen. Der Versuchsaufbau soll wiederholbar sein und für beide Implementierungen dieselben Bedingungen gewährleisten. Die konkrete Umsetzung im Quelltext beschreibt Kapitel 5, die Messergebnisse folgen in Kapitel 6.

4.1 Definition der untersuchten DSP-Bausteine

Drei Kriterien helfen bei der Auswahl der Bausteine.

1. **Unterschiedliche Datenabhängigkeiten.** Die Bausteine sollen sich darin unterscheiden, ob und über wie viele Samples ein Ergebnis von früheren Ergebnissen abhängt. Diese Abhängigkeiten bestimmen, wie der Compiler optimieren kann (Scaringella et al., 2003).
2. **Ein Skalierungsparameter je Baustein.** Jeder Baustein soll einen Parameter besitzen, über den sich der Rechenaufwand gezielt verändern lässt. Etwa die Blockgröße, die Filterlänge oder die Transformationslänge. Erst dadurch lässt sich beobachten, wie sich der Unterschied zwischen beiden Ansätzen mit dem Aufwand verändert.
3. **Nachweisbare Gleichwertigkeit.** Jeder Baustein muss klein genug sein, sodass es sich zeigen lässt, dass beide Implementierungen dasselbe berechnen.

Tabelle 4.1 listet Baustein, Aufwand, Datenabhängigkeit und Skalierung auf.

Die ersten vier Bausteine werden mit den Blockgrößen 64, 128, 256, 512 und 1024 Samples gemessen. Diese Größen entsprechen üblichen Puffergrößen in Audioanwendungen. Die FFT bildet eine Ausnahme. Bei ihr ist die Blockgröße an die Transformationslänge gebunden, weshalb nur die drei Längen 64, 128 und 256 gemessen werden. Der Grund für diese Begrenzung wird in Abschnitt 4.1.1 erläutert.

Der Akkumulator ist gegenüber der Aufzählung in Abschnitt 4.1 hinzugekommen. Er wird ergänzt, weil der Biquad-Filter Rückkopplung und Koeffizientenrechnung miteinander vermischt. Der Akkumulator trennt beides und misst die Rückkopplung allein.

Tabelle 4.1: Untersuchte DSP-Bausteine und ihre Eigenschaften

Baustein	Aufwand	Datenabhängigkeit	Skalierung
Gain	1 Multiplikation	keine	Blockgröße 64–1024
Akkumulator	1 Addition	Rückkopplung (1 Sample)	Blockgröße 64–1024
Biquad-Filter	5 Mult., 4 Add.	Rückkopplung (2 Samples)	Blockgröße 64–1024
FIR-Filter	L Mult., L Add.	keine	Filterlänge $L \in \{32, 64, 256\}$
FFT	$\mathcal{O}(N \log_2 N)$	Butterfly-Graph	FFT-Größe $N \in \{64, 128, 256\}$

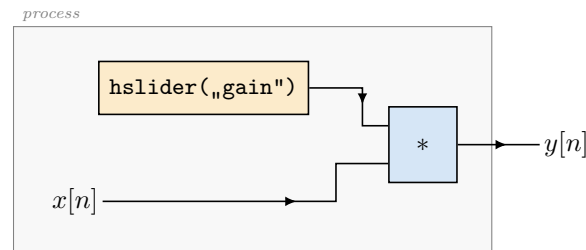


Abbildung 4.1: Verstärkungsstufe (`gain.dsp`). Beide Eingänge der Multiplikation liegen auf der linken Blockkante: das Audiosignal und das Bedienelement. Der Verstärkungsfaktor ist damit ein Signal und keine Übersetzungszeitkonstante; er wird je Aufruf von `compute()` aus dem Objekt gelesen. Zwischen aufeinanderfolgenden Samples besteht keine Abhängigkeit.

Verstärkungsstufe (Gain)

Die Gain-Stufe skaliert jedes Sample mit einem konstanten Faktor $g = 0,5$ gemäß Gleichung 2.9. In Faust wird sie als Multiplikation mit einem Bedienelement beschrieben. Der Vorgabewert beträgt 0,5. Es gibt weder einen Zustand noch eine Datenabhängigkeit zwischen den Samples. Jedes Ausgangssample lässt sich einzeln berechnen. Dadurch sind die Daten komplett parallel.

Der Baustein zeigt zunächst den einfachsten Fall für den Laufzeitvergleich. Bei nur einer Multiplikation pro Sample sollten beide Implementierungen fast denselben Maschinencode erzeugen. Außerdem dient er als Bezugspunkt für die Auflösungsgrenze des Messaufbaus (Abschnitt 4.6). Wenn hier ein Unterschied auftritt, sagt das zuerst etwas über die Messung aus. Erst danach betrifft es Faust. Weil die Operation komplett vektorisierbar ist, kann damit geprüft werden, ob die Codegenerierung die automatische Vektorisierung des C++-Compilers stört.

Die Aussagekraft ist begrenzt. Ein Baustein mit einer Operation je Sample ist nicht rechenintensiv, sondern speicherzugriffsbegrenzt. Gemessen wird hier vor allem, wie schnell Werte geladen, geschrieben werden und wie gut gerechnet wird. Zur Beantwortung der Frage nach der Qualität der Codegenerierung ist die Verstärkungsstufe der schwächste Baustein.

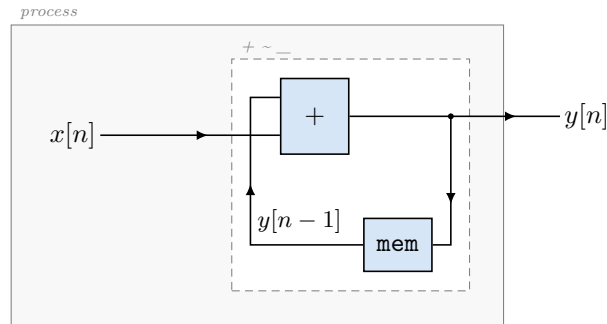


Abbildung 4.2: Akkumulator (`accum.dsp`, `process = + ~ _;`). Der innere Rahmen ist die rekursive Komposition. Wie in der Faust-Darstellung üblich ist der rückgekoppelte Teil gespiegelt gezeichnet: Sein Eingang liegt rechts, sein Ausgang links, sodass der Draht die Schleife schließt. Faust fügt die Verzögerung um ein Sample implizit ein; sie ist hier als `mem` sichtbar gemacht. Die Rückkopplung ist der einzige Inhalt des Bausteins, Koeffizienten kommen nicht vor.

Akkumulator (Accum)

Der Akkumulator berechnet

$$y[n] = x[n] + y[n - 1]. \quad (4.1)$$

Jedes Ausgangssample ist die Summe aller bisherigen Eingangssamples. In Faust entspricht das dem Rekursionsoperator aus Tabelle 3.1 in seiner kürzesten Form, also einer Addition mit Rückführung über ein Sample.

Abbildung 4.2 zeigt das zugehörige Faust-Blockdiagramm.

Der Baustein enthält weder Filterkoeffizienten noch weitere Logik und isoliert damit ausschließlich die Kosten der Rückkopplung. Er beantwortet keine eigenständige Frage, sondern trennt zwei mögliche Erklärungen. Zeigt sich beim Biquad-Filter ein Laufzeitunterschied zwischen Faust und C++, lässt sich am Akkumulator prüfen, ob dieser aus der Rückkopplung selbst oder aus der Art der Koeffizientenverrechnung stammt. Ohne diesen Vergleichsfall bliebe eine solche Beobachtung unentscheidbar.

Als Audio-Baustein ist der Akkumulator hingegen ohne praktischen Nutzen. Seine Ausgabe wächst unbegrenzt, und der aufsummierte Rundungsfehler nimmt mit der Blocklänge zu. Er dient rein als Diagnosefall und nicht als Anwendungsfall.

Biquad-Filter

Der Biquad-Filter ist ein Tiefpass zweiter Ordnung mit einer Grenzfrequenz von 800 Hz bei einer Abtastfrequenz von 48 kHz. Er verwendet die Koeffizienten $b_0 = b_2 = 0,00255054$, $b_1 = 0,00510107$, $a_1 = -1,85214649$ und $a_2 = 0,86234863$. Es wird darauf geachtet, dass die Gleichverstärkung 1 beträgt. Beide Implementierungen verwenden dieselben Koeffizienten und realisieren dieselbe Übertragungsfunktion nach Gleichung 2.11. Jedes Ausgangssample hängt von den beiden vorherigen Ein- und Ausgangswerten ab.

Abbildung 4.3 zeigt das zugehörige Faust-Blockdiagramm.

Der Baustein ist der aussagekräftigste der fünf, weil er zu drei Fragen zugleich beiträgt. Zur Laufzeit liefert er den Fall, in dem die Datenabhängigkeit die Optimierung

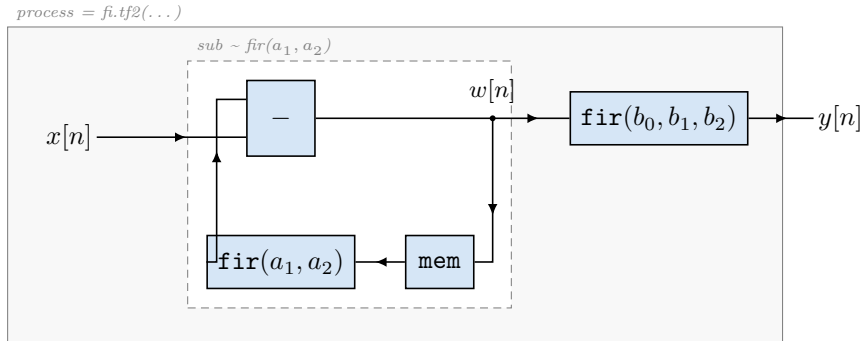


Abbildung 4.3: Biquad-Filter (`biquad.dsp`, `fi.tf2`). Die Bibliotheksfunktion ist eine sequenzielle Komposition aus einem Rekursionsblock und einem FIR-Block: `sub ~ fir(a1,a2) : fir(b0,b1,b2)`. Beide greifen auf dasselbe Zwischensignal w zu, weshalb nur eine Verzögerungskette entsteht – das ist die Direktform II. Die Direktform I der manuellen Implementierung (Abb. 4.4) ließe sich so nicht ausdrücken, weil ihr Vorwärtzweig über das Eingangssignal und ihr Rückkopplungszweig über das Ausgangssignal läuft und sie deshalb zwei getrennte Ketten benötigt.

tatsächlich einschränkt, weil das Umordnen von Gleitkommaoperationen kann die Länge der Abhängigkeitskette verändern (Abschnitt 4.4). Zur numerischen Genauigkeit liefert er den einzigen Fall, in dem sich Rundungsfehler über die Rückkopplung fortpflanzen statt sich nur zu addieren.

Die manuelle Implementierung nutzt die Direktform I. Sie überträgt Gleichung 2.11 direkt. Jedes Ausgangssample wird direkt aus den beiden vorherigen Eingangs- und den beiden vorherigen Ausgangswerten berechnet. Der Filter speichert deshalb vier Werte.

Faust rechnet in zwei Schritten. Zuerst entsteht aus dem Eingang und zwei zurückgeführten Werten ein Zwischensignal w . Aus diesem wird dann das Ausgangssignal gebildet.

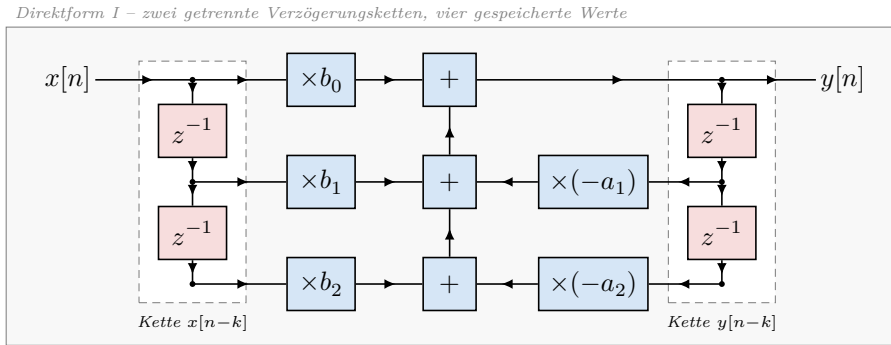
$$w[n] = x[n] - a_1 w[n-1] - a_2 w[n-2], \quad (4.2)$$

$$y[n] = b_0 w[n] + b_1 w[n-1] + b_2 w[n-2]. \quad (4.3)$$

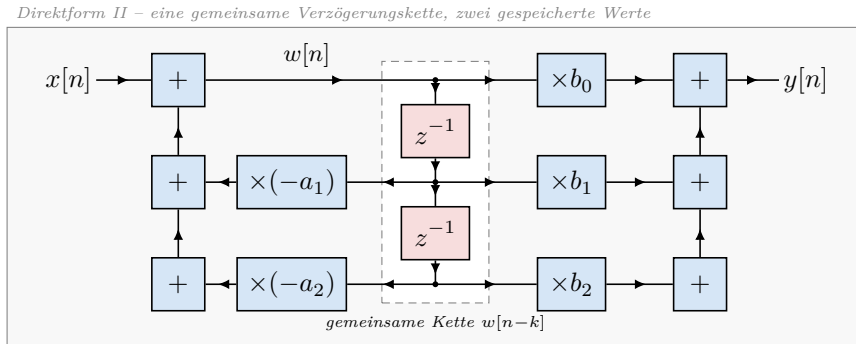
Das ist die Direktform II. Sie speichert zwei Werte, weil beide Schritte die gleiche Verzögerungskette benutzen.

Für Faust ist diese Form die naheliegendste. Der Rekursionsoperator aus Tabelle 3.1 beschreibt genau den ersten Schritt. Der zweite Schritt ist ein gewöhnlicher FIR-Filter. Die Bibliotheksfunktion `fi.tf2` besteht deshalb aus diesen beiden Teilen. Die Direktform I lässt sich so nicht ausdrücken. Der Vorwärtzweig läuft über das Eingangssignal. Der Rückkopplungszweig läuft über das Ausgangssignal. Es wären also zwei getrennte Verzögerungsketten nötig.

Der Unterschied ist beabsichtigt. Jede Seite nutzt die Formulierung, die für sie am naheliegendsten ist. Genau dieser Unterschied soll durch den Vergleich sichtbar werden. Die Direktform I verschiebt pro Sample vier Werte. Die Direktform II verschiebt nur zwei Werte. Die unterschiedliche Formulierung kann einen Unterschied in der gemessenen Laufzeit verursachen. Das liegt nicht nur an der Codegenerierung. Mit dem vorliegenden



(a) Direktform I: getrennte Ketten für Eingangs- und Ausgangswerte



(b) Direktform II: beide Zweige teilen sich dieselbe Kette

Abbildung 4.4: Die beiden Realisierungsformen des Biquad-Filters. Beide berechnen dieselbe Übertragungsfunktion und liefern – bis auf Rundung – dasselbe Ausgangssignal, unterscheiden sich aber im Zustand. Die Direktform I (a) ist die wörtliche Übertragung der Differenzengleichung (Gleichung 2.11): Sie führt eine eigene Kette für die vergangenen Eingangs- und eine zweite für die vergangenen Ausgangswerte und speichert deshalb vier Werte. Die Direktform II (b) zieht die Rekursion vor und lässt beide Zweige auf dasselbe Zwischensignal w zugreifen; sie kommt mit einer Kette und zwei gespeicherten Werten aus. Die manuelle Implementierung folgt der Form (a) (Abb. 4.4a), die Faust-Bibliotheksfunktion `fi.tf2` erzeugt die Form (b) (Abb. 4.4b). Die Verzögerungsglieder sind zur Hervorhebung abgesetzt gezeichnet.

Aufbau kann man beide Ursachen nicht voneinander trennen. In Abschnitt 5.6 wird gezeigt, dass beide Varianten trotzdem dasselbe Ergebnis liefern.

FIR-Filter

Untersucht werden FIR-Filter mit $L \in 32, 64, 256$ Koeffizienten. Die Koeffizienten bilden eine normierte Rampe

$$b_k = \frac{2, (k+1)}{L, (L+1)}, \quad k = 0, \dots, L-1, \quad (4.4)$$

deren Summe eins ergibt.

In Faust wird der Filter über die Bibliotheksfunktion `fi.fir` mit einer erzeugten Koeffizientenliste beschrieben. Jedes Ausgangssample ist die gewichtete Summe der letzten L Eingangssamples. Es gibt keine Rückkopplung. Die manuelle Implementierung hält die vergangenen Eingangswerte in einem Puffer. Ihre Umsetzung zeigt Abschnitt 5.1.

Abbildung 4.5 zeigt das zugehörige Faust-Blockdiagramm.

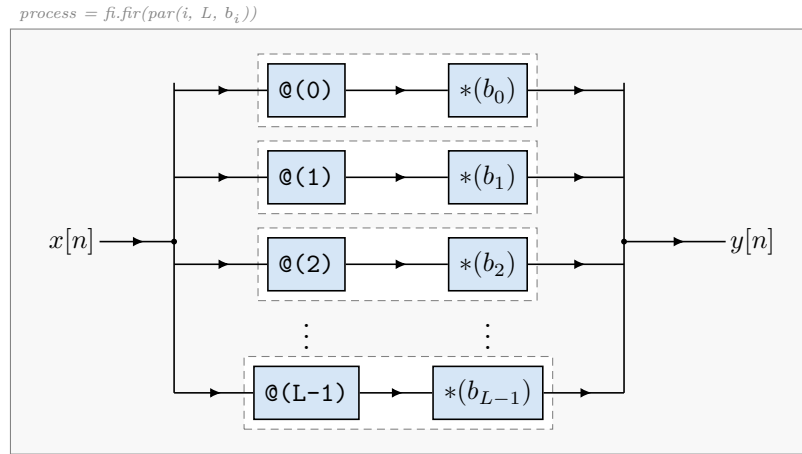


Abbildung 4.5: FIR-Filter (`fir32.dsp` u.a., `fi.fir`). Das Eingangssignal wird auf L Zweige verteilt ($<:$); jeder Zweig ist eine sequenzielle Komposition aus Verzögerung und Gewichtung, danach werden alle Zweige zusammengeführt ($:>$). Gezeichnet sind die ersten drei und der letzte Zweig; gemessen wird mit $L \in \{32, 64, 256\}$. Die Struktur ist vollständig parallel – es gibt keine Rückkopplung, und der Faust-Compiler schreibt die Summe im erzeugten C++-Code als eine einzige ausgeschriebene Zeile aus (Abb. 5.2).

Der Baustein trägt zur Beantwortung der Frage nach dem Skalierungsverhalten bei. Er ist der einzige, bei dem sich der Rechenaufwand unabhängig von der Blockgröße verändern lässt, und drei Filterlängen zeigen, ob ein Unterschied zwischen beiden Ansätzen besteht, ob der Aufwand bei beiden Ansätzen wächst, schrumpft oder gleich bleibt. Da die Faltung ohne Rückkopplung auskommt, ist sie zugleich der Fall, in dem der Nutzen der automatischen Vektorisierung am deutlichsten zu sehen ist. Schließlich ist er für die Binärgröße aufschlussreich, weil der Faust-Compiler die Faltung vollständig ausrollt und die Codegröße daher mit L wachsen dürfte.

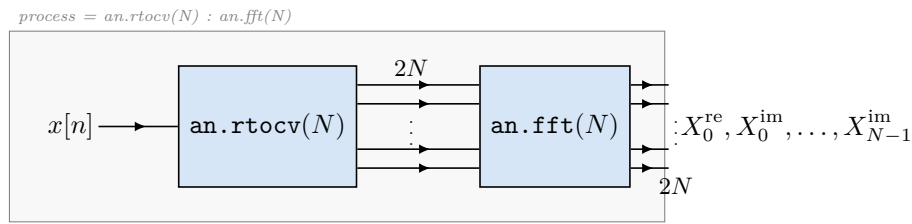
4.1.1 Fast-Fourier-Transformation

Untersucht werden FFTs mit $N \in 64, 128, 256$. Beide Implementierungen berechnen eine gleitende Fenster-FFT: Für jedes Eingangssample wird die N -Punkt-FFT aus den letzten N Samples berechnet. Der Baustein hat einen Eingang und $2N$ Ausgänge, die Real- und Imaginärteil abwechselnd ausgeben.

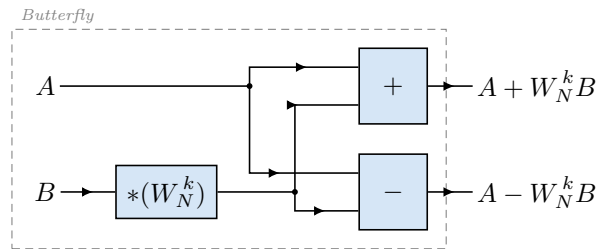
In Faust wird er über die Bibliotheksfunktionen `an.rtcv(N)` und `an.fft(N)` beschrieben. Die manuelle Implementierung ist eine iterative Radix-2-FFT (Lyons et al., 2004) nach Cooley-Tukey (Cooley & Tukey, 1965) mit Dezimierung im Zeitbereich. Ihren Aufbau zeigt Abschnitt 5.1.

Abbildung 4.6 zeigt das passende Faust-Blockdiagramm.

Die FFT ist der einzige Baustein, bei dem der Aufwand nicht linear wächst, und ergänzt die Skalierungsfrage. Vor allem aber ist sie der Fall, in dem sich die Codegenerierung am stärksten von gewöhnlichem C++ unterscheidet. Der Faust-Compiler rollt die gesamte Butterfly-Struktur (Lyons et al., 2004) aus, während die manuelle Implementierung sie in verschachtelten Schleifen durchläuft. Deshalb ist die FFT der wichtigste Baustein für die Fragen nach Binärgröße und Speicherbedarf und zugleich der einzige, an dem die Übersetzungsdauer des Faust-Compilers spürbar wird.



(a) Gesamtstruktur: ein Eingang, $2N$ Ausgänge



(b) Elementare Butterfly-Operation; $\log_2 N$ Stufen zu je $N/2$ Stück

Abbildung 4.6: FFT (`fft.dsp`). `an.rtcv` bildet aus dem Eingangssignal ein gleitendes Fenster der Länge N und serialisiert es – mit dem neuesten Sample zuerst – als $2N$ Signale, in denen Real- und Imaginärteil abwechseln. `an.fft` transformiert dieses Fenster. Anders als beim FIR-Filter ist die innere Struktur in Faust nicht als Schleife formuliert, sondern als Blockdiagramm aus $\log_2 N$ Stufen zu je $N/2$ Butterflies, das der Compiler vollständig ausrollt. Für $N = 256$ sind das 1024 Butterflies je Ausgangssample. Darin liegt der wesentliche Unterschied zur manuellen Implementierung, die dieselbe Struktur in verschachtelten Schleifen durchläuft.

Der Vergleich weist hier zwei Schwächen auf. Die gleitende Fenster-FFT berechnet für jedes einzelne Sample eine vollständige Transformation und ist damit weit aufwendiger als die in der Praxis übliche Blockverarbeitung mit Fenstervorschub. Diese Form ergibt sich aus der Semantik der verwendeten Faust-Funktion und wurde nicht gewählt, weil sie praxisnah wäre. Zudem sind die untersuchten Transformationslängen klein. Die Zwischenspeicher der Testplattform fassen alle drei Größen mühelos, sodass sich Effekte der Speicherhierarchie nicht zeigen.

Zwei weitere Festlegungen sind wesentlich. Erstens wird bewusst keine Fremdbibliothek verwendet. Eine frühere Version des Versuchsaufbaus benutzte FFTW (Lyons et al., 2004). Dieser Vergleich hätte Faust mit einer von Hand optimierten Bibliothek vergleichen sollen, Assembler-Kernen gestellt und nicht gegen von Hand geschriebenen C++-Code. Zweitens serialisiert Faust das Analysefenster mit dem neuesten Sample zuerst. Die manuelle Implementierung kommt als Nächstes, dann folgt die Referenzimplementierung, dieser Reihenfolge, weil andernfalls beide Varianten unterschiedliche Spektren berechnen würden.

Grenzen der Auswahl

Die fünf Bausteine decken nur einen schmalen Ausschnitt dessen, was in der Audio-Signalverarbeitung zu finden ist. Sie sind alle linear, zeitinvariant und arbeiten mit konstanten Koeffizienten. Nicht untersucht werden nichtlineare Verarbeitung, zeitvariable Parameter, Mehrkanalverarbeitung sowie Bausteine mit großen Verzögerungsspeichern, wie sie etwa in Nachhallalgorithmen auftreten. Ebenso wenig untersucht wird eine vollständige Signalkette, in der mehrere Bausteine hintereinander geschaltet sind. Die Ergebnisse gelten deshalb für die beschriebenen Bausteine auf der beschriebenen Plattform.

Eine zweite Einschränkung betrifft die Vergleichsbasis selbst. Die manuellen Implementierungen sind im Rahmen dieser Arbeit entstanden und nicht von Personen geschrieben, die auf handoptimierten DSP-Code spezialisiert sind. Ein erfahrener Entwickler könnte die C++-Seite an mehreren Stellen weiter optimieren. Umgekehrt bleibt auf der Faust-Seite der Vektor-Modus ungenutzt. Der Vergleich betrifft daher jeweils die naheliegende Umsetzung beider Wege und nicht das jeweils beste Ergebnis. Abschnitt 7.3 geht darauf erneut ein.

4.2 Benchmark-Tools und Messgrößen

4.2.1 Messwerkzeuge

Die Laufzeitmessung erfolgt ausschließlich mit Google Benchmark. Die Bibliothek ist als Submodul eingebunden und wird gemeinsam mit dem Projekt übersetzt. Aus einem einzigen Messlauf werden sowohl die Mittelwerte mit Fehlerbalken sowie Median, 95. Perzentil und Interquartilsabstand abgeleitet.

Jeder Baustein wird mit 100 Wiederholungen gemessen. Die Option `report_aggregates_only` wird dabei auf `false` gesetzt. Sie sorgt dafür, dass die einzelnen Wiederholungen und nicht nur deren Aggregate in die JSON-Ausgabe gelangen. Erst dadurch lassen sich Median, Perzentile und die statistischen Tests aus Abschnitt 4.6 berechnen.

Innerhalb der Messschleife verhindern die Hilfsfunktionen `DoNotOptimize` und `ClobberMemory`, dass der Compiler den Aufruf als toten Code entfernt (Google LLC, 2023). Die Messprozesse werden mit `taskset` auf einen festen CPU-Kern gesetzt. Das verhindert, dass während der Messung der Kern gewechselt wird. So bleiben die schnellen Caches erhalten (Mytkowicz et al., 2009).

Gemessen wird für die Blockgrößen $N \in 64, 128, 256, 512, 1024$. Für die FFT wird je Transformationslänge ein eigenes ausführbares Programm erzeugt, weil der Faust-Compiler die FFT je Größe vollständig ausrollt und die generierte Klasse, damit sie größenabhängig ist.

Es existiert ein zweiter, unabhängiger Messstand auf Basis von `std::chrono::steady_clock`. Er misst jeden einzelnen `compute()`-Aufruf getrennt, wärmt vor jeder Runde beide Codepfade vollständig auf und wechselt die Reihenfolge von Faust- und C++-Runden rundenweise. Dies gilt nur als Gegenprobe.

4.2.2 Messgrößen

Tabelle 4.2 fasst die erhobenen Messgrößen zusammen.

Tabelle 4.2: Erhobene Messgrößen und ihre Ermittlung

Messgröße	Ermittlung	Werkzeug
CPU-Zeit je Block	Median über 100 Wiederholungen	Google Benchmark
Streuung	95. Perzentil, Interquartilsabstand	Auswertungsskript
Durchsatz	Verarbeitete Samples je Sekunde	Google Benchmark
Binärgröße	Gestrippte Größe minus Basislinie	<code>strip</code> , <code>stat</code>
Zustandsgröße	<code>sizeof</code> der DSP-Klasse	Probe-Programm
Speicherbedarf	Maximaler RSS, Heap-Spitze	GNU <code>time</code> , Valgrind
Genauigkeit	Fehler gegenüber float64-Referenz	Vergleichsprogramme
Codelänge	Codezeilen je Implementierung	Auswertungsskript
Compile-Zeit	Laufzeit des Faust-Compilers	Messskript

Die wichtigste Messgröße ist die Prozessorzeit pro Audioblock. Außerdem werden Speicherverbrauch und Binärgröße gemessen. Diese Werte bestimmen, wie gut sich eine Implementierung in der Praxis nutzen lässt, nicht nur wie schnell sie läuft. Codelänge und Implementierungskomplexität zeigen zusätzlich, wie aufwändig die Arbeit für Entwickler ist. Eine genaue Beschreibung dieser Größen, die Zahl der Wiederholungen und die statistische Auswertung stehen in Kapitel ??.

Die *Binärgröße* zeigt nicht die tatsächliche Dateigröße an. Ein übersetztes Programm besteht zu hauptsächlich aus Startup-Code und Laufzeit-Code der Standardbibliothek. Deshalb misst man die Differenz zwischen einem Probe-Programm, das genau einen Baustein enthält, und einer Basislinie mit leerer `main`-Funktion. Beide werden vorher gestrippt, entfernt. Für die Für jede Filterlänge existiert ein eigenes Probe-Programm, damit nicht alle drei generierten Klassen in dasselbe Binary gelangen.

Der *Speicherbedarf* wird auf drei Ebenen erfasst. Die Zustandsgröße `sizeof` beschreibt den Speicher, den die DSP-Klasse selbst belegt. Der maximale Resident-Set-Size-Wert und die Heap-Spitze beschreiben den Speicherbedarf des laufenden Prozesses. Von beiden wird die Basislinie abgezogen. Zusätzlich zählt das Probe-Programm über

einen überladenen `new`-Operator die Speicheranforderungen innerhalb von `compute()`. Diese Zahl muss null sein. Eine Speicheranforderung im Audio-Pfad im Echtzeitbetrieb wirkt sich sehr negativ aus, weil der Allocator Sperren hält, Speicher beim Betriebssystem nachfordert und Verfahren mit unbeschränkter Laufzeit verwenden kann (Buttazzo, 1997).

Die *numerische Genauigkeit* wird als maximaler und mittlerer Absolutfehler sowie als quadratisches Mittel des Fehlers gegenüber einer Referenz in doppelter Genauigkeit angegeben. Der relative Fehler wird auf den größten Betrag der Referenz bezogen und als Vielfaches der Maschinengenauigkeit von `float` ausgedrückt, also von $2^{-23} \approx 1,19 \cdot 10^{-7}$. Der Effektivwert wäre hier ein ungeeigneter Bezugswert: Bei abklingenden Signalen wie einer Impulsantwort sinkt er mit wachsender Signallänge, während der absolute Fehler konstant bleibt. Die Fehlerquote würde dann scheinbar steigen, obwohl sich nichts verschlechtert hat.

Die *Codelänge* gibt an, wie viele Codezeilen vorhanden sind. Eine Codezeile ist eine Zeile, die weder leer ist noch ausschließlich aus einem einzeiligen Kommentar bleibt. Die Zählung erfolgt getrennt nach der Faust-Quelldatei, dem zugehörigen C++-Wrapper und der manuellen Implementierung. Die Trennung ist nötig, weil der Wrapper für alle Bausteine nahezu identisch ist und damit nicht zur eigentlichen Beschreibung gehört.

Die *Compile-Zeit* des Faust-Compilers wird zusätzlich gemessen. Dazu werden aus Vorlagen Programme mit wachsender Rekursionstiefe, wachsender Baumtiefe und wachsende FFT-Größe werden erzeugt und übersetzt. Läufe mit einer Übersetzungen, die länger als 30 Sekunden dauern, werden abgebrochen.

4.3 Hardwareumgebung

Alle Messungen laufen auf der in Tabelle 4.3 beschriebenen Plattform. Die Umgebung wird bei jedem Messlauf automatisch protokolliert. Das Protokoll enthält neben Hardware und Werkzeugversionen. Zusätzlich wird die Systemlast zum Zeitpunkt der Messung dokumentiert. Als Richtwert für belastbare Messungen gilt eine Ein-Minuten-Last unter 0,3.

Tabelle 4.3: Messumgebung

Komponente	Ausprägung
Prozessor	AMD Ryzen 7 4700U, 8 Kerne, 1 Thread je Kern
Cache	L1d 256 KiB, L2 4 MiB, L3 4 MiB
Betriebssystem	Ubuntu 24.04 unter WSL2, Kernel 6.18
C++-Compiler	g++ 13.3.0
Build-System	CMake 3.29.3, Ninja 1.11.1
Faust-Compiler	Faust 2.83.1
Auswertung	Python 3.12.3, NumPy 2.5.1

Die Wahl der Plattform bringt zwei Einschränkungen mit sich, die bei der Interpretation der Ergebnisse zu berücksichtigen sind. Erstens laufen die Messungen in einer virtuellen Maschine unter WSL2. Zweitens lässt sich der Frequenzregler des Prozessors in dieser Umgebung weder auslesen noch festlegen. Der verwendete Prozessor ist ein

Mobilprozessor, dessen Taktfrequenz von Temperatur und Energiebudget abhängt.

Für den Versuchsaufbau wurden drei Vorkehrungen getroffen. Beide Implementierungen werden innerhalb desselben Prozesses und desselben Zeitfensters gemessen, die Messungen werden an einen festen Kern gebunden, und es wird eine hohe Zahl von Wiederholungen ausgewertet. Deshalb werden die Verhältnisse zwischen beiden Implementierungen berichtet. Abschnitt 7.3 greift diese Einschränkung auf.

4.4 Compilerkonfigurationen

4.4.1 Aufruf des Faust-Compilers

Der Faust-Compiler wird für alle Bausteine gleich aufgerufen, nämlich mit der Option `-cn` für den Namen der erzeugten Klasse und ohne weitere Schalter.

Damit wird das voreingestellte C $\pm$ -Backend im skalaren Modus verwendet. Die Optionen für den Vektor-Modus und den Parallel-Modus werden bewusst nicht gesetzt. Die Forschungsfrage vergleicht den generierten C++-Code mit manuell geschriebenem C++-Code. Würde auf der Faust-Seite zusätzlich der Vektor-Modus oder OpenMP aktiviert, vergliche man zwei unterschiedliche Parallelisierungsstrategien statt zwei Beschreibungswege. Die Wirkung dieser Modi ist zudem bereits Gegenstand eigener Untersuchungen (Letz et al., 2010; Scaringella et al., 2003). Die automatische Vektorisierung durch `g++` bleibt für beide Seiten aktiv.

4.4.2 Flag-Matrix

Beide Implementierungen werden immer mit identischen Compilerflags übersetzt. Für die Untersuchung des Einflusses der Compileroptimierung wird die in Tabelle 4.4 gezeigte Matrix durchlaufen. Jede Konfiguration erhält ein eigenes Build-Verzeichnis, sodass keine Zwischenergebnisse aus einer anderen Konfiguration wiederverwendet werden können.

Tabelle 4.4: Untersuchte Compilerkonfigurationen

Bezeichnung	Flags	Zweck
O0	-O0	Unoptimierter Bezugspunkt
O1	-O1	Grundoptimierungen
O2	-O2	Üblicher Standard
Os	-Os	Optimierung auf Codegröße
O3	-O3	Vektorisierung aktiv
O3_native	-O3 -march=native	Befehlssatz der Hardware
O3_native_fast	-O3 -march=native -ffast-math	Gelockerte FP-Regeln

Alle Konfigurationen enthalten zusätzlich `-DNDEBUG`. Als Sprachstandard wird C++17 verwendet. Die Optimierungsflags werden im Build-System als erzwungener Cache-Eintrag gesetzt, damit der zwischengespeicherte Wert nicht von den tatsächlich verwendeten Flags abweichen kann.

Die Aufnahme von `-ffast-math` aktiviert unter anderem `-fassociative-math` und erlaubt dem Compiler damit, Gleitkommaadditionen ohne Beachtung der IEEE-754-

Semantik beliebig umzustellen. Bei rückgekoppelten Filtern kann das dazu führen, dass sich die Abhängigkeitskette verlängert. Dadurch steigt die Laufzeit an, anstatt zu sinken.

4.5 Testdatengenerierung

Alle Testsignale werden durch ein Python-Skript erzeugt und als binäre Rohdateien im Format `float32` abgelegt. Als Zufallsgenerator dient der Standardgenerator von NumPy mit dem festen Startwert 42. Ziel ist es, bei beliebig vielen Läufen und in beiden Implementierungen die gleichen Signale zu verwenden. Die Faust-Variante und die C++-Referenz lesen die gleiche Datei.

Es werden zwei Kategorien von Testdaten erzeugt.

Signale für die Laufzeitmessung

Für die Blockgrößen 64, 128, 256, 512, 1024 und 2048 Samples wird jeweils ein Signal aus gleichverteiltem Rauschen im Wertebereich $[-1; 1]$ erzeugt.

Signale für den Genauigkeitsvergleich

Für den Genauigkeitsvergleich werden die in Tabelle 4.5 aufgeführten Signaltypen in den Längen 64, 256, 1024 und 4096 Samples erzeugt. Sie decken sowohl typische Audiosignale als auch Randfälle ab.

Tabelle 4.5: Signaltypen für den Genauigkeitsvergleich

Typ	Definition	Zweck
<code>impulse</code>	Einheitsimpuls	Impulsantwort der Filter
<code>step</code>	Sprungfunktion	Sprungantwort, Einschwingverhalten
<code>silence</code>	Nullsignal	Trivialer Randfall
<code>dc</code>	Konstantwert 0,5	Einfachste Linearitätsprüfung
<code>sine</code>	1-kHz-Sinus bei 48 kHz	Typisches Audiosignal
<code>extreme</code>	Alternierend $\pm 1,0$	Ungünstigster Fall für Rundung
<code>noise</code>	Rauschen, eigener Startwert	Unabhängiger Zufallsfall

Referenz in doppelter Genauigkeit

Ein Vergleich, der ausschließlich Faust gegen die C++-Referenz stellt, kann einen systematischen Fehler nicht aufdecken, den beide Implementierungen gemeinsam machen. Deshalb gibt es für jeden Baustein und jedes Testsignal eine Referenzausgabe in doppelter Genauigkeit mit NumPy berechnet und als Rohdatei gespeichert. Die Referenzalgorithmen geben die Struktur der C++-Bausteine sind ähnlich, sie unterscheiden sich nur im Datentyp. Beide Implementierungen werden anschließend gegen diese Referenz gemessen.

4.6 Auswertungsverfahren

4.6.1 Kennzahlen und statistische Prüfung

Aus den Einzelwiederholungen jedes Bausteins wird der Median der CPU-Zeit gebildet. Der Vergleich beider Implementierungen erfolgt über das Verhältnis

$$r = \frac{\tilde{t}_{\text{Faust}}}{\tilde{t}_{\text{C++}}}, \quad (4.5)$$

wobei $\tilde{t}$ jeweils den Median bezeichnet. Ein Wert $r < 1$ bedeutet, dass die Faust-Variante schneller ist.

Hierzu wird mit dem Bootstrap-Verfahren gearbeitet (Efron & Tibshirani, 1993). Aus den 100 gemessenen Werten werden zufällig erneut 100 gezogen. Derselbe Wert kann dabei mehrmals gezogen werden. Für diese neue Stichprobe wird das Verhältnis erneut berechnet. Nach 10000 Wiederholungen liegen 10000 Verhältnisse vor. Die mittleren 95 Prozent davon bilden das Konfidenzintervall. Der Startwert des Zufallsgenerators ist fest. So bleibt das Intervall reproduzierbar.

Ergänzend wird der Mann-Whitney-U-Test gerechnet (Mann & Whitney, 1947). Er vergleicht nicht die Messwerte selbst. Stattdessen ordnet er sie der Größe nach. Alle 100 Werte werden sortiert, und der Test prüft, ob die Werte einer Seite systematisch weiter vorne liegen. Weil nur die Reihenfolge zählt, verschiebt ein einzelner Ausreißer das Ergebnis kaum. Das passt zu Laufzeitmessungen, die nach unten durch die Hardware begrenzt sind, nach oben aber durch das Betriebssystem beliebig weit gestreckt werden können.

Das Intervall beschreibt allerdings nur, wie stark das Verhältnis schwanken würde, wenn die Messung unter identischen Bedingungen wiederholt würde. Da jede der 100 Wiederholungen bereits ein Mittelwert über viele Aufrufe ist, liegen diese Werte eng beieinander, und die Intervalle fallen entsprechend schmal aus. Ein schmales Intervall belegt daher eine gut reproduzierbare Messung und nicht, dass das Verhältnis den wahren Leistungsunterschied genau wiedergibt. Die Wahl der Compilerflags, die unterschiedliche Filterstruktur oder die Virtualisierung der Testplattform gehen nicht in das Intervall ein.

4.6.2 Auflösungsgrenze des Messaufbaus

Ein signifikanter Unterschied ist nicht automatisch ein inhaltlich bedeutsamer Unterschied. Bei einer hohen Zahl von Wiederholungen wird auch reines Messrauschen signifikant. Der Versuchsaufbau bestimmt deshalb eine Auflösungsgrenze und kennzeichnet alle Effekte unterhalb dieser Grenze als nicht auflösbar.

Als Bezugspunkt dient die Verstärkungsstufe. Die Annahme lautet, dass beide Implementierungen für diesen Baustein erzeugen praktisch identischen Code, der dort gemessene Abweichung, also Messrauschen, darstellt. Der größte bei diesem Baustein gemessene Abstand $|r - 1|$ über alle Blockgrößen wird als Auflösungsgrenze verwendet. Die Annahme ist Teil des Verfahrens und kein Ergebnis. Sie wird in Kapitel 6 mit dem erzeugten Code und den gemessenen Werten überprüft.

Aus Verhältnis, Konfidenzintervall und Auflösungsgrenze ergibt sich für jeden Messpunkt eine von drei Aussagen:

- Enthält das Konfidenzintervall den Wert eins, ist kein Unterschied nachweisbar.
- Liegt $|r-1|$ unterhalb der Auflösungsgrenze, ist der Unterschied mit diesem Aufbau nicht auflösbar.
- Andernfalls wird der Unterschied als Prozentsatz angegeben.

4.7 Übersicht der Messreihen

Aus den bisherigen Festlegungen ergeben sich neun Messreihen. Sie laufen als zusammenhängende Kette ab und werden nicht einzeln von Hand angestoßen. Tabelle 4.6 ordnet jeder Reihe zu, was sie erhebt und zu welcher Teilfrage aus Abschnitt 1.1 sie beiträgt.

Tabelle 4.6: Messreihen und ihr Beitrag zur Beantwortung der Teilfragen

Messreihe	Erhebt	Beitrag
Umgebungsprotokoll	Hardware, Werkzeuge, Git-Stand	Voraussetzung aller Reihen
Referenzausgaben	Sollwerte in doppelter Genauigkeit	Grundlage der Genauigkeitsprüfung
Genauigkeitsprüfung	Fehler beider Varianten	Genauigkeit; Gültigkeitsnachweis
Laufzeitmessung	CPU-Zeit je Block, fünf Blockgrößen	Laufzeit und Skalierung
Optimierungs-Sweep	Dieselbe Messung, sieben Flagsätze	Einfluss der Compileroptimierung
Binärgrößenmessung	Gestrippte Größe je Baustein	Binärgröße
Speichermessung	Zustand, RSS, Heap, Allokationen	Speicherverbrauch
Quelltextumfang	Codezeilen je Variante	Codelänge
Übersetzungsdauer	Laufzeit des Faust-Compilers	Ergänzend, keine Teilfrage

Die Reihenfolge ist nicht zufällig. Zuerst entstehen die Referenzausgaben. Danach wird die Genauigkeit geprüft und erst zuletzt die Laufzeit gemessen. Die nächsten Abschnitte erklären für jede Reihe, welche Behauptung sie prüft, welches Ergebnis sie liefern soll und welche Frage damit beantwortet wird.

4.7.1 Voraussetzungen

Die ersten drei Reihen liefern keine Ergebnisse zur Forschungsfrage. Sie stellen sicher, dass die restlichen Zahlen überhaupt eine Bedeutung haben.

Das *Umgebungsprotokoll* prüft die Behauptung, dass die Messungen wiederholbar sind. Es speichert Hardware, Compiler, Faust- und NumPy-Version sowie den Git-Stand des Repositories und die Systemlast fest und warnt, wenn zum Zeitpunkt der Messung nicht eingetragene Änderungen vorliegen.

Die *Referenzausgaben* in doppelter Genauigkeit geben die Sollwerte für jeden Baustein und jedes Testsignal an. Sie sind nötig, weil ein Vergleich, der nur Faust mit der

C++-Referenz vergleicht, einen Fehler nicht erkennen kann, den beide Seiten gemeinsam machen. Nur eine unabhängig berechnete Referenz zeigt solche Fälle.

Die *Genauigkeitsprüfung* prüft die zentrale Voraussetzung des gesamten Versuch, dass beide Implementierungen die gleiche Funktion berechnen. Als Ergebnis liefert sie den maximalen und den mittleren Absolutfehler sowie das quadratische Mittel des Fehlers beider Varianten gegenüber der Referenz, bezogen auf die Auflösung von `float`. Die Voraussetzung wäre widerlegt, wenn ein Fehler deutlich über dieser Auflösung liegt. Dann rechnen beide Seiten nicht dasselbe. Jeder Laufzeitvergleich wäre dann sinnlos. Aus diesem Grund endet die Messkette an dieser Stelle. Sie fährt nicht mit der Laufzeitmessung fort. Gleichzeitig beantwortet die Reihe die Teilfrage nach der numerischen Genauigkeit.

4.7.2 Laufzeit und Compilerverhalten

Die nächsten beiden Reihen geben die Antwort auf die wichtigste Forschungsfrage.

Die *Laufzeitmessung* prüft die Behauptung, dass Faust-generierter Code einer von Hand geschriebenen C++-Referenz mindestens genauso gut ist. Als Ergebnis liefert sie die Median-CPU-Zeit je Block für fünf Blockgrößen und alle Bausteine, dazu das Verhältnis beider Implementierungen mit Konfidenzintervall und Signifikanztest. Bei den drei Filterlängen und drei Transformationslängen beantworten die Teilfrage zum Skalierungsverhalten. Mitgeprüft wird eine Annahme des Versuchsaufbaus selbst. Dass beide Seiten für die Verstärkungsstufe fast den gleichen Code erzeugen, zeigt, dass sie sich als Auflösungsgrenze eignen. (Abschnitt 4.6.2). Fällt diese Annahme durch, ist die Grenze anders zu bestimmen.

Der *Optimierungs-Sweep* prüft zwei Behauptungen. Die erste lautet, dass Compiler-Optimierungen auf beide Implementierungen gleich wirken. Die zweite stammt aus der Literatur und besagt, dass sich vektorisierbare Ausdrücke von solchen mit Datenabhängigkeiten trennen lassen und nur die erste Gruppe von Die automatische Vektorisierung bringt Vorteile (Scaringella et al., 2003). Als Ergebnis gibt die gleichzeitige Laufzeitmessung für die Reihe unter sieben verschiedenen Flag-Sets an. Bestätigt wäre die zweite Behauptung, wenn Verstärkungsstufe und FIR-Filter beim Übergang zu `-O3` deutlich gewinnen, Akkumulator und Biquad-Filter dagegen kaum verlieren. Widerlegt wäre die erste, wenn ein Flagsatz eine Seite bevorzugt. Bei `-ffast-math` ist genau das zu erwarten, weil das Umordnen von Gleitkommaoperationen die Abhängigkeitskette rekursiver Filter verlängern kann (Parhi, 1999). Beantwortet wird damit die Teilfrage nach dem Einfluss von Compileroptimierung und Vektorisierung.

4.7.3 Ressourcenverbrauch

Die *Binärgrößenmessung* prüft die Behauptung, dass der Faust-Compiler die Berechnung ausrollt und die manuelle Implementierung sie in Schleifen belässt. Trifft sie zu, so wächst die Codegröße der Faust-Variante mit der Filterlänge und Transformationslänge, während die der C++-Referenz gleich bleibt. Als Ergebnis liefert die Reihe die gestrippte Größe je Baustein abzüglich einer leeren Basislinie für drei Filterlängen und die FFT. Die Teilfrage zur Binärgröße wird beantwortet. Das Ergebnis ist zugleich der Beleg für den Unterschied bei der Codegenerierung.

Die *Speichermessung* prüft die Angabe, dass Faust Code mit deterministischem und

konstantem Speicherverbrauch erzeugt (Orlarey et al., 2009). Als Ergebnis liefert sie die Zustandsgröße der DSP-Klasse, den maximalen Speicherbedarf des Prozesses, die Heap-Spitze und die Zahl der Speicheranforderungen innerhalb von `compute()`. Beantwortet wird die Teilfrage nach dem Speicherverbrauch. Zustandsgröße und Heap-Spitze sind dabei gemeinsam zu lesen, weil die Faust-Variante ihren Zustand im Objekt hält, die manuelle Implementierung ihn dagegen dynamisch anfordert.

4.7.4 Entwicklungsaufwand

Der *Quelltextumfang* prüft die Behauptung, dass Faust-Beschreibungen kürzer als handgeschriebene C++-Implementierungen. Für physikalische Modelle wird dieser Vorteil mit 22,91 bis 80,20 Prozent angegeben (Michon & Smith, 2011). Offen ist, ob er sich auf elementare Bausteine übertragen lässt. Als Ergebnis liefert die Reihe die Codezeilen getrennt nach Faust-Beschreibung, C++-Wrapper und manueller Implementierung. Die Trennung ist entscheidend, weil der Wrapper für alle Bausteine nahezu identisch ist. Zählt man ihn mit, kann der Vorteil bei kleinen Bausteinen verschwinden. Beantwortet wird die Teilfrage nach Codelänge und Implementierungskomplexität.

Die *Übersetzungsdauer* des Faust-Compilers beantwortet keine der Teilfragen. Sie wird dennoch erhoben, weil sie das Ausrollen von einer zweiten Seite zeigt. Was sich in der Binärgröße als gewachsener Code niederschlägt, muss zuvor erzeugt werden. Als Ergebnis liefert die Reihe die Übersetzungsdauer über wachsende Transformationslängen und Rekursionstiefen, mit Abbruch nach 30 Sekunden. Für die praktische Arbeit mit der Sprache ist das ein spürbarer Faktor.

4.7.5 Einordnung und Grenzen des Messplans

Die neun Reihen teilen sich auf vier Aufgaben auf. Die ersten drei sichern die Gültigkeit, die beiden folgenden beantworten die eigentliche Frage, drei weitere decken die Nebenaspekte Binärgröße, Speicher und Codelänge ab, und die letzte stützt die Einordnung. Zwei der geprüften Behauptungen stammen nicht aus der Literatur, sondern aus dem Versuchsaufbau selbst. Dass beide Implementierungen dasselbe berechnen, und dass die Verstärkungsstufe als Auflösungsgrenze taugt. Beide sind in Kapitel 6 ausdrücklich zu prüfen und nicht vorauszusetzen.

Drei Größen werden bewusst nicht erhoben. Es wird keine Verteilung der Rechenzeit einzelner Blöcke ausgewertet, weshalb sich zur Einhaltung der Echtzeitfrist keine Aussage treffen lässt. Es wird keine vollständige Signalkette gemessen, sondern nur einzelne Bausteine. Und es werden keine alternativen Codegenerierungsmodi von Faust untersucht. Die Gründe sind in den jeweiligen Abschnitten genannt. Kapitel 7 greift sie gesammelt auf.

4.8 Zusammenfassung

Der Versuchsaufbau vergleicht fünf DSP-Bausteine mit unterschiedlichen Datenabhängigkeiten: eine Verstärkungsstufe ohne Rückkopplung, einen Akkumulator als reinen Rekursionsfall, einen Biquad-Filter, FIR-Filter in drei Längen und FFTs in drei Größen. Beide Implementierungen verwenden dieselben Koeffizienten, dieselben Testda-

ten, dieselbe Schnittstelle und dieselben Compilerflags. SIMD-Intrinsics und externe DSP-Bibliotheken sind auf beiden Seiten ausgeschlossen. Gemessen werden CPU-Zeit je Block, Streuung, Binärgröße, Speicherbedarf, numerische Genauigkeit und Codelänge. Die Auswertung stützt sich auf Mediane, Bootstrap-Konfidenzintervalle und einen verteilungsfreien Test und kennzeichnet Effekte unterhalb der Auflösungsgrenze des Aufbaus als nicht auflösbar. Das folgende Kapitel beschreibt, wie diese Festlegungen im Quelltext umgesetzt sind.

Kapitel 5

Implementierung der Benchmarks

Dieses Kapitel zeigt, *wie* Kapitel 4 im Quelltext umgesetzt ist. Die Faust-Beschreibungen und die manuellen C++-Implementierungen, die gemeinsame Schnittstelle, das Build-System, den Ablauf der Messkette und die Maßnahmen für eine faire Vergleichsbasis. Die Messergebnisse folgen in Kapitel 6.

Abbildung 5.1 zeigt den Ablauf beider Varianten. Der Faust-Compiler erzeugt aus der .dsp-Datei eine C++-Klasse. Ein Wrapper bindet sie an dieselbe Schnittstelle, die auch die manuelle Implementierung erfüllt. Ab dem Punkt durchlaufen sie dasselbe Messprogramm mit den gleichen Testdaten und den gleichen Compilerflags.

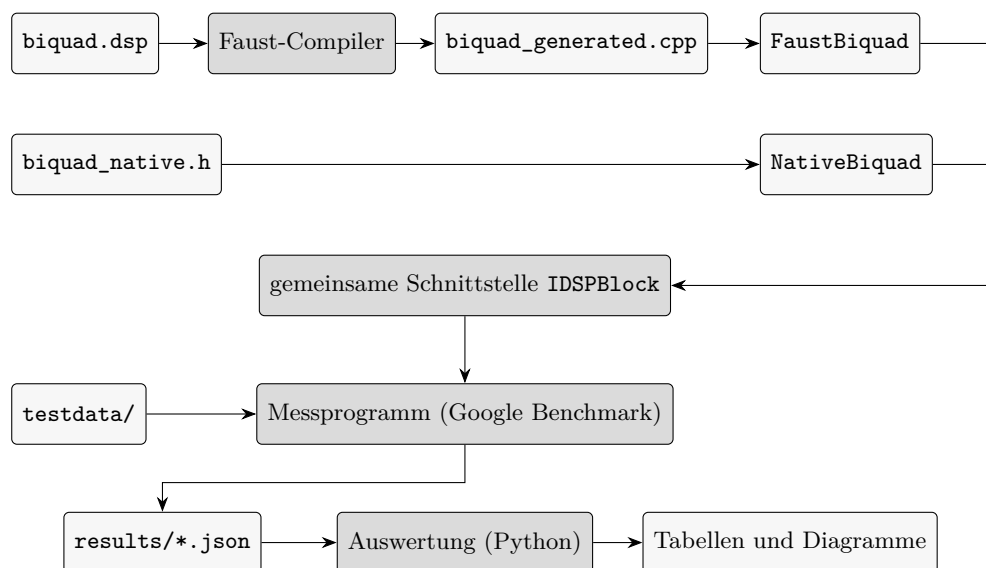


Abbildung 5.1: Vom Quelltext zum Messwert am Beispiel des Biquad-Filters.

5.1 Umsetzung der DSP-Bausteine

Die Faust-Quellbeschreibungen liegen in `dsp/`, die Wrapper in `src/faust/` und die manuellen Implementierungen in `src/native/`. Tabelle 5.1 ordnet jedem Baustein aus Tabelle 4.1 zu. Für den FIR-Filter gibt es zwei manuelle Implementierungen; Abschnitt 5.1.2 begründet, warum.

Tabelle 5.1: Zuordnung der Bausteine zu den Quelldateien

Baustein	Faust-Beschreibung	Wrapper	manuelle Implementierung
Gain	<code>dsp/gain.dsp</code>	<code>gain_wrapper.h</code>	<code>gain_native.h</code>
Akkumulator	<code>dsp/accum.dsp</code>	<code>accum_wrapper.h</code>	<code>accum_native.h</code>
Biquad-Filter	<code>dsp/biquad.dsp</code>	<code>biquad_wrapper.h</code>	<code>biquad_native.h</code>
FIR-Filter	<code>dsp/fir{32,64,256}.dsp</code>	<code>fir{32,64,256}_wrapper.h</code>	<code>fir_native.h</code> , <code>fir_native_linear.h</code>
FFT	<code>dsp/fft.dsp</code>	<code>fft_wrapper.h</code>	<code>fft_native.h</code>

5.1.1 Die Faust-Beschreibungen

Programm 5.1 zeigt die fünf Quellbeschreibungen vollständig.

Zu erkennen ist, dass ein bis drei Zeilen lang sind. Die Signalverarbeitung wird hier nicht ausprogrammiert, sondern als eine Zusammensetzung von elementare Bausteinen dargestellt.

Programm 5.1: Die fünf Faust-Quellbeschreibungen

```

1  // gain.dsp
2  process = *(hslider("gain", 0.5, 0, 2, 0.01));
3
4  // accum.dsp
5  import("stdfaust.lib");
6  process = + ~ _;
7
8  // biquad.dsp
9  import("stdfaust.lib");
10 process = fi.tf2(0.00255054, 0.00510107, 0.00255054,
11 -1.85214649, 0.86234863);
12
13 // fir32.dsp (fir64.dsp und fir256.dsp unterscheiden sich nur in N)
14 import("stdfaust.lib");
15 N = 32;
16 process = fi.fir(par(i, N, 2.0*(i+1)/(N*(N+1))));
17
18 // fft.dsp
19 import("stdfaust.lib");
20 N = 64;
21 process = an.rtcv(N) : an.fft(N);
22
```

Programm 5.2: Manuelle Biquad-Implementierung in Direktform I

```

1   for (int i = 0; i < count; ++i) {
2       float x0 = in[i];
3       float y0 = b0_*x0 + b1_*x1_ + b2_*x2_ - a1_*y1_ - a2_*y2_;
4       x2_ = x1_; x1_ = x0;
5       y2_ = y1_; y1_ = y0;
6       out[i] = y0;
7   }
8

```

Die Verstärkungsstufe verwendet keine Bibliotheksfunktion. Der Akkumulator besteht aus dem Rekursionsoperator aus Tabelle 3.1 in seiner kürzesten Form. `fi.tf2` nimmt die fünf Biquad-Koeffizienten entgegen, `fi.fir` eine Koeffizientenliste, die `par` nach der Vorschrift aus Abschnitt 4.1 erzeugt.

Keine der Beschreibungen enthalten Schleifen, Puffer oder Indexrechnung. Dieser Teil ergänzt nämlich der Faust-Compiler aus den Signalgleichungen.

5.1.2 Die manuellen C++-Implementierungen

Die manuellen Implementierungen sind Header-only-Klassen, geschrieben so, wie ein Entwickler den Baustein ohne Kenntnis der Faust-Seite formulieren würde.

Verstärkungsstufe und Akkumulator. Beide sind eine Schleife über die Samples mit einer Multiplikation beziehungsweise Addition auf einer Zustandsvariablen im Objekt. Weiterer Zustand und dynamische Speicheranforderungen kommen nicht vor.

Biquad-Filter. Die manuelle Implementierung folgt der Direktform I (Abbildung 4.4a); Programm 5.2 zeigt die Schleife. Sie hält die beiden vorherigen Ein- und Ausgangswerte im Objekt und schiebt sie nach jedem Sample weiter. Warum jede Seite ihre naheliegende Form verwendet und wie sich Direktform I und II unterscheiden, behandelt Abschnitt 4.1.

FIR-Filter. Der FIR-Filter ist ein Klassen-Template über die Filterlänge. Die Länge muss eine Zweierpotenz sein, damit der Ringpuffer über eine Bitmaske statt über eine Division adressiert werden kann. `static_assert` sichert das zur Übersetzungszeit ab. Der Faust-Compiler wählt einen anderen Weg und schreibt die gesamte Faltung als eine Summe aus. Abbildung 5.2 stellt beide Formen nebeneinander.

Die manuelle C++-Referenz implementiert den FIR-Filter als Klassentemplate über die Filterlänge L . Da ein FIR-Filter für jedes neue Ausgangssample den Zustand der vergangenen L Eingangswerte benötigt, kommt zur Vermeidung von aufwendigen Speicherverschiebungen *Memory Moves* ein zirkulärer Puffer *Ringpuffer* zum Einsatz. Dabei wird jeweils das älteste Sample durch den neuen Eingangswert überschrieben und der Zeiger wird inkrementiert..

Der Index muss nach Erreichen des Pufferende wieder an den Anfang. Ein Modulo-Operation `index % L` wird dazu verwendet. Die Division wird vermieden, da ganzzah-

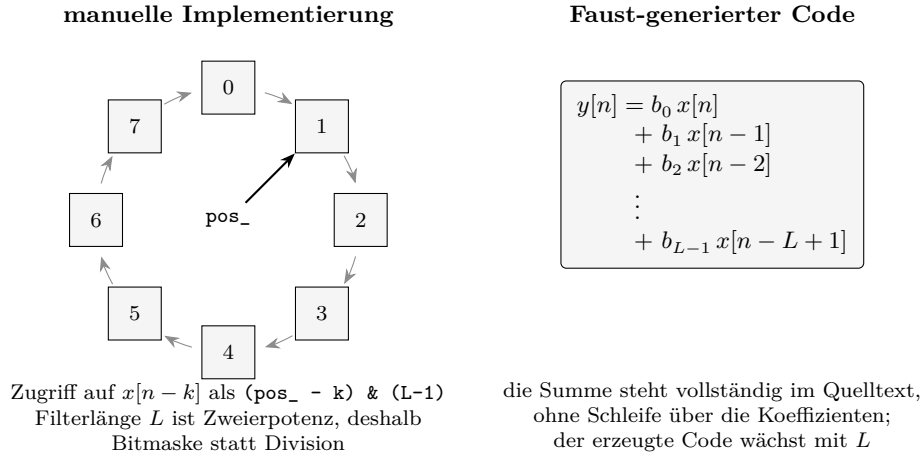


Abbildung 5.2: Zwei Wege zur selben Faltung. Die manuelle Implementierung hält die vergangenen Eingangswerte in einem Ringpuffer und läuft in einer Schleife über die Koeffizienten. Der Faust-Compiler schreibt dieselbe Summe vollständig aus. Abbildung nach (Smith, 2007).

lige Division kostenintensiv und mehrere Taktzyklen und somit mehr Latenz verursacht. Ist die Puffergröße L jedoch eine Zweierpotenz $L = 2^k$, lässt sich die Modulo-Operation verlustfrei durch eine bitweise UND-Verknüpfung mit der Maske $L - 1$ ersetzen (**warren2013hackers**).

$$\text{index} \pmod{L} \equiv \text{index} \& (L - 1) \quad (5.1)$$

Diese Bitmaskierung wird in Hardware als elementare Bitoperation in einem einzigen Taktzyklus ausgeführt. Um die Gültigkeit dieser Bedingung zwingend einzuhalten, stellt ein `static_assert` bereits zur Übersetzungszeit sicher, dass für L ausschließlich Zweierpotenzen instanziiert werden.

Der Faust-Compiler verfolgt einen grundlegend anderen Ansatz. Er verzichtet zur Laufzeit auf eine dynamische Ringpuffer-Adressierung und expandiert die diskrete Faltungssumme stattdessen zu einem vollständig ausgerollten Ausdruck mit expliziten Verzögerungsvariablen. Abbildung 5.2 stellt die beiden resultierenden Kontroll- und Datenflussstrukturen gegenüber.

Beide Varianten sind idiomatisches C++ und werden in derselben Messreihe nacheinander gemessen. Damit lässt sich ein Unterschied zwischen den Sprachen von einem Unterschied zwischen zwei C++-Formulierungen trennen. Genauigkeitsprüfung und Binärgrößenmessung verwenden nur die Ringpuffer-Variante, weil dort nicht die Codeform untersucht wird.

Fast-Fourier-Transformation. Aufbau und Semantik der gleitenden Fenster-FFT werden in Abschnitt 4.1.1 beschrieben. Hier geht es um drei Details zur Umsetzung. Bit-Umkehrtabelle und Twiddle-Faktoren werden einmalig in `initForSize()` berechnet (Lyons et al., 2004). Diese Methode legt auch die Puffer an. Sie muss deshalb vor dem ersten Aufruf von `compute()` ausgeführt werden. Sie ist das Gegenstück zur Faust-Seite.

Programm 5.3: Die gemeinsame Schnittstelle IDSPBlock

```

1  class IDSPBlock {
2      public:
3          virtual void init(int sampleRate) = 0;
4          virtual void compute(int count, float** inputs, float** outputs) = 0;
5          virtual int  getNumInputs() = 0;
6          virtual int  getNumOutputs() = 0;
7          virtual ~IDSPBlock() = default;
8  };
9

```

Dort ist die Größe bereits beim Übersetzen festgelegt. In der Verarbeitungsschleife wird für jedes Sample das Fenster aus dem Verlaufspuffer serialisiert. Es wird transformiert und als Real- und Imaginärteil auf $2N$ Ausgänge geschrieben. Die Fensterorientierung, die über die Gleichheit beider Varianten entscheidet, ist im Quelltext als Übersetzungsschalter festgehalten und dokumentiert. So kann sie sich nicht unbemerkt ändern.

5.1.3 Zustandshaltung und Speicherbedarf

Beide Seiten speichern ihren Zustand auf unterschiedliche Weise. Die Faust-Klassen haben Verzögerungsleitungen 3.1 und Koeffizienten als feste Felder im Objekt. Die manuellen Implementierungen von FIR und FFT verwenden `std::vector` und holen sich ihren Zustand beim Erstellen des Objekts oder in `initForSize()`. Ihr `sizeof` ist deshalb klein. Ihr Bedarf an Heap-Speicher ist entsprechend größer. Gain und Akkumulator kommen auf beiden Seiten ohne Speicherzuweisung aus.

Entscheidend ist, dass diese Anforderungen nur außerhalb von `compute()` stattfinden. Innerhalb muss die vom Probe-Programm gezählte Zahl null sein. Zustandsgröße und Heap-Spitze sind bei der Auswertung deshalb gemeinsam zu lesen. Betrachtet man jede der beiden Zahlen einzeln, führt das in die Irre.

5.2 Gemeinsame Test-Schnittstelle

Beide Varianten halten sich an die abstrakte Schnittstelle aus Programm 5.3. So können dieselben Messprogramme beide vermessen.

Abbildung 5.3 zeigt die Vererbungsbeziehung und die Nutzer der Schnittstelle.

Die Faust-Wrapper geben alle vier Methoden an das Objekt der erzeugten Klasse weiter. Bei der Initialisierung rufen sie `instanceInit()` auf und nicht das umfangreichere `init()`. Der Unterschied betrifft klassenweite Nachschlagetabellen. Die fünf untersuchten Beschreibungen nutzen diese nicht. Die Prüfung aus Abschnitt 5.6 würde eine unvollständige Initialisierung sofort als Abweichung zeigen.

Die Aufteilung hat drei Folgen für die Messung. Erstens ist jedes Messprogramm eine Template-Funktion des Baustein-Typs, sodass beide Varianten denselben Quelltext durchlaufen. Zweitens ist der Wrapper für alle Bausteine nahezu identisch. Er wird deshalb bei der Codelängenmessung gemäß Abschnitt 4.2.2 getrennt gezählt. Drittens beschränkt sich sein Aufwand auf einen virtuellen Aufruf je Block statt je Sample. Dieser

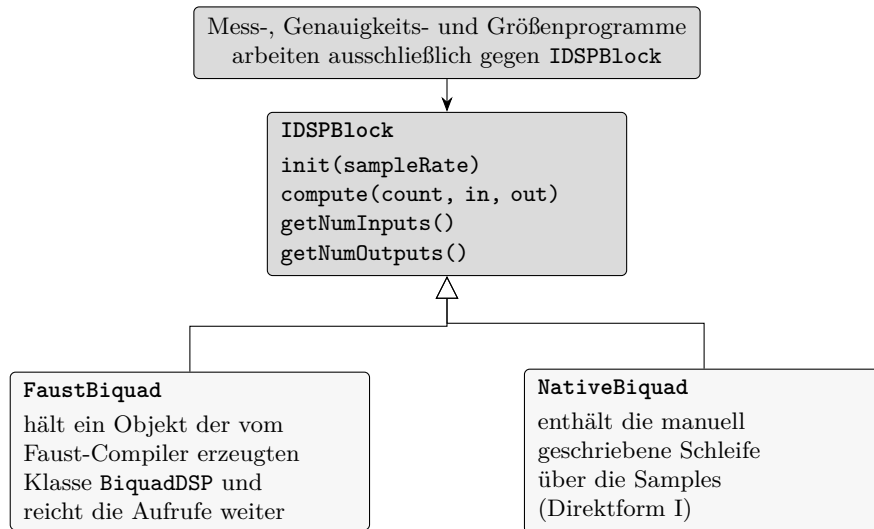


Abbildung 5.3: Gemeinsame Test-Schnittstelle. Beide Varianten erben von derselben abstrakten Klasse. Die Messprogramme kennen nur diese Schnittstelle und können deshalb für beide Varianten unverändert verwendet werden.

Aufwand fällt für beide Varianten gleich hoch aus.

Zwei Bausteine weichen von diesem Muster ab. Bei den FIR-Filtern bindet der gemeinsame Wrapper bewusst keine generierte Datei ein. Das machen erst die drei größen-spezifischen Wrapper. Sonst hätte jede Übersetzungseinheit alle drei generierten Klassen und die Binärgrößenmessung wäre ungültig. Bei der FFT ist die Faust-Klasse je Transformationslänge eine eigene Klasse. Die manuelle Implementierung bekommt die Länge erst zur Laufzeit. Deshalb gibt es, wie in Abschnitt 4.2 genannt, ein eigenes Programm je Länge.

5.3 Build-System

Das Projekt wird mit CMake ab Version 3.20 und Ninja gebaut. Google Benchmark ist als Submodul eingebunden. Es wird mit denselben Flags gebaut wie der Messcode.

5.3.1 Aufruf des Faust-Compilers

Der Aufruf ist als CMake-Funktion gekapselt (Programm 5.4). Sie meldet die erzeugte Datei als Build-Artefakt an, sodass CMake nach einer Änderung an der `.dsp`-Datei neu übersetzt.

Für die Hauptmessung bleibt `${ARGN}` leer. Es gilt der in Abschnitt 4.4 begründete Aufruf mit `-cn` und ohne weitere Schalter. Der zusätzliche Parameter existiert nur für die Gegenprobe aus Abschnitt 5.3.2.

Zwei Punkte kommen zu den Vorgaben aus Abschnitt 4.4 hinzu. Erstens ist der Standardwert der Flag-Variablen `-O3 -march=native -ffast-math -DNDEBUG`. Das ist die stärkste Einstellung aus Tabelle 4.4. Wenn man nichts angibt, läuft die Messung mit ge-lockerten Gleitkommaregeln. Zweitens schreibt der Build eine Datei mit den tatsächlich

Programm 5.4: Einbindung des Faust-Compilers in das Build-System

```

1  function(faust_compile DSP_FILE OUT_CPP CLASS_NAME)
2  set(_extra ${ARGN})
3  add_custom_command(
4  OUTPUT ${OUT_CPP}
5  COMMAND ${FAUST_EXECUTABLE} -cn ${CLASS_NAME} ${_extra}
6  ${DSP_FILE} -o ${OUT_CPP}
7  DEPENDS ${DSP_FILE}
8  COMMENT "Faust-Kompilierung: ${DSP_FILE} -> ${OUT_CPP}"
9  VERBATIM)
10 endfunction()
11

```

verwendeten Übersetzungsbefehlen. Abschnitt 5.5 wertet diese Datei aus.

Die generierten Dateien werden nicht einzeln übersetzt. Stattdessen bindet der Wrapper sie ein. Die Faust-Bausteine sind reine Schnittstellen-Bibliotheken. Für jede FFT-Länge erstellt das Build-System aus einer Vorlage eine eigene Quellbeschreibung in einem eigenen Verzeichnis. Der Wrapper bleibt gleich. Über den Include-Pfad findet er genau die für sein Ziel erzeugte Datei. Auch die Probe-Programme für Binärgrößen- und Speichermessung entstehen aus einer einzigen Quelldatei. Header und Klassename werden über Präprozessor-Definitionen festgelegt.

5.3.2 Gegenprobe mit dem Faust-Vektor-Modus

Die Hauptmessung nutzt nur den skalaren Modus. Ein Befund lässt sich damit aber nicht klar zuordnen. Das kann daran liegen, wie Faust die Berechnung beschreibt. Es kann aber auch daran liegen, wie gut der C++-Compiler die jeweilige Codeform vektorisiert (Scaringella et al., 2003).

Das Build-System erzeugt deshalb zusätzlich eine Variante mit `-vec -vs 32`. Hier zerlegt Faust die Verarbeitung schon in Blöcke von 32 Samples. Ist diese Variante deutlich schneller, hat der C++-Compiler den voreingestellten Faust-Code nicht vektorisiert, und der Unterschied liegt an der Codeform. Sind beide gleich schnell, gibt es eine andere Ursache. Die Gegenprobe ist vorbereitet. Sie war aber nicht Teil des ausgewerteten Messlaufs. Kapitel 6 listet sie unter den offenen Punkten auf.

5.4 Ablauf der Messkette

Abschnitt 4.7 fordert, dass die neun Messreihen als eine zusammenhängende Kette und in einer festen Reihenfolge ablaufen. Das ist als ein einziges Skript umgesetzt. Tabelle 5.2 ordnet jedem Schritt die passende Messreihe zu.

Der in Abschnitt 4.7 geforderte Abbruch bei verletzter Toleranz ist als harte Sperre umgesetzt. Das Prüfprogramm gibt einen Fehlercode zurück, und das Skript stoppt, statt mit der Laufzeitmessung fortzufahren.

Drei weitere Vorkehrungen helfen dabei, die Ergebnisse nachzuvollziehen. Erstens stoppt der Ablauf schon vor dem Bauen, wenn es nicht eingetragene Änderungen gibt.

Tabelle 5.2: Schritte der Messkette und die zugehörigen Messreihen

Schritt	Inhalt	Messreihe
0	Prüfung des Arbeitsbaums und der Werkzeuge	Voraussetzung
1	Bauen und Verifikation der Flags	Voraussetzung
2	Protokollierung der Umgebung	Umgebungsprotokoll
3	Testdaten, Referenz, Genauigkeitsprüfung	Referenzausgaben, Genauigkeit
4	Laufzeit über alle Bausteine und Größen	Laufzeitmessung
5	Wiederholung unter sieben Flagsätzen	Optimierungs-Sweep
6	Binärgröße, Speicher, Allokationszähler	Binärgröße, Speicher
7	Binärgrößen-Sweep einschließlich <code>-Os</code>	Binärgröße
8	Zählung der Codezeilen	Quelltextumfang
9	Übersetzungsdauer des Faust-Compilers	Übersetzungsdauer
10	Erneute Protokollierung der Umgebung	Umgebungsprotokoll
11	Plausibilitätsprüfung der Ergebnisdateien	Voraussetzung

Ein Messstand, der keinem Commit zugeordnet ist, lässt sich nicht wiederherstellen. Eine frühere Version hat in diesem Fall nur nachgefragt, was man überspringen konnte. Die Plausibilitätsprüfung hat den betroffenen Lauf danach zurecht verworfen. Zweitens werden die Testsignale in Schritt 3 neu erzeugt und nicht einfach vorausgesetzt. So veralten sie nicht, wenn sich am Generator etwas ändert. Durch den festen Startwert bleibt das Ergebnis gleich. Drittens wird die Umgebung zu Beginn und am Ende festgehalten. So fällt auf, wenn sich der Commit-Stand während der Messung ändert.

Der in Abschnitt 4.2 erwähnte zweite Messstand ist als eigenes Programm für jeden Baustein umgesetzt. Für jede Blockgröße laufen zehn Runden mit jeweils hundert Messungen. Vor jeder Runde werden beide Codepfade mit hundert Aufrufen aufgewärmt, weil sie nach einem Wechsel kalt sind. Er dient als Gegenprobe. Die Ergebnisse fließen nicht in die berichteten Verhältnisse ein.

5.5 Sicherstellung einer fairen Vergleichsbasis

Der Vergleich ist nur aussagekräftig, wenn sich beide Varianten ausschließlich darin unterscheiden, wie der C++-Code entstanden ist. Fünf Maßnahmen stellen das sicher.

Nachweis gleicher Compilerflags. Beide Varianten werden absichtlich im selben Build-Verzeichnis mit derselben Flag-Variablen übersetzt. Das allein ist aber noch kein Beweis. Ein Prüfskript prüft deshalb in drei Schritten: Build-Typ im Cache, gesetzte Release-Flags und die wirklichen Übersetzungsbefehle jeder einzelnen Übersetzungseinheit. Nur die dritte Stufe gilt als Beweis. Wenn eine Einheit abweicht, endet die Messkette.

Gleiche Eingangsdaten. Beide Varianten lesen die gleiche Datei mit den Testsignalen aus Abschnitt 4.5.

Feste Pufferlage. GoogleBenchmark ruft die Messfunktion bei jeder Wiederholung erneut auf. Legt sie ihre Puffer lokal an, ändert sich deren Lage im Speicher von Wiederho-

lung zu Wiederholung. Bei Bausteinen, die durch die Speicherbandbreite begrenzt sind, beeinflusst dies die gemessene Zeit stark. Für die Verstärkungsstufe wurden bei einer Blockgröße von 1024 je nach Lage 33 ns und 68 ns gemessen, ein Faktor von 2,03. Rückgekoppelte Bausteine sind nicht betroffen. Bei ihnen bestimmt die Abhängigkeitskette, und nicht die Speicherbandbreite, die Laufzeit.

Alle Laufzeitmessungen beziehen ihre Puffer deshalb aus einem einzigen, an 4096 Byte ausgerichteten Bereich. Je Kombination aus Blockgröße und Kanalzahl wird der Puffersatz einmal vergeben und danach unverändert zurückgegeben. Damit ist die Lage über alle Wiederholungen konstant. Beide Varianten messen auf denselben Adressen. Jeder Block ist auf 64 Byte ausgerichtet, und aufeinanderfolgende Blöcke liegen nie um ein Vielfaches von 4096 Byte auseinander (Hennessy et al., 2025).

Die Maßnahme ist wichtig für die Auswertung. Abschnitt 4.6.2 leitet die Auflösungsgrenze aus der Verstärkungsstufe ab, also aus dem Baustein mit dem stärksten Effekt. Ohne feste Pufferlage würde diese Grenze ein Artefakt der Speicherverwaltung und nicht das Messrauschen beschreiben.

Gleicher Messrahmen. Beide Varianten werden im gleichen Prozess, im gleichen Zeitfenster und über die gleiche Template-Funktion gemessen. Neben den Vorgaben aus Abschnitt 4.2 wird die Laufzeitmessung über drei separate Prozessstarts wiederholt. So werden Effekte sichtbar, die nur bei einem einzelnen Prozess auftreten.

Getrennte Programme je Messgröße. Die Genauigkeitsprogramme werden in zwei Versionen übersetzt. Eine Version nutzt strenge Gleitkommaregeln, da hier die tatsächliche Rundung gemessen werden soll. Die andere Version verwendet die Release-Flags. So sieht man, wie sehr die Optimierung das Ergebnis verändert. Die Probe-Programme für die Binärgrößenmessung enthalten jeweils genau einen Baustein.

Zwei Unterschiede bleiben bestehen. Der Biquad-Filter liegt auf beiden Seiten in unterschiedlicher Direktform vor. Die FFT ist auf der Faust-Seite je Länge ausgerollt. Auf der C++-Seite wird sie zur Laufzeit parametrisiert. Beide Punkte ergeben sich daraus, dass jede Seite in ihrer naheliegenden Form formuliert wurde. In Kapitel 6 werden sie bei der Deutung erneut aufgegriffen.

5.6 Verifikation der Ergebnisse

Die Verifikation findet vor jeder Laufzeitmessung in zwei Schritten statt. Zuerst vergleicht sie Faust direkt mit C++. Danach werden beide mit der in Abschnitt 4.5 beschriebenen Referenz in doppelter Genauigkeit verglichen. Die Fehlermaße stehen in Abschnitt 4.2.2.

Dass die zweite Stufe nötig ist, hat sich im Verlauf der Arbeit bestätigt. Beim Vergleich der FFT ergab eine der beiden möglichen Fensterorientierungen ein exakt negatives Spektrum. Beide Implementierungen waren sich einig. Sie lagen aber gemeinsam falsch. Ohne eine dritte, unabhängig gerechnete Instanz wäre nicht entscheidbar gewesen, welche Variante der Konvention folgt.

Jeder Baustein hat eine Toleranzschwelle (Tabelle 5.3). Das Prüfprogramm speichert diese Werte, und man kann sie beim Aufruf ändern. Die Messkette nimmt die Vorgabe-

werte. Die Staffellung richtet sich nach der erwarteten Fehlerfortpflanzung. Die Verstärkungsstufe rechnet bei jedem Sample eine Multiplikation und bekommt die strengste Schwelle. Biquad und FIR summieren mehrere Produkte. Der Akkumulator summiert über die ganze Signallänge. Bei der FFT gibt es keinen direkten Vergleich, da die Referenz von der Fensterorientierung abhängt.

Tabelle 5.3: Toleranzschwellen des maximalen Absolutfehlers

Baustein	Faust gegen C++	beide gegen die Referenz
Gain	10^{-5}	10^{-3}
Akkumulator	10^{-3}	10^{-3}
Biquad-Filter	10^{-4}	10^{-3}
FIR-Filter	10^{-4}	10^{-3}
FFT	—	10^{-3}

Zwei weitere Prüfungen ergänzen den Nachweis. Ein eigenes Programm lässt den Akkumulator über viele Millionen Blöcke laufen und protokolliert seinen Zustand. So lässt sich beobachten, an welchem Punkt die Auflösung der einfachen Genauigkeit erschöpft ist. Das zeigt, dass der beobachtete Fehler eine Eigenschaft des Bausteins ist und nicht von der Implementierung kommt. Außerdem zählen die Probe-Programme über einen überladenen Speicheroperator die Anforderungen innerhalb der Verarbeitungsmethode mit. Sie schreiben das Ergebnis je Baustein und Variante in eine Ergebnisdatei.

Am Ende steht eine Plausibilitätsprüfung für alle Ergebnisdateien. Sie prüft, ob die erwarteten Dateien vorhanden sind. Außerdem stellt sie fest, ob der Commit-Stand eindeutig ist. Sie kontrolliert auch, ob die Werte in den erwarteten Größenordnungen liegen. Ein Lauf gilt erst dann als zitierfähig, wenn diese Prüfung ohne Beanstandung verläuft.

5.7 Verfügbarkeit des Quelltexts

Der komplette Messaufbau befindet sich in einem öffentlichen Repository. Es enthält Faust-Beschreibungen, Wrapper, manuelle Implementierungen, das Build-System, Messprogramme, Skripte der Messkette, Auswertungsskripte und die Ergebnisdateien des berichteten Laufs. Jeder Lauf protokolliert den Commit-Stand. So lässt sich zu jeder Zahl in Kapitel 6 der passende Quelltextstand bestimmen.

5.8 Zusammenfassung

Beide Varianten jedes Bausteins entstehen aus derselben algorithmischen Vorgabe. Sie erfüllen dieselbe Schnittstelle. Das Messprogramm verarbeitet beide auf die gleiche Weise. Die Faust-Seite umfasst je Baustein ein bis drei Zeilen. Die manuelle Seite enthält eine Header-only-Klasse mit ausformulierter Schleifen- und Indexrechnung.

Drei Entscheidungen betreffen mehr als nur das Versuchsdesign und bestimmen die Grundlage für den Vergleich. Für den FIR-Filter gibt es eine zweite manuelle Implementierung, damit man den Unterschied zwischen den Programmiersprachen von dem

Unterschied zwischen zwei C++-Varianten unterscheiden kann. Alle Puffer liegen an festen, ausgerichteten Adressen. Andernfalls hätte ihre Lage einen Einfluss, der so groß ist wie der untersuchte Unterschied. Außerdem ist der Faust-Vektor-Modus als Gegenprobe geplant. Er fließt aber nicht in die Hauptmessreihen ein.

Die Messkette führt die neun Messreihen in fester Reihenfolge aus, bricht bei einer verletzten Toleranz ab und verweigert den Start bei einem nicht eingetragenen Arbeitsbaum. Kapitel 6 berichtet über die Ergebnisse.

Kapitel 6

Evaluation der Messergebnisse

Dieses Kapitel berichtet, was gemessen wurde. Es prüft zuerst, ob die Messung gültig ist, und stellt danach die Ergebnisse je Messgröße dar. Jeder Abschnitt nennt den *Befund* in einem Satz und darunter unter *Beleg* die Stelle in den Messdaten, an der er sich ablesen lässt. Erklärungen der beobachteten Effekte und die Beantwortung der Teilfragen bleiben Kapitel 7 vorbehalten.

Alle Zahlen stammen aus einem einzigen Lauf der Messkette aus Abschnitt 5.4. Gemessen wurde mit der Voreinstellung des Build-Systems, also mit `-O3 -march=native -ffast-math`; Abschnitt 6.4 berichtet zusätzlich die übrigen sechs Konfigurationen. Die Rohdaten liegen im Repository aus Abschnitt 5.7.

6.1 Gültigkeit der Messung

6.1.1 Numerische Übereinstimmung

Befund. Die Faust-Variante und C++-Referenz liefern für alle Bausteine und Testsignale dasselbe Ergebnis im Rahmen der einfachen Gleitkommagenauigkeit. Bei Gain und Akkumulator stimmen sie bitgenau überein.

Beleg. Die Prüfung umfasst 168 Vergleiche zwischen beiden Varianten und 364 gegen die Referenz in doppelter Genauigkeit. Tabelle 6.1 zeigt die größten Fehler. Keiner überschreitet seine Toleranz aus Tabelle 5.3; alle 532 Zeilen tragen den Status „OK“. Die Fehler gegenüber der Referenz sind für beide Varianten nahezu gleich groß.

6.1.2 Keine Speicheranforderungen im Audio-Pfad

Befund und Beleg. Der Allokationszähler der Probe-Programme meldet für alle sieben Bausteine und beide Varianten, also in 14 von 14 Fällen, den Wert null.

6.1.3 Auflösungsgrenze des Messaufbaus

Befund. Die Auflösungsgrenze beträgt 2,04 Prozent. Sie wurde nicht nach dem in Abschnitt 4.6.2 vorgesehenen Verfahren bestimmt.

Tabelle 6.1: Größter beobachteter Fehler je Baustein

Baustein	Faust gegen C++ max. Absolutfehler	Faust gegen Referenz max. Fehler in ε	C++ gegen Referenz max. Fehler in ε
Gain	0	0,0	0,0
Akkumulator	0	56,1	56,1
Biquad-Filter	$1,9 \cdot 10^{-6}$	37,3	37,9
FIR-32	$6,0 \cdot 10^{-8}$	4,4	4,4
FIR-64	$4,5 \cdot 10^{-8}$	4,6	2,3
FIR-256	$3,7 \cdot 10^{-8}$	5,0	5,0
FFT	—	1,2	1,4

Beleg. Das vorgesehene Verfahren nimmt den größten Abstand $|r-1|$ der Verstärkungsstufe als Grenze. Dieser Abstand beträgt in der Messung 6,8 Prozent (Abschnitt 6.2). Verwendet wurde stattdessen ein Nullvergleich: Jede Messreihe wird in ihre erste und zweite Hälfte geteilt und aus beiden das Verhältnis der Mediane gebildet, sodass derselbe Code unter denselben Bedingungen gegen sich selbst antritt. Aus 81 solchen Vergleichen ergibt sich ein Median von 0,57 Prozent, ein 95. Perzentil von 2,04 Prozent und ein Maximum von 7,26 Prozent. Als Grenze dient das 95. Perzentil.

6.1.4 Stabilität und Signifikanz

Befund. Alle 33 Messpunkte sind eingipfelig. Die p-Werte des Signifikanztests liegen fast durchgehend am unteren Anschlag der Gleitkommadarstellung.

Beleg. Die Stabilitätsprüfung meldet für keinen Messpunkt eine zweigipfelige Verteilung. Der relative Interquartilsabstand bleibt bei allen Bausteinen außer der Verstärkungsstufe unter 4,5 Prozent; für die Verstärkungsstufe erreicht er 9,6 Prozent bei einem Ausreißeranteil von 7,7 Prozent. Bei 31 der 33 Messpunkte beträgt der p-Wert des Mann-Whitney-U-Tests $1,05 \cdot 10^{-99}$. Auch der Akkumulator, dessen Unterschied innerhalb der Auflösungsgrenze liegt, erreicht p-Werte um 10^{-24} .

6.2 Laufzeit der Bausteine

Befund. Der Laufzeitunterschied fällt je Baustein sehr unterschiedlich aus und reicht von -49 bis $+17$ Prozent.

Beleg. Abbildung 6.1 zeigt alle 33 Messpunkte als Verhältnis r aus dem Median der Faust-Variante und dem der C++-Referenz; Tabelle 6.2 fasst sie je Baustein zusammen.

Im Einzelnen: Bei der *Verstärkungsstufe* ist die Faust-Variante bei $N = 64, 128$ und 256 um 6,8, 4,0 und 5,3 Prozent langsamer; bei $N = 512$ und 1024 enthält das Konfidenzintervall den Wert eins. Beim *Akkumulator* liegt das Verhältnis bei allen Blockgrößen innerhalb der Auflösungsgrenze. Beim *Biquad-Filter* ist die Faust-Variante bei allen Blockgrößen um rund 38 Prozent schneller. Bei den *FIR-Filtern* ist sie gegenüber der Ringpuffer-Variante zwischen 46 und 49 Prozent schneller. Bei der *FFT* wechselt das

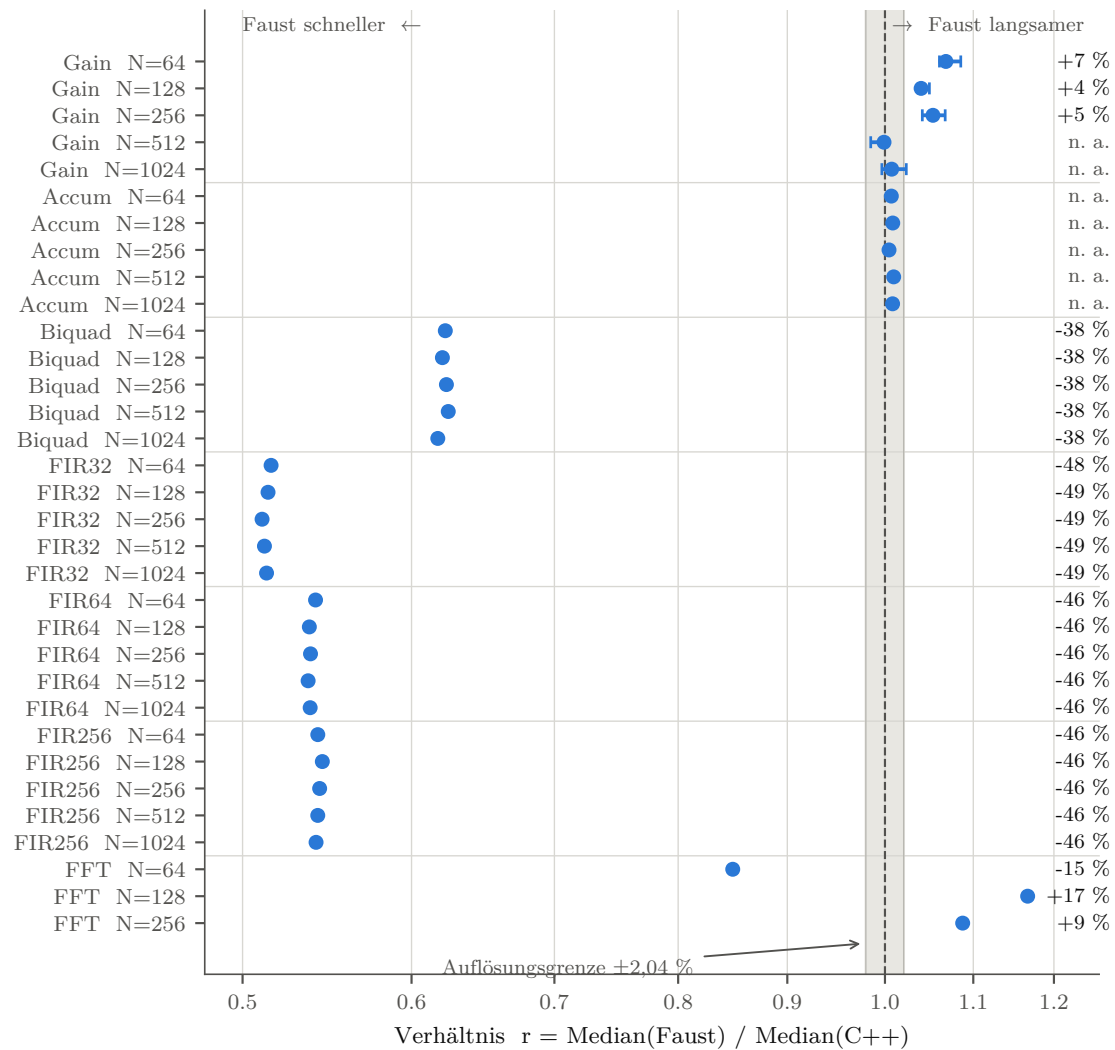


Abbildung 6.1: Laufzeitverhältnis für alle 33 Messpunkte mit 95-Prozent-Konfidenzintervall. Das graue Band ist die Auflösungsgrenze; Werte links der gestrichelten Linie bedeuten, dass die Faust-Variante schneller ist.

Vorzeichen: bei $N = 64$ ist sie 15,1 Prozent schneller, bei $N = 128$ um 16,6 und bei $N = 256$ um 8,7 Prozent langsamer; die Konfidenzintervalle dieser drei Punkte überlappen nicht.

6.3 Skalierungsverhalten

6.3.1 Blockgröße

Befund. Die Rechenzeit je Sample bleibt über die Blockgröße nahezu konstant; das Verhältnis zwischen beiden Varianten ändert sich mit der Blockgröße nicht.

Tabelle 6.2: Laufzeitverhältnis je Baustein über alle Blockgrößen

Baustein	Verhältnis r	Unterschied	Urteil
Gain	0,999–1,068	−0,1 bis +6,8 %	uneinheitlich
Akkumulator	1,004–1,009	+0,4 bis +0,9 %	nicht auflösbar
Biquad-Filter	0,617–0,624	−38,3 bis −37,6 %	Faust schneller
FIR-32	0,511–0,516	−48,9 bis −48,4 %	Faust schneller
FIR-64	0,537–0,541	−46,3 bis −45,9 %	Faust schneller
FIR-256	0,541–0,545	−45,9 bis −45,5 %	Faust schneller
FFT	0,849–1,166	−15,1 bis +16,6 %	uneinheitlich

Beleg. Abbildung 6.2 trägt die Rechenzeit je Sample über die Blockgröße auf. Zwischen $N = 64$ und $N = 1024$ ändert sie sich beim Akkumulator um $-0,5$ Prozent, beim Biquad-Filter um $-0,8$, bei FIR-32 um $-0,4$ und bei FIR-256 um weniger als $0,1$ Prozent. Die Verstärkungsstufe weicht mit $-8,0$ Prozent ab. Die Spannweite des Verhältnisses je Baustein in Tabelle 6.2 beträgt außer bei Gain und FFT höchstens $0,7$ Prozentpunkte.

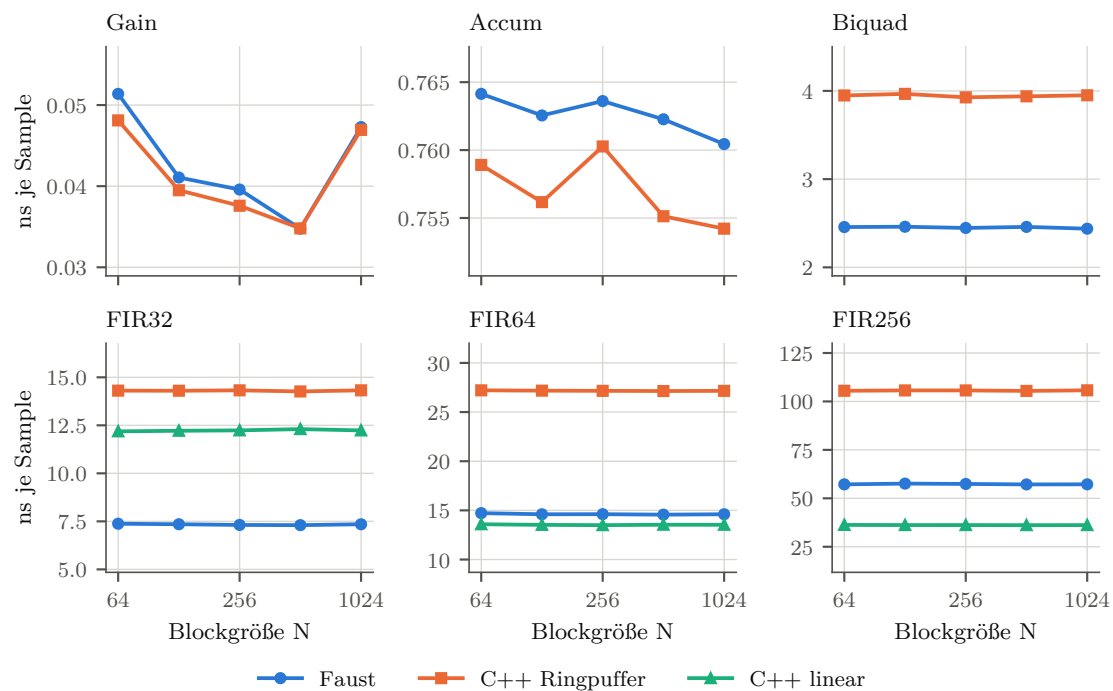


Abbildung 6.2: Rechenzeit je Sample über der Blockgröße. Waagerechte Verläufe bedeuten, dass der Aufwand linear mit der Blockgröße wächst.

6.3.2 Filterlänge

Befund. Gegenüber der Ringpuffer-Variante bleibt das Verhältnis über alle Filterlängen nahezu gleich. Gegenüber der linear adressierten Variante steigt es mit der Filterlänge und überschreitet zwischen $L = 32$ und $L = 64$ den Wert eins.

Beleg. Tabelle 6.3 und Abbildung 6.3 zeigen beide Vergleiche. Gegen den Ringpuffer liegt das Verhältnis bei 0,51, 0,54 und 0,54. Gegen den linearen Puffer liegt es bei 0,60, 1,08 und 1,58.

Tabelle 6.3: FIR-Filter: drei Implementierungen bei Blockgröße 1024

Filterlänge	CPU-Zeit je Block [μ s]			Verhältnis Faust zu	
	Faust	C++ Ring	C++ linear	Ring	linear
$L = 32$	7,53	14,67	12,53	0,51	0,60
$L = 64$	14,96	27,81	13,86	0,54	1,08
$L = 256$	58,60	108,27	37,05	0,54	1,58

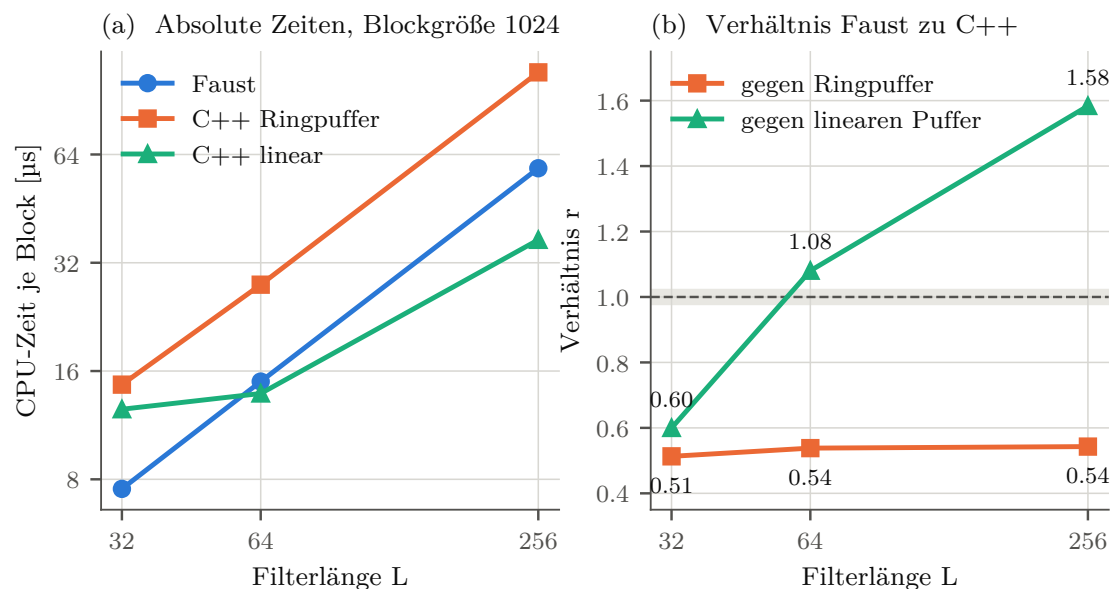


Abbildung 6.3: Links die absoluten Zeiten: Die Kurve der linear adressierten C++-Variante kreuzt die Faust-Kurve zwischen $L = 32$ und $L = 64$. Rechts dasselbe als Verhältnis.

6.3.3 Transformationslänge

Befund und Beleg. Von $N = 64$ auf $N = 256$ steigt die Zeit der Faust-Variante von 28,0 auf 850,3 Mikrosekunden, also um den Faktor 30,3. Die manuelle Implementierung steigt im selben Bereich von 33,0 auf 781,9 Mikrosekunden, also um den Faktor 23,7.

6.4 Compilerkonfigurationen

Befund. Das Verhältnis hängt stark vom Flagsatz ab. Bei drei der sechs Bausteine wechselt sein Vorzeichen über die sieben Konfigurationen.

Beleg. Abbildung 6.4 zeigt das Verhältnis über alle Flagsätze bei Blockgröße 1024. Beim Biquad-Filter reicht die Spanne von 1,48 unter `-Os` bis 0,62 unter `-O3 -march=native -ffast-math`. Bei FIR-256 liegt das Verhältnis gegen die lineare Variante unter `-O3` bei 0,86 und unter voller Optimierung bei 1,61. Bei der Verstärkungsstufe beträgt es unter `-O0` 0,44 und ab `-O1` zwischen 0,99 und 1,09.

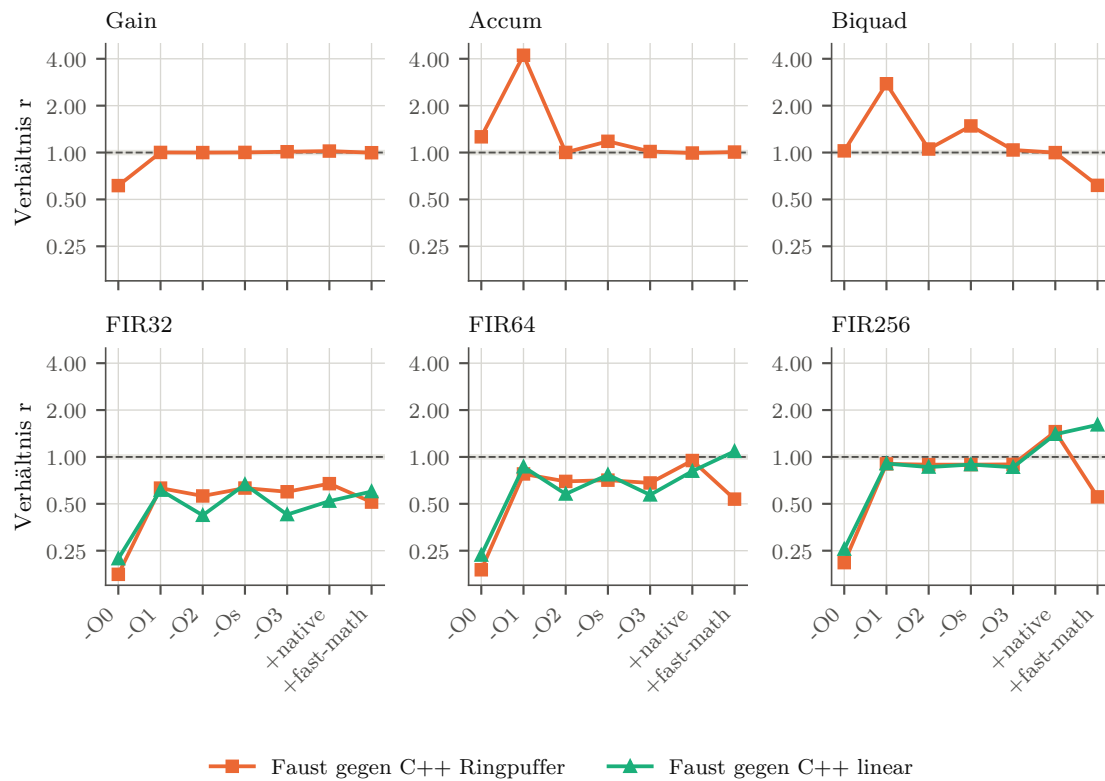


Abbildung 6.4: Laufzeitverhältnis über die sieben untersuchten Flagsätze, Blockgröße 1024.

Biquad-Filter im Detail. Abbildung 6.5 zeigt die absoluten Zeiten. Von `-O2` bis `-O3 -march=native -ffast-math` bleibt die Faust-Variante zwischen 2,35 und 2,48 Mikrosekunden je Block. Die C++-Referenz liegt über denselben Flagsätze zwischen 2,24 und 2,48 Mikrosekunden, steigt jedoch im letzten Schritt auf 4,03 Mikrosekunden.

FIR-256 im Detail. Beim Übergang von `-O3` zur vollen Optimierung sinkt die Zeit bei Blockgröße 1024 von 202,3 auf 36,9 Mikrosekunden bei der linearen Variante (Faktor 5,5), von 173,7 auf 59,4 Mikrosekunden bei Faust (Faktor 2,9) und von 193,9 auf 107,2 Mikrosekunden beim Ringpuffer (Faktor 1,8). Unter `-march=native` ohne `-ffast-math` steigt die Zeit der Faust-Variante dagegen von 173,7 auf 278,0 Mikrosekunden, während beide C++-Varianten innerhalb von 2 Prozent unverändert bleiben.

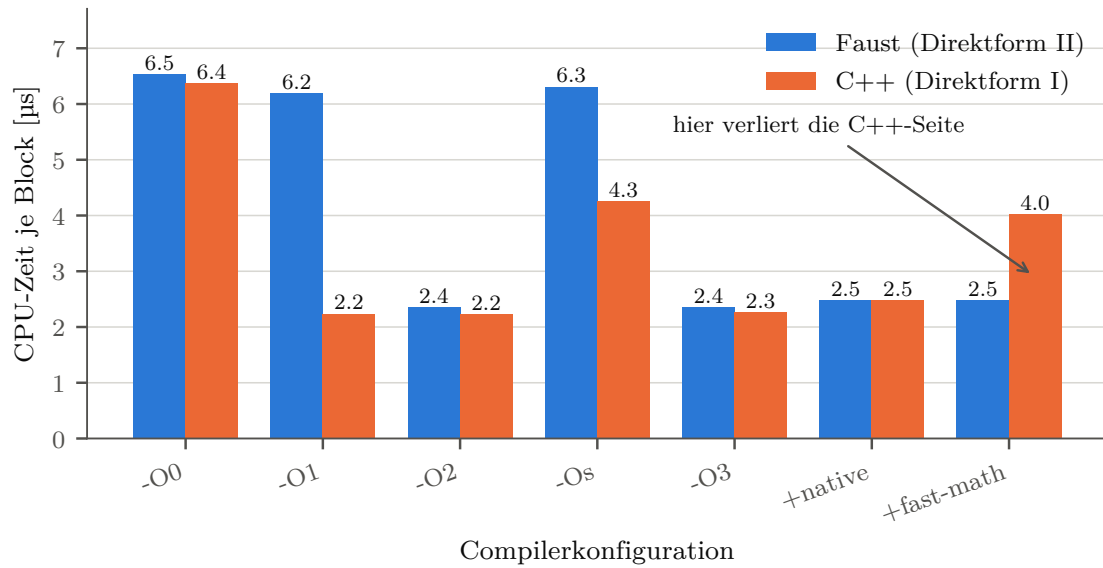


Abbildung 6.5: Absolute Rechenzeit des Biquad-Filters bei Blockgröße 1024.

6.5 Speicherbedarf und Binärgröße

Befund. Die Zustandsgröße der Faust-Klassen ist größer, der Gesamtspeicherbedarf beider Varianten praktisch gleich. Der erzeugte Code ist bei Faust in jeder Konfiguration größer.

Beleg Speicher. Abbildung 6.6 zeigt die Zustandsgrößen. Bei FIR-256 belegt die Faust-Klasse 1048 Byte im Objekt, die manuelle 64 Byte. Der maximale Speicherbedarf des Prozesses liegt für alle Bausteine außer der FFT bei rund 3,8 Megabyte und unterscheidet sich zwischen beiden Varianten um weniger als 3 Prozent; bei der FFT liegt er bei 4,4 beziehungsweise 4,7 Megabyte.

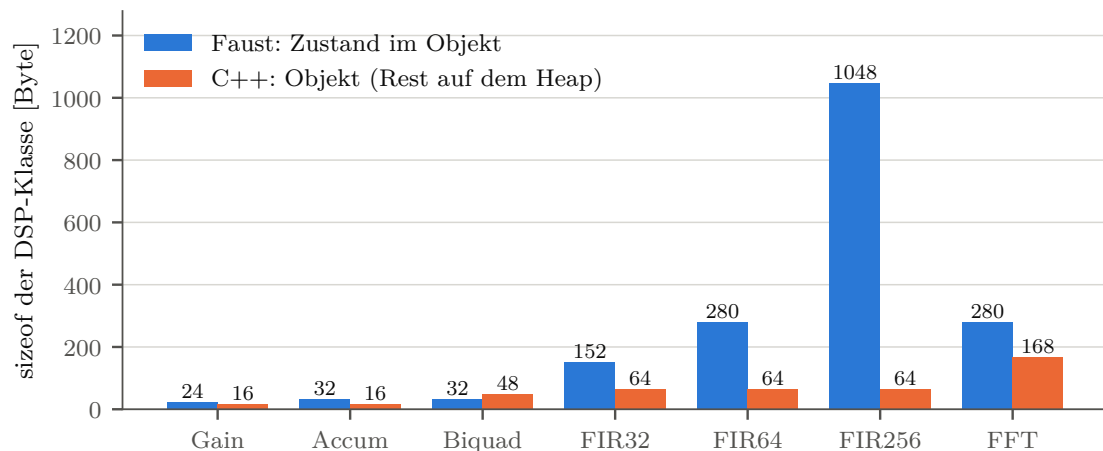


Abbildung 6.6: Größe der DSP-Klasse in Byte.

Beleg Binärgröße. Abbildung 6.7 zeigt das Verhältnis der Codegrößen über fünf Konfigurationen. Bei den kleinen Bausteinen liegt es zwischen 1,1 und 1,8, bei FIR-256 zwischen 2,8 und 4,5, bei der FFT zwischen 2,6 und 7,2. Die höchsten Werte für FIR-256 und FFT treten unter `-Os` auf. Gemessen ist die Größe des Codeabschnitts abzüglich einer Basislinie mit leerer `main`-Funktion.

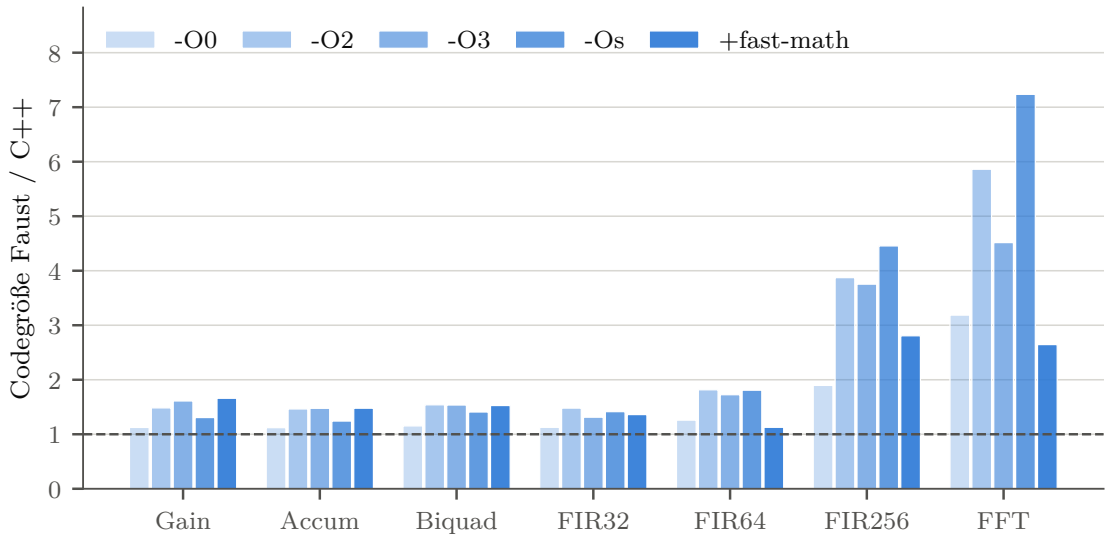


Abbildung 6.7: Verhältnis der Codegröße von Faust zu C++ über fünf Compilerkonfigurationen.

6.6 Codeumfang

Befund und Beleg. Die Faust-Beschreibung ist bei jedem Baustein kürzer als die manuelle Implementierung, das Verhältnis reicht von 9:1 bis 31:1. Zählt man den Wrapper hinzu, liegt es bei Gain und Akkumulator unter eins. Tabelle 6.4 und Abbildung 6.8 zeigen beide Zählweisen.

Tabelle 6.4: Codezeilen je Baustein

Baustein	Faust	Wrapper	Faust gesamt	manuell	nur Beschreibung
Gain	1	17	18	17	17,0
Akkumulator	2	17	19	18	9,0
Biquad-Filter	2	17	19	23	11,5
FIR (je Länge)	3	4	7	37	12,3
FFT	3	17	20	92	30,7

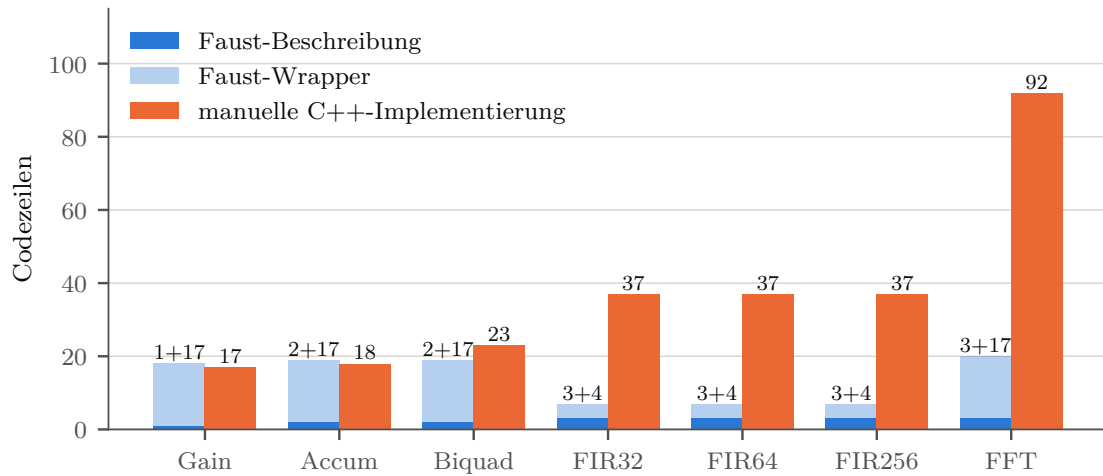


Abbildung 6.8: Codezeilen je Baustein. Die linke Säule trennt die Faust-Beschreibung vom Wrapper, die rechte zeigt die manuelle Implementierung.

6.7 Übersetzungsdauer des Faust-Compilers

Befund und Beleg. Die Übersetzungsdauer wächst überproportional mit der Problemgröße; eine Kurvenanpassung bevorzugt in allen drei untersuchten Mustern ein exponentielles Modell gegenüber einem Potenzgesetz. Für die FFT steigt sie von 0,28 Sekunden bei $N = 64$ über 1,70 Sekunden bei $N = 128$ auf 18,75 Sekunden bei $N = 256$; bei $N = 512$ greift der Abbruch nach 30 Sekunden. Für eine Rekursionskette wird die Abbruchgrenze bei Tiefe 1024 erreicht, für eine Baumstruktur bei Tiefe 16. Die fünf tatsächlich verwendeten Beschreibungen übersetzen sämtlich in unter einer halben Sekunde.

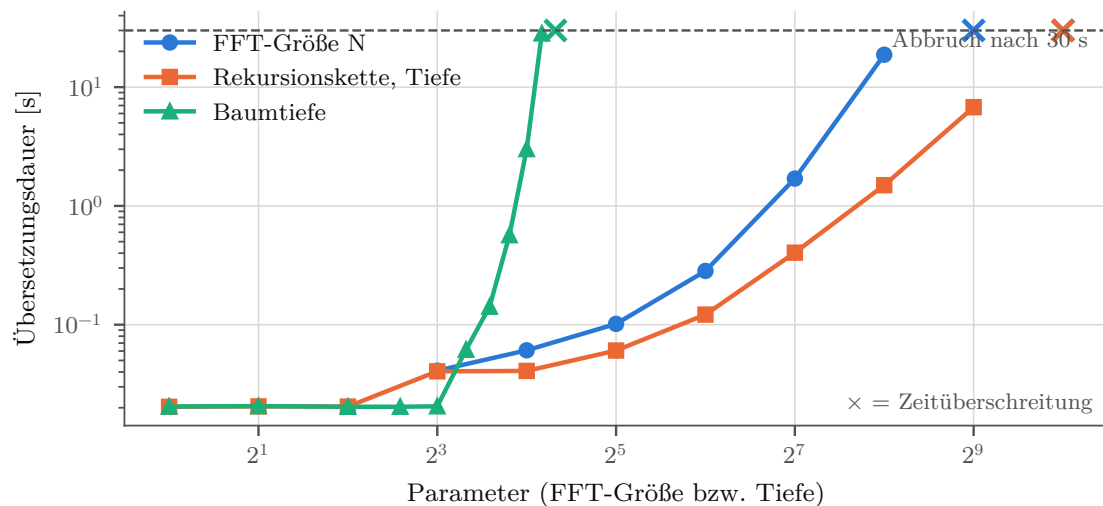


Abbildung 6.9: Übersetzungsdauer über der Problemgröße, beide Achsen logarithmisch. Kreuze markieren Läufe, die nach 30 Sekunden abgebrochen wurden.

6.8 Zusammenfassung der Messergebnisse

Die Messung ist gültig: Beide Varianten berechnen dasselbe, keine fordert im Audio-Pfad Speicher an, und alle Messreihen sind eingipfelig. Die Auflösungsgrenze beträgt 2,04 Prozent und wurde über einen Nullvergleich statt über die Verstärkungsstufe bestimmt.

Die Laufzeitunterschiede reichen von -49 Prozent bei FIR-32 bis $+17$ Prozent bei der FFT mit $N = 128$. Beim Akkumulator ist kein Unterschied auflösbar. Beim FIR-Filter hängt das Verhältnis davon ab, gegen welche der beiden C++-Varianten verglichen wird: gegen den Ringpuffer bleibt es bei etwa 0,53, gegen den linearen Puffer steigt es von 0,60 auf 1,58.

Über die Blockgröße ist der Aufwand linear und das Verhältnis konstant. Über die Compilerkonfigurationen ist es das nicht: Bei drei von sechs Bausteinen wechselt es das Vorzeichen, beim Biquad-Filter zwischen 1,48 und 0,62.

Bei den übrigen Messgrößen sind die Befunde einheitlich. Der Speicherbedarf beider Varianten ist praktisch gleich, die Zustandsgröße der Faust-Klassen jedoch größer. Der erzeugte Code ist bei Faust in jeder Konfiguration größer, bei FFT und FIR-256 um ein Mehrfaches. Die Faust-Beschreibung ist um Faktor 9 bis 31 kürzer, mit Wrapper gerechnet bei zwei Bausteinen jedoch länger. Die Übersetzungsdauer wächst exponentiell und erreicht bei $N = 256$ knapp 19 Sekunden.

Kapitel 7 ordnet diese Ergebnisse ein und beantwortet damit die Teilfragen.

Kapitel 7

Diskussion

Kapitel 6 hat berichtet, was gemessen wurde. Dieses Kapitel ordnet die Ergebnisse ein: Es erklärt die beobachteten Effekte, beantwortet damit die Teilfragen aus Abschnitt 1.1 und benennt anschließend, was der Aufbau nicht leisten kann.

7.1 Interpretation der Ergebnisse im Hinblick auf die Forschungsfrage

7.1.1 Der Laufzeitvergleich hängt an der Wahl der Referenz

Das auffälligste Ergebnis der Messung ist kein Wert, sondern eine Abhängigkeit. Beim FIR-Filter liefert derselbe Faust-Code zwei gegensätzliche Urteile, je nachdem, welche C++-Implementierung als Vergleich dient: Gegen die Ringpuffer-Variante ist er bei allen Filterlängen rund 46 bis 49 Prozent schneller, gegen die linear adressierte Variante bei $L = 256$ dagegen 58 Prozent langsamer (Tabelle 6.3).

Beide C++-Varianten berechnen dasselbe und sind beide idiomatisch geschrieben. Sie unterscheiden sich allein darin, wie sie den Verlaufspuffer adressieren. Der Ringpuffer berechnet seinen Index über eine Bitmaske; für den Compiler sieht das aus wie ein Zugriff auf verstreute Adressen, und er vektorisiert die Schleife nicht. Die zweite Variante hält den Zustand doppelt und adressiert dadurch fortlaufend, was sich in SIMD-Befehle übersetzen lässt. Wie stark dieser Unterschied wiegt, zeigt der Optimierungs-Sweep: Beim Übergang von `-O3` zur vollen Optimierung wird die lineare Variante um den Faktor 5,5 schneller, die Faust-Variante um 2,9 und der Ringpuffer nur um 1,8 (Abschnitt 6.4).

Daraus folgt eine methodische Einsicht, die über diese Arbeit hinausreicht. Ein Vergleich zwischen einer Sprache und einer handgeschriebenen Referenz misst immer beides zugleich: die Qualität der Codegenerierung und die Qualität der gewählten Referenzformulierung. Ohne die zweite C++-Variante wäre diese Arbeit zu dem Schluss gekommen, Faust sei bei FIR-Filtern durchgehend rund 46 Prozent schneller. Dieser Schluss wäre falsch gewesen: Er hätte eine Eigenschaft der Referenz als Eigenschaft der Sprache ausgewiesen. Eine einzelne Referenzimplementierung reicht deshalb nicht aus, um eine Aussage über eine Sprache zu stützen.

7.1.2 Die Compilerkonfiguration wiegt schwerer als die Sprachwahl

Bei drei der sechs Bausteine wechselt das Verhältnis über die sieben Flagsätze sein Vorzeichen (Abbildung 6.4). Am deutlichsten ist das beim Biquad-Filter, und dort lässt sich die Ursache genau benennen.

Die absoluten Zeiten in Abbildung 6.5 zeigen, dass die Faust-Variante von -02 bis zur vollen Optimierung nahezu unverändert bei rund 2,4 Mikrosekunden je Block bleibt. Die C++-Referenz liegt über dieselben Flagsätze zunächst gleichauf und steigt erst im letzten Schritt auf 4,03 Mikrosekunden. Der Vorsprung der Faust-Seite entsteht also nicht dadurch, dass Faust schneller wird, sondern dadurch, dass die C++-Referenz langsamer wird.

Das entspricht genau der Vorhersage, die Abschnitt 4.4 aus der Literatur abgeleitet hat. `-ffast-math` aktiviert unter anderem `-fassociative-math` und erlaubt dem Compiler damit, Gleitkommaadditionen frei umzuordnen (*Using the GNU Compiler Collection (GCC), Version 13.3.0*, 2024). Bei rückgekoppelten Filtern kann das die Abhängigkeitskette verlängern statt sie zu verkürzen (Parhi, 1999). Beide Seiten sind davon unterschiedlich betroffen, weil sie unterschiedliche Direktformen verwenden: Die Direktform I der C++-Referenz führt zwei getrennte Verzögerungsketten und bietet dem Compiler mehr Spielraum zum Umordnen, die Direktform II der Faust-Variante kommt mit einer Kette aus. Damit ist eine aus der Literatur übernommene Behauptung an eigenen Messdaten bestätigt — und der auffälligste Einzelbefund der Arbeit ist kein Argument für Faust.

Beim FIR-Filter wirkt derselbe Flagsatz in die andere Richtung. Die Reihenfolge der Beschleunigungsfaktoren — lineare C++-Variante vor Faust vor Ringpuffer — entspricht der Erwartung, dass sich vektorisierbare Ausdrücke von solchen mit Datenabhängigkeiten trennen lassen und nur die erste Gruppe von der automatischen Vektorisierung profitiert (Scaringella et al., 2003).

Eine Beobachtung bleibt unerklärt: Unter `-march=native` ohne `-ffast-math` wird die Faust-Variante von FIR-256 um 60 Prozent langsamer, während beide C++-Varianten unverändert bleiben. Aus den vorliegenden Daten lässt sich dafür keine Ursache ableiten; eine Untersuchung des erzeugten Maschinencodes wäre nötig.

7.1.3 Skalierungsverhalten

Über die Blockgröße verhalten sich beide Varianten gleich. Die Rechenzeit je Sample bleibt konstant, der Aufwand wächst also linear, und das Verhältnis ändert sich nicht (Abbildung 6.2). Die einzige Ausnahme ist die Verstärkungsstufe mit -8,0 Prozent zwischen $N = 64$ und $N = 1024$. Sie hat den größten Anteil an blockweisem Festaufwand, und genau dieser Festaufwand erklärt auch ihren Laufzeitunterschied: Die Faust-Variante liest den Verstärkungsfaktor als Signal aus dem Objekt, weil er in der Beschreibung als Bedienelement angelegt ist, während die manuelle Implementierung ihn als konstantes Feld hält. Bei rund drei Nanosekunden je Block fällt dieser eine zusätzliche Zugriff ins Gewicht; mit wachsender Blockgröße verteilt er sich und der Unterschied verschwindet.

Beim Akkumulator ist kein Unterschied auflösbar. Das ist ein inhaltliches Ergebnis und kein Nullbefund: Der Baustein wurde aufgenommen, um zwei mögliche Ursachen für den Biquad-Unterschied zu trennen. Weil die reine Rückkopplung auf beiden Seiten

gleich viel kostet, kann der Biquad-Unterschied nicht aus ihr stammen — was mit der Erklärung über die Direktformen aus Abschnitt 7.1.2 zusammenpasst.

Bei der FFT wechselt das Vorzeichen zwischen $N = 64$ und $N = 128$. Der Faust-Compiler rollt die Butterfly-Struktur vollständig aus; für $N = 64$ sind das 192 Butterflies je Ausgangssample, für $N = 256$ bereits 1024. Die naheliegende Erklärung ist, dass der ausgerollte Code die Kapazität des Befehls-Zwischenspeichers überschreitet, während die manuelle Implementierung unabhängig von N dieselbe kleine Schleife ausführt. Die Binärgrößen stützen das, ein direkter Nachweis über Hardware-Zähler fehlt jedoch; Abschnitt 7.3 greift diese Einschränkung auf.

7.1.4 Speicherbedarf und Binärgröße

Beim Speicher ist der Befund eindeutig und wenig überraschend: Beide Varianten fordern im Audio-Pfad keinen Speicher an, und ihr Gesamtbedarf unterscheidet sich um weniger als 3 Prozent. Die auffällige Asymmetrie der Zustandsgrößen ist eine Frage der Ablage, nicht des Verbrauchs. Faust hält den Zustand im Objekt, die manuellen Implementierungen von FIR und FFT fordern ihn beim Anlegen des Objekts an. Wer nur `sizeof` betrachtet, hält die Faust-Seite für die speicherhungrigere. Für den Echtzeitbetrieb ist die Ablage im Objekt sogar die günstigere Form, weil sie ohne Allokator auskommt. Die Angabe aus der Literatur, Faust erzeuge Code mit konstantem und deterministischem Speicherverbrauch (Orlarey et al., 2009), ist damit bestätigt — gilt hier aber ebenso für die manuellen Implementierungen.

Bei der Binärgröße ist die Spalte `-Os` in Abbildung 6.7 die aussagekräftigste. Diese Option weist den C++-Compiler an, auf Codegröße zu optimieren; trotzdem erreicht das Verhältnis dort bei FIR-256 mit 4,5 und bei der FFT mit 7,2 die höchsten Werte überhaupt. Der C++-Compiler kann den bereits ausgerollten Faust-Code also nicht wieder zu einer Schleife zusammenfassen. Damit ist belegt, dass das Ausrollen eine Eigenschaft des von Faust erzeugten Quelltexts ist und keine Entscheidung des nachgelagerten Compilers.

Dieselbe Eigenschaft erklärt die Übersetzungsdauer: Was sich in der Binärgröße als gewachsener Code niederschlägt, muss zuvor erzeugt werden. Für die praktische Arbeit bedeutet beides zusammengekommen, dass große Filter und Transformationen in Faust einen Preis haben, der sich nicht wegoptimieren lässt — an Programmspeicher und an Wartezeit beim Übersetzen.

7.1.5 Codelänge und Implementierungskomplexität

Der Umfangsvorteil der Faust-Beschreibung skaliert mit der Komplexität des Bausteins: Eine Multiplikation je Sample lässt sich in C++ genauso kurz hinschreiben wie in Faust, eine Radix-2-FFT mit Bit-Umkehrtabelle, Twiddle-Faktoren und Verlaufspuffer nicht. Der Größenordnung nach deckt sich das mit dem für physikalische Modelle berichteten Vorteil von 22,91 bis 80,20 Prozent (Michon & Smith, 2011) und weitet ihn auf elementare Bausteine aus.

Die Trennung vom Wrapper ist dabei keine Formalie. Zählt man ihn mit, ist die Faust-Seite bei Gain und Akkumulator sogar länger. Der Wrapper ist allerdings für alle Bausteine nahezu gleich und einmal wiederverwendbar — das gemeinsame FIR-Template kommt mit vier Zeilen aus. Wer Faust für ein einzelnes triviales Modul ein-

setzt, gewinnt nichts; wer es für viele oder für komplexe Module einsetzt, gewinnt deutlich.

Codezeilen messen jedoch den Umfang und nicht den Aufwand. Drei Beobachtungen aus der Umsetzung ergänzen das Bild und lassen sich nicht in Zahlen fassen.

Die Beschreibung ist kurz, die Einarbeitung nicht. Eine Zeile wie `process = fi.tf2(...)` setzt voraus, dass man weiß, welche Direktform `fi.tf2` erzeugt und in welcher Reihenfolge die Koeffizienten erwartet werden. Beides steht nicht im Programm. Der Aufwand verschiebt sich vom Schreiben zum Nachschlagen.

Die erzeugte Form ist nicht frei wählbar. Die Direktform I ließ sich mit `fi.tf2` nicht ausdrücken. Wer aus numerischen oder strukturellen Gründen eine bestimmte Form braucht, muss sie in Faust von Hand nachbauen und verliert dabei genau den Vorteil, der die Sprache attraktiv macht.

Konventionen sind nicht sichtbar. Dass `an.rtcv` das Analysefenster mit dem neuesten Sample zuerst serialisiert, war weder der Beschreibung noch der Dokumentation zu entnehmen. Aufgefallen ist es erst durch den Vergleich gegen die unabhängig gerechnete Referenz, wo sich ein exakt negiertes Spektrum zeigte. Beide Implementierungen waren sich einig und lagen gemeinsam falsch — ein Fehler, der ohne dritte Instanz unentdeckt geblieben wäre.

Dem steht gegenüber, dass die Integration unkompliziert ist: Der erzeugte Code ist eine gewöhnliche C++-Klasse ohne Laufzeitabhängigkeiten, und die Einbindung in ein bestehendes CMake-Projekt kostete eine Funktion von zehn Zeilen.

7.1.6 Antworten auf die Teilfragen

Tabelle 7.1 fasst zusammen, was sich aus den Messergebnissen für jede Teilfrage ableiten lässt.

7.1.7 Antwort auf die übergeordnete Forschungsfrage

Die Forschungsfrage lautet, wie sich Faust-generierter C++-Code von einer manuell implementierten C++-Referenz bei ausgewählten DSP-Bausteinen und FFTs hinsichtlich Laufzeit, Speicherverbrauch, Binärgröße, Codelänge und Implementierungskomplexität unterscheidet.

Für die Laufzeit lautet die Antwort: Es gibt keinen stabilen Unterschied, der sich der Sprache zuschreiben ließe. Die gemessenen Verhältnisse reichen von 0,51 bis 1,17, aber sie hängen an zwei Faktoren, die nichts mit der Codegenerierung zu tun haben — an der Compilerkonfiguration und an der Wahl der C++-Referenzformulierung. Wo beide Faktoren festgehalten werden, liegen die Varianten bei den einfachen Bausteinen gleichauf; die verbleibenden Unterschiede lassen sich auf konkrete strukturelle Ursachen zurückführen, nicht auf eine allgemeine Überlegenheit einer Seite.

Für die übrigen Größen ist die Antwort eindeutiger, weil die Befunde über alle Compilerkonfigurationen stabil bleiben. Der Speicherverbrauch ist praktisch gleich. Der Binärcode ist bei Faust durchgängig größer, bei großen Filtern und Transformationen

Tabelle 7.1: Teilfragen und ihre Beantwortung

Teilfrage	Antwort	Grundlage
Performanzunterschied je Baustein	Kein einheitlicher Befund. Gain und Akkumulator gleichauf, Biquad und FIR mit deutlichem Vorsprung für Faust, FFT je nach Größe in beide Richtungen. Die Vorsprünge gelten nur für die verwendete Compilerkonfiguration und die gewählte Referenzformulierung.	6.1, 6.2
Abhängigkeit von Blockgröße, Filterlänge und FFT-Größe	Von der Blockgröße unabhängig: Der Aufwand wächst linear, das Verhältnis bleibt konstant. Von der Filterlänge abhängig, aber nur im Vergleich zur vektorisierbaren C++-Variante. Von der FFT-Größe stark abhängig, mit Vorzeichenwechsel zwischen $N = 64$ und $N = 128$.	6.2, 6.3
Speicherverbrauch	Kein praktischer Unterschied. Beide fordern im Audio-Pfad keinen Speicher an; der Zustand liegt lediglich an unterschiedlicher Stelle.	6.6
Binärgröße	Faust erzeugt durchgängig größeren Code, bei großen Filtern und Transformationen um ein Mehrfaches. Der Abstand bleibt auch unter <code>-Os</code> bestehen und ist damit eine Eigenschaft des erzeugten Quelltexts.	6.7
Compileroptimierung und Vektorisierung	Der stärkste Einflussfaktor überhaupt. Drei von sechs Bausteinen wechseln das Vorzeichen ihres Unterschieds über die Flagsätze; beim Biquad-Filter ist die Ursache auf <code>-ffast-math</code> und die Direktform der Referenz zurückführbar.	6.4, 6.5
Codelänge und Implementierungskomplexität	Die Beschreibung ist um Faktor 9 bis 31 kürzer, der Vorteil wächst mit der Komplexität des Bausteins. Mit Wrapper gerechnet verschwindet er bei trivialen Bausteinen. Der Aufwand verschiebt sich vom Schreiben zum Nachschlagen.	6.8, 6.4

um ein Mehrfaches, und das Ausrollen lässt sich nicht wegoptimieren. Die Beschreibung ist erheblich kürzer, sofern der wiederverwendbare Wrapper getrennt gezählt wird.

Praktisch heißt das: Faust ist bei diesen Bausteinen keine Frage der Geschwindigkeit. Es ist eine Abwägung zwischen einer deutlich kürzeren Beschreibung auf der einen und größerem Binärcode, längerer Übersetzungsdauer sowie geringerer Kontrolle über die erzeugte Struktur auf der anderen Seite.

7.2 Limitationen des Versuchsaufbaus

Fünf Einschränkungen begrenzen die Reichweite der Ergebnisse.

Eine einzige Plattform. Alle Messungen laufen auf einem AMD Ryzen 7 4700U unter WSL2 mit g++ 13.3.0. Aussagen über andere Prozessorarchitekturen, andere Betriebssysteme oder andere C++-Compiler lassen sich daraus nicht ableiten. Gerade weil sich der Einfluss der Compilerkonfiguration als dominierend erwiesen hat, ist zu erwarten, dass ein anderer Compiler zu anderen Verhältnissen führt.

Nur der skalare Faust-Modus. Die Hauptmessung verwendet ausschließlich das voreingestellte C++-Backend. Die in Abschnitt 5.3.2 vorbereitete Gegenprobe mit dem Vektor-Modus war nicht Teil des ausgewerteten Laufs. Damit bleibt offen, ob sich der Rückstand der Faust-Variante bei FIR-256 unter voller Optimierung durch eine andere Codegenerierungsstrategie aufholen ließe. Der Parallel-Modus wurde ebenfalls nicht untersucht.

Einzelne Bausteine statt einer Signalkette. Gemessen werden fünf isolierte Bausteine. Effekte, die erst durch die Verkettung mehrerer Bausteine entstehen — etwa die Wiederverwendung von Zwischenergebnissen über eine ganze Verarbeitungskette hinweg, für die der Faust-Compiler den vollständigen Signalfluss analysieren kann — bleiben unsichtbar. Genau dort könnte der Ansatz seine Stärke haben.

Eine nicht handoptimierte Referenz. Die manuellen Implementierungen sind im Rahmen dieser Arbeit entstanden. Für den FIR-Filter liegen zwei Formulierungen vor, für die übrigen Bausteine jeweils eine. Abschnitt 7.1.1 zeigt, wie stark die Wahl der Formulierung das Ergebnis verschiebt; für die Bausteine mit nur einer Referenz ist dieselbe Unsicherheit anzunehmen, ohne dass sie sich beziffern ließe.

Eine praxisferne FFT-Form. Beide Varianten berechnen eine gleitende Fenster-FFT, also für jedes Sample eine vollständige Transformation. Diese Form ergibt sich aus der Semantik der verwendeten Faust-Funktion und nicht aus einer praktischen Anforderung; übliche Anwendungen arbeiten mit Blockverarbeitung und Fenstervorschub. Die untersuchten Transformationslängen sind zudem klein.

7.3 Bedrohung der Validität

7.3.1 Interne Validität

Die Systemlast lag über dem Richtwert. Das Umgebungsprotokoll weist zu Beginn der Messung eine Ein-Minuten-Last von 4,19 und am Ende von 1,00 aus, während Abschnitt 4.3 einen Richtwert unter 0,3 nennt. Weil beide Varianten im selben Prozess und im selben Zeitfenster gemessen werden und weil Verhältnisse statt absoluter Zeiten berichtet werden, sind die Verhältnisse davon nur schwach betroffen. Die absoluten Zeiten dürfen jedoch nicht als Plattformkennwerte gelesen werden.

Die Auflösungsgrenze beruht auf einem geänderten Verfahren. Abschnitt 4.6.2 sieht vor, die Grenze aus der Verstärkungsstufe abzuleiten, weil beide Seiten für sie praktisch denselben Code erzeugen sollten. Die Messung widerlegt diese Annahme: Die Verstärkungsstufe weicht bei kleinen Blockgrößen um bis zu 6,8 Prozent ab. Nach dem ursprünglichen

Verfahren läge die Grenze folglich bei 6,8 Prozent und würde jeden Befund verschlucken, der kleiner ist als der eigene Bezugsfall. Der verwendete Nullvergleich kommt ohne Annahme über Codegleichheit aus und erfasst zusätzlich die Drift während des Laufs. Die Abweichung vom Versuchsdesign ist damit begründet, sie bleibt aber eine nachträgliche Anpassung des Auswertungsverfahrens an die Daten.

Der Signifikanztest ist wirkungslos. Bei 31 der 33 Messpunkte liegt der p-Wert am unteren Anschlag der Gleitkommadarstellung. Bei 300 Wiederholungen je Messpunkt wird jede noch so kleine Verschiebung signifikant — ein Fall, den Abschnitt 4.6.2 vorwegnimmt. Der Test trennt hier nichts; tragend sind Verhältnis, Konfidenzintervall und Auflösungsgrenze. Die schmalen Konfidenzintervalle belegen zudem nur eine gut wiederholbare Messung und nicht, dass das Verhältnis den wahren Leistungsunterschied trifft.

Eine Messartefakt-Quelle wurde nachträglich entdeckt. Die Lage der Puffer im Speicher veränderte die gemessene Zeit der Verstärkungsstufe um den Faktor 2,03, bevor sie durch eine feste Pufferlage stabilisiert wurde (Abschnitt 5.5). Dass ein Effekt dieser Größenordnung überhaupt auftreten konnte, mahnt zur Vorsicht: Es ist nicht auszuschließen, dass weitere Artefakte gleicher Art unentdeckt geblieben sind, insbesondere bei den Bausteinen mit den kleinsten absoluten Zeiten.

Die Deutung des FFT-Befunds ist nicht belegt. Die Erklärung über den Befehls-Zwischenspeicher stützt sich auf die gemessene Codegröße und nicht auf Hardware-Zähler. Sie ist plausibel und passt zu den übrigen Beobachtungen, bleibt aber eine Vermutung.

7.3.2 Konstruktvalidität

Codezeilen messen nicht Implementierungskomplexität. Die Teilfrage nach Codelänge *und* Implementierungskomplexität wird nur für den ersten Teil quantitativ beantwortet. Eine Zeile Faust setzt Kenntnis der Blockdiagrammalgebra und der Bibliotheksfunktionen voraus, eine Zeile C++ nicht. Die in Abschnitt 7.1.5 berichteten Beobachtungen sind Erfahrungen einer einzelnen Person und ersetzen keine Untersuchung des Entwicklungsaufwands.

„Naheliegende Formulierung“ ist kein scharfes Konstrukt. Der Vergleich beruht darauf, dass jede Seite in ihrer naheliegenden Form formuliert wird. Was naheliegend ist, hängt von der Erfahrung der schreibenden Person ab. Der FIR-Befund zeigt, dass innerhalb dieses Spielraums Unterschiede von mehr als Faktor zwei liegen können.

Das Verhältnis verbirgt die Größenordnung. Ein Verhältnis von 1,07 bei der Verstärkungsstufe entspricht einem Unterschied von 0,2 Nanosekunden je Block. Für die Echtzeitfähigkeit ist das bedeutungslos, im Verhältnis aber sichtbar. Verhältnisse sollten deshalb stets zusammen mit den absoluten Zeiten aus Kapitel 6 gelesen werden.

7.3.3 Externe Validität

Die Ergebnisse gelten für fünf lineare, zeitinvariante Bausteine mit konstanten Koeffizienten, auf einer Plattform, mit einem Compiler und in einer Faust-Version. Nichtlineare Verarbeitung, zeitvariable Parameter, Mehrkanalverarbeitung und Bausteine mit großen Verzögerungsspeichern wurden nicht untersucht.

Besonders einzuschränken ist die Übertragbarkeit der Laufzeitbefunde. Da sich die Compilerkonfiguration als der stärkste Einflussfaktor erwiesen hat und die Hauptmessung mit einem einzigen, noch dazu aggressiven Flagsatz gefahren wurde, sind die berichteten Prozentwerte nicht als Kennzahlen für Faust zu verstehen. Übertragbar ist dagegen der strukturelle Befund: dass der Faust-Compiler Faltung und Transformation vollständig ausrollt, mit den daraus folgenden Konsequenzen für Binärgröße und Übersetzungsdauer. Diese Eigenschaft ist vom Flagsatz unabhängig und in jeder untersuchten Konfiguration sichtbar.

Kapitel 8

Zusammenfassung und Ausblick

8.1 Zusammenfassung der zentralen Ergebnisse

8.2 Ausblick auf künftige Forschungsarbeiten

Quellenverzeichnis

Literatur

- Buttazzo, G. C. (1997). *Hard real-time computing systems: predictable scheduling algorithms and applications*. Springer. (Siehe S. 28).
- Cooley, J. W., & Tukey, J. W. (1965). An Algorithm for the Machine Calculation of Complex Fourier Series. *Mathematics of Computation*, 19(90), 297–301. <https://doi.org/10.1090/S0025-5718-1965-0178586-1> (siehe S. 25).
- Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall. (Siehe S. 31).
- Google LLC. (2023). *Google Benchmark: A microbenchmark support library* [Library for microbenchmarking C++ code]. <https://github.com/google/benchmark> (siehe S. 27).
- Hennessy, J. L., Patterson, D. A., & Kozyrakis, C. (2025). *Computer architecture: a quantitative approach*. Morgan kaufmann. (Siehe S. 44).
- Ifeachor, E. C., & Jervis, B. W. (2002). *Digital signal processing: a practical approach*. Pearson Education. (Siehe S. 1).
- Letz, S., Orlarey, Y., & Foer, D. (2010). Work Stealing Scheduler for Automatic Parallelization in Faust. In LAC (Hrsg.), *Proceedings of Linux Audio Conference*. <https://hal.science/hal-02158924> (siehe S. 2, 17, 29).
- Levine, N., & Skolnick, D. (1997). DSP 101 Part 3: Implement Algorithms on a Hardware Platform. *Analog Dialogue*, 31(3). Verfügbar 4. August 2026 unter <https://www.analog.com/en/resources/analog-dialogue/articles/dsp-101-part-3.html> (siehe S. 1).
- Lyons, R. G., et al. (2004). *Understanding digital signal processing* (Bd. 2). Prentice Hall PTR Upper Saddle River, NJ, USA: (Siehe S. 9, 25, 26, 39).
- Mann, H. B., & Whitney, D. R. (1947). On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics*, 18(1), 50–60. <https://doi.org/10.1214/aoms/1177730491> (siehe S. 31).
- Michon, R., & Smith, J. O. (2011). FAUST-STK: A Set of Linear and Nonlinear Physical Models for the FAUST Programming Language. *Proceedings of the 14th International Conference on Digital Audio Effects (DAFx-11)* (siehe S. 1, 2, 12, 34, 59).
- Mytkowicz, T., Diwan, A., Hauswirth, M., & Sweeney, P. F. (2009). Producing wrong data without doing anything obviously wrong! *ACM Sigplan Notices*, 44(3), 265–276 (siehe S. 27).

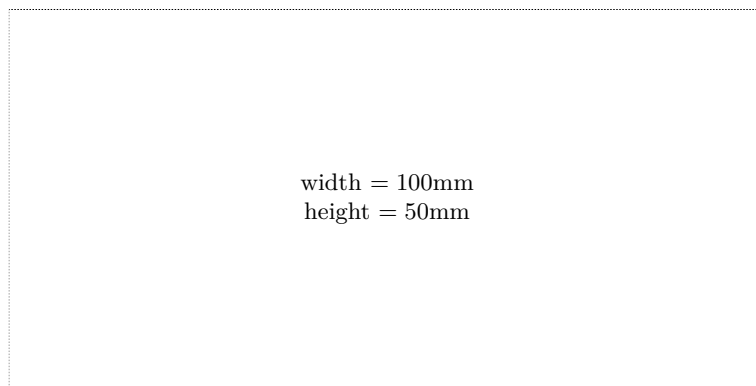
- Orlarey, Y., Fober, D., & Letz, S. (2009). FAUST : an Efficient Functional Approach to DSP Programming. In E. D. FRANCE (Hrsg.), *NEW COMPUTATIONAL PARADIGMS FOR COMPUTER MUSIC* (S. 65–96). <https://hal.science/hal-02159014> (siehe S. 1, 2, 7, 10, 12–18, 34, 59).
- Parhi, K. K. (1999). *VLSI Digital Signal Processing Systems: Design and Implementation*. John Wiley & Sons. (Siehe S. 33, 58).
- Rossing, T. (2007). *Springer Handbook of Acoustics*. Springer New York. https://books.google.at/books?id=4ktVwGe_dSMC (siehe S. 4).
- Scaringella, N., Orlarey, Y., Letz, S., & Fober, D. (2003). Automatic vectorization in Faust. In JIM (Hrsg.), *Actes des Journées d'Informatique Musicale JIM2003*. <https://hal.science/hal-02158949> (siehe S. 2, 7, 8, 17, 19, 29, 33, 42, 58).
- Smith, J. O. (2007). *Introduction to digital filters: with audio applications* (Bd. 2). Julius Smith. (Siehe S. 6–8, 39).
- Tan, L., & Jiang, J. (2018). *Digital Signal Processing: Fundamentals and Applications*. Academic Press. <https://books.google.at/books?id=MxlxDwAAQBAJ> (siehe S. 4–6, 9, 10).
- Using the GNU Compiler Collection (GCC), Version 13.3.0* [Abschnitt „Options That Control Optimization“]. (2024). Free Software Foundation. <https://gcc.gnu.org/onlinedocs/gcc-13.3.0/gcc/Optimize-Options.html> (siehe S. 58).
- Zölzer, U. (2022). *Digital audio signal processing*. John Wiley & Sons. (Siehe S. 4, 6, 7).

Online-Quellen

- GRAME - Centre National de Création Musicale. (2025). *Faust Manual*. Verfügbar 20. Oktober 2025 unter <https://faustdoc.grame.fr/manual/introduction/> (siehe S. 1, 17).

Check Final Print Size

— Check final print size! —



— Remove this page after printing! —